

УНИВЕРСАЛЬНОЕ ПОСОБИЕ ПО АЛГОРИТМАМ И СТРУКТУРАМ
ПОЛНЫЙ ИСХОДНЫЙ КОД ПРОЕКТА (ВСЕ ФАЙЛЫ, ВСЕ

=====

Дата генерации: 11.09.2026, 18:14:07
Файлов исходного кода: 81
Суммарный объём кода: 1.11 МВ
Глав/билетов в пособии: 30

=====

ОГЛАВЛЕНИЕ: ГЛАВЫ И БИЛЕТЫ

=====

1. 1. Дерево отрезков с операциями на отрезках [id: seg-tree]
2. 2. Двумерная разреженная матрица (Sparse Table)
3. 3. Декартово дерево (Treap) [id: treap]
4. 4. Splay-дерево [id: splay-tree]
5. 5. Двумерная матрица префиксных сумм [id: pref-sum]
6. Планарность: K5, K3,3 и формула Эйлера [id: planarity]
7. 6. Графы. Представление, DFS, BFS [id: graph-dfs-bfs]
8. 7. Графы. Планарные. Покраска [id: graph-planarity-coloring]
9. 8. Графы. Компоненты связности [id: graph-components]
10. 9. Графы. Топологическая сортировка [id: topological-sort]
11. 10. Графы. Компоненты сильной связности (SCC)
12. 11. Графы. Мосты [id: graph-bridges]
13. 12. Графы. Точки сочленения [id: graph-articulation-points]
14. 13. Графы. Эйлеров цикл [id: graph-euler]
15. Билет 14. Дейкстра [id: dijkstra]
16. Билет 15. Форд-Беллман [id: bellman-ford]
17. Билет 16. Флойд-Уоршелл [id: floyd]
18. 17. Графы. Ускорения поиска [id: pathfinding-accelerations]
19. 17. Графы. Алгоритм Джонсона (Фокус с перевзвешиванием)
20. 18. Графы. Остовное дерево. Краскал [id: mst-kruskal]
21. 19. Графы. Остовное дерево. Прима [id: mst-prims]
22. 20. Графы. Остовное дерево. Борувка [id: mst-boruvka]
23. 21. Алгоритм Кнута-Морриса-Патта (KMP) [id: kmp]
24. 22. Z-функция [id: string-z-func]
25. Билет 23. Алгоритм Ахо-Корасик [id: aho-corasick]

Билеты (учебный контент)

- 23. src/data/tickets/ticket1_5.ts
- 24. src/data/tickets/ticket14_20.ts
- 25. src/data/tickets/ticket21_24.ts
- 26. src/data/tickets/ticket6_13.ts

Интерактивные компоненты и симуляторы

- 27. src/components/ArticulationPointsViz.tsx
- 28. src/components/BellmanFordViz.tsx
- 29. src/components/BfsViz.tsx
- 30. src/components/ChapterImage.tsx
- 31. src/components/ChapterNav.tsx
- 32. src/components/ChapterView.tsx
- 33. src/components/DfsBridgesSimulator.tsx
- 34. src/components/DijkstraViz.tsx
- 35. src/components/DPVisualizer.tsx
- 36. src/components/EulerSimulator.tsx
- 37. src/components/ExpressQuiz.tsx
- 38. src/components/FloydViz.tsx
- 39. src/components/GraphTraversalViz.tsx
- 40. src/components/HeapViz.tsx
- 41. src/components/hints/demoKit.tsx
- 42. src/components/hints/demosExtra.tsx
- 43. src/components/hints/demosGraph.tsx
- 44. src/components/hints/demosStruct.tsx
- 45. src/components/hints/MiniDemo.tsx
- 46. src/components/hints/SimulatorModal.tsx
- 47. src/components/hints/TermPopover.tsx
- 48. src/components/JohnsonViz.tsx
- 49. src/components/KosarajuViz.tsx
- 50. src/components/KruskalSimulator.tsx
- 51. src/components/MnemonicCards.tsx
- 52. src/components/MobileToc.tsx
- 53. src/components/Navbar.tsx
- 54. src/components/PlanarityDemo.tsx
- 55. src/components/QueueViz.tsx
- 56. src/components/SalmonAutomatonWidget.tsx
- 57. src/components/SegmentTreeVisualizer.tsx

ФАЙЛ 1/81: add.md

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 1224 | РАЗМЕР:

Изменённые файлы

```text

```
A .github/workflows/deploy-pages.yml
M .github/workflows/rebuild-pdf.yml
M index.html
M public/export/full_code_all_pages.pdf
M public/export/full_code_all_pages.txt
A public/favicon.svg
M src/App.tsx
M src/components/Navbar.tsx
A src/components/PythonCompiler.tsx
M vite.config.ts
```

```

`github/workflows/deploy-pages.yml`

Полное содержимое после изменений

```yaml

name: Deploy app to GitHub Pages

on:

push:

branches:

- main

# Позволяет проверить Pages прямо из рабочей ветки

- "arena/\*\*"

workflow\_dispatch:

permissions:

```
deploy:
 name: Publish to GitHub Pages
 needs: build
 runs-on: ubuntu-latest
 environment:
 name: github-pages
 url: ${ steps.deployment.outputs.page_url }
 steps:
 - name: Deploy
 id: deployment
 uses: actions/deploy-pages@v5
....

Patch относительно `main`

```diff
diff --git a/.github/workflows/deploy-pages.yml b/.github/workflows/deploy-pages.yml
new file mode 100644
index 00000000..17418e3
--- /dev/null
+++ b/.github/workflows/deploy-pages.yml
@@ -0,0 +1,60 @@
+name: Deploy app to GitHub Pages
+
+on:
+  push:
+    branches:
+      - main
+      # Позволяет проверить Pages прямо из рабочей ветки
+      - "arena/**"
+  workflow_dispatch:
+
+permissions:
+  contents: read
+  pages: write
+  id-token: write
+
+concurrency:
```

Универсальное пособие • полный исходный код

```
-----
+       name: github-pages
+       url: ${ steps.deployment.outputs.page_url }
+     steps:
+       - name: Deploy
+         id: deployment
+         uses: actions/deploy-pages@v5
+     ....

## `github/workflows/rebuild-pdf.yml`

### Полное содержимое после изменений

```yaml
name: Rebuild code PDF

Пайплайн пересборки PDF при каждом коммите:
#
исходный код → full_code_all_pages.txt (script)
|
|→ page-0001.png ... page-NNNN.png (
|
|→ full_code_all_page
#
Готовые TXT и PDF коммитаются обратно в ветку (их отда
промежуточные PNG сохраняются как artifacts запуска.

on:
 push:
 branches:
 - "**"
 paths-ignore:
 # чтобы коммит самого workflow не запускал бескон
 - "public/export/**"
 - "**/*.md"
 workflow_dispatch:
 inputs:
 density:
 description: "DPI растеризации страниц (по умол
```

- ```
        sudo sed -i 's/rights="none" pattern="\{P
    fi
done
convert -version
```
- name: Install dependencies
run: npm ci --no-audit --no-fund
 - name: Step 1 – код → txt
run: npm run export:txt
 - name: Step 2 – txt → png
env:
 EXPORT_DENSITY: \${github.event.inputs.density}
run: npm run export:png
 - name: Step 3 – png → pdf
run: npm run export:pdf
 - name: Type-check приложения
run: npx tsc --noEmit
continue-on-error: true
 - name: Summary
run: |
 {
 echo "### Экспорт пересобран"
 echo ""
 echo "| Артефакт | Размер |"
 echo "| --- | --- |"
 echo "| \public/export/full_code_all_pages
 echo "| \public/export/full_code_all_pages
 echo "| PNG-страниц | \${ls tmp/export-pages.
 } >> "\$GITHUB_STEP_SUMMARY"
 - name: Upload artifacts (PDF + TXT)
uses: actions/upload-artifact@v7
with:

```
- name: Checkout
- uses: actions/checkout@v4
+ uses: actions/checkout@v6
  with:
    fetch-depth: 0
    persist-credentials: true

- name: Setup Node.js
- uses: actions/setup-node@v4
+ uses: actions/setup-node@v7
  with:
-   node-version: "22"
+   node-version: "24"
    cache: npm

- name: Install ImageMagick + DejaVu fonts
@@ -97,7 +97,7 @@ jobs:
    } >> "$GITHUB_STEP_SUMMARY"

- name: Upload artifacts (PDF + TXT)
- uses: actions/upload-artifact@v4
+ uses: actions/upload-artifact@v7
  with:
    name: full-code-export
    path: |
@@ -107,7 +107,7 @@ jobs:
    retention-days: 30

- name: Upload artifacts (PNG-страницы)
- uses: actions/upload-artifact@v4
+ uses: actions/upload-artifact@v7
  with:
    name: full-code-pages-png
    path: tmp/export-pages/*.png
....

## `index.html`
```

````xml

```
<svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 64 64"
 <rect width="64" height="64" rx="16" fill="#4f46e5"/>
 <path d="M18 17h28v8H26v8h16v8H26v8h20v8H18z" fill="#4f46e5"/>
 <circle cx="47" cy="21" r="4" fill="#34d399"/>
</svg>
```

````

Patch относительно `main`

````diff

```
diff --git a/public/favicon.svg b/public/favicon.svg
new file mode 100644
index 00000000..3bbbba0
--- /dev/null
+++ b/public/favicon.svg
@@ -0,0 +1,5 @@
+<svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 64 64"
+ <rect width="64" height="64" rx="16" fill="#4f46e5"/>
+ <path d="M18 17h28v8H26v8h16v8H26v8h20v8H18z" fill="#4f46e5"/>
+ <circle cx="47" cy="21" r="4" fill="#34d399"/>
+</svg>
```

````

`src/App.tsx`

Полное содержимое после изменений

````tsx

```
import { useCallback, useEffect, useMemo, useState } from 'react';
import { chapters } from '../data/content';
import { Navbar } from '../components/Navbar';
import { Sidebar } from '../components/Sidebar';
import { MobileToc } from '../components/MobileToc';
import { ChapterView } from '../components/ChapterView';
import { ChapterNav } from '../components/ChapterNav';
import { SimulatorModal } from '../components/hints/SimulatorModal';
```



```
<Navbar activeTab={activeTab} setActiveTab={setActiveTab} />
```

```
{activeTab === "guide" ? (
```

```
 <>
```

```
 <MobileToc
```

```
 selectedChapterId={activeChapterId}
```

```
 setSelectedChapterId={goTo}
```

```
 currentTitle={activeChapter.title}
```

```
 index={index}
```

```
 total={chapters.length}
```

```
 />
```

```
 <div className="flex flex-col lg:flex-row max-w-screen-xl" />
```

```
 /* Сайдбар только на десктопе — на мобильном скрывается */
```

```
 <div className="hidden lg:block lg:w-80 shrink-0" />
```

```
 <Sidebar selectedChapterId={activeChapterId} />
```

```
 </div>
```

```
 <main id="main-content" className="flex-1 min-w-0" />
```

```
 /* Шапка главы с быстрыми переходами */
```

```
 <div className="flex items-center justify-between" />
```

```
 <div className="min-w-0" />
```

```
 <div className="text-[10px] uppercase" />
```

```
 {activeChapter.category || "Универсальное пособие"}
```

```
 </div>
```

```
 <h2 className="text-base sm:text-xl font-medium" />
```

```
 {activeChapter.title}
```

```
 </h2>
```

```
 </div>
```

```
 <ChapterNav prev={prev} next={next} index={index} />
```

```
 </div>
```

```
 <div className="h-1 w-full bg-slate-800 rounded" />
```

```
 <div
```

```
 className="h-full bg-gradient-to-r from-slate-800 to-slate-700" />
```

```
 style={{ width: `${progress}%` }}
```

```
 />
```

```
 </div>
```

-----  
 ### Patch относительно `main`

```diff

diff --git a/src/App.tsx b/src/App.tsx

index 2091f33..4583756 100644

--- a/src/App.tsx

+++ b/src/App.tsx

@@ -7,9 +7,10 @@ import { ChapterView } from "../components/ChapterView";

import { ChapterNav } from "../components/ChapterNav";

import { SimulatorModal } from "../components/hints/SimulatorModal";

import { SimulatorHub } from "../components/SimulatorHub";

+import { PythonCompiler } from "../components/PythonCompiler";

export default function App() {

- const [activeTab, setActiveTab] = useState<"guide" | "simulator">("guide");

+ const [activeTab, setActiveTab] = useState<"guide" | "simulator">("guide");

const [activeChapterId, setActiveChapterId] = useState<string>("");

const [modalVizId, setModalVizId] = useState<string>("");

@@ -95,7 +96,7 @@ export default function App() {

</main>

</div>

</>

-) : (

+) : activeTab === "simulator" ? (

<main className="px-4 sm:px-6 lg:px-10 py-6 lg:py-8">

<SimulatorHub

onOpen={setModalVizId}

@@ -105,6 +106,8 @@ export default function App() {

</>

</>

</main>

+) : (

+ <PythonCompiler />

)}

<SimulatorModal vizId={modalVizId} onClose={() => setModalVizId("")} />

```

```
 <h1 className="text-xl font-extrabold text-slate-400">
 Универсальное пособие
 </h1>
 <p className="text-xs text-indigo-400 font-extrabold">
 Подготовка к экзаменам: Алгоритмы, Структуры
 </p>
 </div>
 </div>

 <div className="flex items-center w-full lg:w-auto" >
 >
 <button
 id="tab-guide-btn"
 onClick={() => setActiveTab('guide')}
 className={`flex-1 lg:flex-none flex items-center justify-center
 duration-200 whitespace-nowrap ${
 activeTab === 'guide'
 ? 'bg-indigo-600 text-white shadow-lg shadow-indigo-500'
 : 'text-slate-400 hover:text-white'
 }`}
 >
 <BookOpen className="w-4 h-4" /> Учебное пособие
 </button>
 <button
 id="tab-simulator-btn"
 onClick={() => setActiveTab('simulator')}
 className={`flex-1 lg:flex-none flex items-center justify-center
 duration-200 whitespace-nowrap ${
 activeTab === 'simulator'
 ? 'bg-indigo-600 text-white shadow-lg shadow-indigo-500'
 : 'text-slate-400 hover:text-white'
 }`}
 >
 <Cpu className="w-4 h-4" /> Тренажёры
 </button>
 <button
 id="tab-compiler-btn"
 onClick={() => setActiveTab('compiler')}
 >
```

```

er:bg-amber-600/20 border border-amber-500/
title="Скачать всё пособие одним автономным
>
 {busy ? <Loader2 className="w-4 h-4 animate
</button>
</div>
</div>
</header>
);
};
....
```

### Patch относительно `main`

```
````diff
diff --git a/src/components/Navbar.tsx b/src/components.
index a528778..6f8cd36 100644
--- a/src/components/Navbar.tsx
+++ b/src/components/Navbar.tsx
@@ -1,11 +1,11 @@
  import React, { useState } from 'react';
- import { BookOpen, Cpu, Sparkles, FileDown, FileText,
+ import { BookOpen, Cpu, Sparkles, FileDown, FileText,
  import { chapters } from '../data/content';
  import { downloadBookHtml } from '../utils/exportHtml'

  interface NavbarProps {
-   activeTab: 'guide' | 'simulator';
-   setActiveTab: (tab: 'guide' | 'simulator') => void;
+   activeTab: 'guide' | 'simulator' | 'compiler';
+   setActiveTab: (tab: 'guide' | 'simulator' | 'compile
  }

  export const Navbar: React.FC<NavbarProps> = ({ active
@@ -60,9 +60,20 @@ export const Navbar: React.FC<Navbar
    >
      <Cpu className="w-4 h-4" /> Тренажёры
    </button>
```

Универсальное пособие • полный исходный код

```
-----
const PYODIDE_VERSION = "0.27.7";
const PYODIDE_MODULE_URL = `https://cdn.jsdelivr.net/pyo
const PYODIDE_INDEX_URL = `https://cdn.jsdelivr.net/pyo

const STARTER_CODE = `# Первый запуск загрузит Python в
name = "алгоритмы"

for number in range(1, 4):
    print(f"{number}. Учим {name}!")
`;

type PyodideRuntime = {
  runPythonAsync: (source: string) => Promise<unknown>;
  setStdout: (options: { batched: (message: string) =>
  setStderr: (options: { batched: (message: string) =>
  setStdin: (options: { stdin: () => string | number |
};

type PyodideModule = {
  loadPyodide: (options: { indexURL: string }) => Promi
};

let runtimePromise: Promise<PyodideRuntime> | null = null;

function getPythonRuntime() {
  if (!runtimePromise) {
    runtimePromise = import(/* @vite-ignore */ PYODIDE_I
      (module as PyodideModule).loadPyodide({ indexURL:
    });
  }
  return runtimePromise;
}

function errorText(error: unknown) {
  if (error instanceof Error) return error.message;
  return String(error);
}
```

```

    }, [code, stdin, running, runtimeReady]);

const reset = () => {
  setCode(STARTER_CODE);
  setStdin("");
  setOutput("Нажмите «Запустить», чтобы увидеть резул");
};

return (
  <main className="max-w-screen-2xl mx-auto px-4 sm:px-6">
    <div className="max-w-6xl mx-auto">
      <div className="flex flex-col sm:flex-row sm:items-center">
        <div>
          <div className="inline-flex items-center gap-2">
            <Terminal className="w-4 h-4" /> Боковая панель
          </div>
          <h2 className="text-2xl sm:text-3xl font-extrabold">
            <p className="text-slate-400 mt-2 max-w-2xl">
              Пишите небольшой код и запускайте его прямо в браузере,
              который переводит CPython в WebAssembly.
            </p>
          </div>
          <a
            href="https://pyodide.org/"
            target="_blank"
            rel="noreferrer"
            className="inline-flex items-center gap-1.5"
          >
            Как это работает <ExternalLink className="w-4 h-4" />
          </a>
        </div>

        <div className="grid xl:grid-cols-[minmax(0,1.33fr),minmax(0,1.33fr)]">
          <section className="rounded-2xl border border-slate-200 p-4">
            <div className="flex flex-wrap items-center gap-2">
              <div className="flex items-center gap-2">
                <span className="w-2.5 h-2.5 rounded-full bg-slate-200">
                <span className="text-sm font-bold text-slate-700">

```

```

        />

        <div className="border-t border-slate-800 p-4">
          <label className="block px-4 pt-3 text-slate-700">
            Ввод для input() • по одной строке на к
          </label>
          <textarea
            id="python-stdin"
            value={stdin}
            onChange={(event) => setStdin(event.target.value)}
            spellCheck={false}
            rows={2}
            placeholder={'Например:\nАлиса'}
            className="block w-full resize-y bg-transparent border:1px solid #ccc"
          />
        </div>
      </section>

      <section className="rounded-2xl border border-slate-800 p-4">
        <div className="flex items-center justify-between">
          <div className="flex items-center gap-2">
            <Terminal className="w-4 h-4 text-emerald-500" />
          </div>
          <span className={`inline-flex items-center gap-2`} >
            {runtimeReady && <CheckCircle2 className="w-4 h-4 text-emerald-500" />}
            {runtimeReady ? "готов" : "не загружен"}
          </span>
        </div>
        <pre className="min-h-[360px] max-h-[560px] p-4 font-mono text-slate-300">
          {output}
        </pre>
        <div className="border-t border-slate-800 p-4">
      </section>
    </div>

    <div className="mt-5 grid md:grid-cols-3 gap-3">

```

```

+for number in range(1, 4):
+    print(f"{number}. Учим {name}!")
+`;
+
+type PyodideRuntime = {
+  runPythonAsync: (source: string) => Promise<unknown>
+  setStdout: (options: { batched: (message: string) =>
+  setStderr: (options: { batched: (message: string) =>
+  setStdin: (options: { stdin: () => string | number |
+});
+
+type PyodideModule = {
+  loadPyodide: (options: { indexURL: string }) => Prom
+});
+
+let runtimePromise: Promise<PyodideRuntime> | null = n
+
+function getPythonRuntime() {
+  if (!runtimePromise) {
+    runtimePromise = import(/* @vite-ignore */ PYODIDE
+      (module as PyodideModule).loadPyodide({ indexURL
+    });
+  }
+  return runtimePromise;
+}
+
+function errorText(error: unknown) {
+  if (error instanceof Error) return error.message;
+  return String(error);
+}
+
+export function PythonCompiler() {
+  const [code, setCode] = useState(STARTER_CODE);
+  const [stdin, setStdin] = useState("");
+  const [output, setOutput] = useState("Нажмите «Запус
+  const [running, setRunning] = useState(false);
+  const [runtimeReady, setRuntimeReady] = useState(fal
+  const [runtimeMessage, setRuntimeMessage] = useState

```



```

+
+ return (
+   <main className="max-w-screen-2xl mx-auto px-4 sm:px-6">
+     <div className="max-w-6xl mx-auto">
+       <div className="flex flex-col sm:flex-row sm:items-center">
+         <div>
+           <div className="inline-flex items-center gap-2">
+             <Terminal className="w-4 h-4" /> Боковая панель
+           </div>
+           <h2 className="text-2xl sm:text-3xl font-extrabold">
+             <p className="text-slate-400 mt-2 max-w-2xl">
+               Пишите небольшой код и запускайте его прямо в браузере,
+               который переводит CPython в WebAssembly.
+             </p>
+           </div>
+           <a
+             href="https://pyodide.org/"
+             target="_blank"
+             rel="noreferrer"
+             className="inline-flex items-center gap-1.5"
+           >
+             Как это работает <ExternalLink className="text-slate-400" />
+           </a>
+         </div>
+
+       <div className="grid xl:grid-cols-[minmax(0,1fr),minmax(0,1fr)]">
+         <section className="rounded-2xl border border-slate-200 p-4">
+           <div className="flex flex-wrap items-center gap-2">
+             <div className="flex items-center gap-2">
+               <span className="w-2.5 h-2.5 rounded-full bg-slate-200">
+                 <span className="text-sm font-bold text-slate-400">
+                   <span className="text-[11px] text-slate-400">
+                     </div>
+                 <div className="flex items-center gap-2">
+                   <button
+                     type="button"
+                     onClick={reset}
+                     className="inline-flex items-center gap-1.5"

```

```

+         id="python-stdin"
+         value={stdin}
+         onChange={(event) => setStdin(event.target.value)}
+         spellCheck={false}
+         rows={2}
+         placeholder={'Например:\nАлиса'}
+         className="block w-full resize-y bg-tran
ceholder:text-slate-700"
+       />
+     </div>
+   </section>
+
+   <section className="rounded-2xl border border-slate-200" >
+     <div className="flex items-center justify-between" >
+       <div className="flex items-center gap-2" >
+         <Terminal className="w-4 h-4 text-emerald-500" />
+       </div>
+       <span className={`inline-flex items-center gap-2`} >
+         <CheckCircle2 className="w-4 h-4 text-emerald-500" />
+         {runtimeReady ? "готов" : "не загружен"}
+       </span>
+     </div>
+     <pre className="min-h-[360px] max-h-[560px] leading-6 text-slate-300">
+       {output}
+     </pre>
+     <div className="border-t border-slate-800 p-4" >
+   </section>
+ </div>
+
+ <div className="mt-5 grid md:grid-cols-3 gap-3" >
+   <div className="flex gap-2 rounded-xl border border-slate-200" >
+     <Info className="w-4 h-4 shrink-0 text-indigo-500" />
+     <span>Ctrl или Cmd + Enter запускает код. Ищет файл python.py в папке
+   </div>
+   <div className="flex gap-2 rounded-xl border border-slate-200" >
+     <Info className="w-4 h-4 shrink-0 text-indigo-500" />
+     <span>Первый запуск скачивает примерно нес

```

```

server: {
  host: "0.0.0.0",
  port: 5173,
  strictPort: true,
  // предпросмотр может открываться через внешний про
  allowedHosts: true,
},
preview: {
  host: "0.0.0.0",
  port: 4173,
  strictPort: true,
  allowedHosts: true,
},
});

```

Patch относительно `main`

```

````diff
diff --git a/vite.config.ts b/vite.config.ts
index c0456e7..e19d945 100644
--- a/vite.config.ts
+++ b/vite.config.ts
@@ -10,6 +10,9 @@ const __dirname = path.dirname(__file)

// https://vite.dev/config/
export default defineConfig({
+ // На GitHub Pages приложение живёт в /algv0/, локал
+ // VITE_BASE можно переопределить, если репозиторий
+ base: process.env.VITE_BASE || "/",
 plugins: [react(), tailwindcss(), viteSingleFile()],
 resolve: {
 alias: {

```

Универсальное пособие • полный исходный код

-----  
- "Душные" детали (код, формулы, сложные доказательства)  
4. **\*\*Не ломай верстку:\*\*** Не придумывай свои CSS-классы в файлах `src/data/content.ts`. Не используй чистый markdown.

-----  
ФАЙЛ 4/81: index.html

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 13 | РАЗМЕР: 512 Б

-----  
<!doctype html>  
<html lang="ru" class="dark bg-slate-950 text-slate-100">  
 <head>  
 <meta charset="UTF-8" />  
 <meta name="viewport" content="width=device-width, height=device-height, initial-scale=1.0" />  
 <title> Дискретка для тупых – Интерактивный гид по дискретке  
 </head>  
 <body class="bg-slate-950 text-slate-100 min-h-screen">  
 <div id="root"></div>  
 <script type="module" src="/src/main.tsx"></script>  
 </body>  
</html>

-----  
ФАЙЛ 5/81: metadata.json

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 8 | РАЗМЕР: 312 Б

-----  
{  
 "name": "Remix Remix: ExamPrep Reader",  
 "description": "Удобная web-читалка для подготовки к экзаменам",  
 "requestFramePermissions": [],  
 "majorCapabilities": [  
 "MAJOR\_CAPABILITY\_SERVER\_SIDE\_GEMINI\_API"  
 ]  
}

## Универсальное пособие • полный исходный код

```

 "esbuild": "^0.28.2",
 "tailwindcss": "4.1.17",
 "typescript": "5.9.3",
 "vite": "7.3.2",
 "vite-plugin-singlefile": "2.3.0"
 },
 "description": "Интерактивное пособие по алгоритмам и
}
```

-----  
ФАЙЛ 7/81: README.md

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 115 | РАЗМЕР:

-----  
# algv0 – Универсальное пособие по алгоритмам и структуре

Интерактивная web-читалка (React + Vite + Tailwind) с слайдами  
и автоматическим экспортом всего исходного кода в PDF.

## Быстрый старт

```
```bash  
npm install  
npm run dev          # предпросмотр на http://0.0.0.0:5174  
npm run build        # сборка в dist/ (single-file)  
```
```

## Экспорт кода: код → txt → png → pdf

Пайплайн собирает **\*\*весь\*\*** исходный код репозитория в о

| Команда              | Шаг       | Результат           |
|----------------------|-----------|---------------------|
| ---                  | ---       | ---                 |
| `npm run export:txt` | код → txt | `public/export/ful` |
| `npm run export:png` | txt → png | `tmp/export-pages/` |
| `npm run export:pdf` | png → pdf | `public/export/ful` |
| `npm run export`     | всё сразу | txt + png + pdf     |

## Универсальное пособие • полный исходный код

-----  
подчёркиваются, а по наведению/тапу показывается карта  
мини-анимацией и кнопкой «Показать симулятор» (симуля  
Словарь — `src/data/glossary.ts`, мини-анимации — `src  
\* **Реестр интерактивов** — `src/components/vizRegistry`  
и для панели под главой, и для модалки из подсказки.

### ## Структура

---

src/

|                    |                                   |
|--------------------|-----------------------------------|
| App.tsx            | — оболочка, привязка глав к визу. |
| components/        | — интерактивные симуляторы (Дейк  |
| data/              | — контент пособия (главы/билеты   |
| scripts/export/    | — пайплайн код → txt → png → pdf  |
| public/export/     | — сгенерированные TXT и PDF (обн  |
| .github/workflows/ | — автоматизация                   |

---

---

### # Структура приложения и парсинг элементов

Если вы хотите парсить HTML конспект внешними инструмен  
азуемые теги и классы.

### ## Заголовки

\* Заголовки секций: Используют стандартный тег `

## ` и \* Подзаголовки: Используют тег ``.

### ## Текст и параграфы

\* Обычный текст содержится в тегах `

`.  
\* Блоки "Шиза" (Аналогии) и другая текстовая инфографика  
параграфы `

`.

### ## Математические формулы

В приложении используется MathJax.

\* В самом исходном коде страницы (и внутри тегов) форму  
k).

Универсальное пособие • полный исходный код

## Уровень 1. Нет вообще ничего: ни аналогии, ни 2D

|                    |           |                      |
|--------------------|-----------|----------------------|
| Глава              | id        | Что делать           |
| ---                | ---       | ---                  |
| Алгоритмы на карте | `alg-map` | Страница-заглушка (8 |

---

## Уровень 2. Темы, которых в пособии НЕТ ВООБЩЕ (дыры)

Это самая большая проблема: визуализаторы для них написаны, но соответствующих глав в `src/data/content.ts` нет — и

|               |                                |                             |
|---------------|--------------------------------|-----------------------------|
| Билет         | Тема                           | Статус                      |
| ---           | ---                            | ---                         |
| <b>**1**</b>  | Дерево отрезков (Segment Tree) | Главы нет. К                |
|               | ит вхолостую.                  |                             |
| <b>**7**</b>  | Планарность и раскраска графов | Номерной гла                |
|               | огии <b>**</b> .               |                             |
| <b>**11**</b> | Мосты в графах                 | Номерной главы нет; есть бе |
| <b>**13**</b> | Эйлеровы пути и циклы          | Номерной главы нет; с       |

Дополнительно — «осиротевшие» визуализаторы без глав (к

- \* `stack-dfs` → `StackViz.tsx` — **\*\*Стек и DFS\*\***
- \* `queue-bfs` → `QueueViz.tsx` + `BfsViz.tsx` — **\*\*Очередь и BFS\*\***
- \* `heap-beam-search` → `HeapViz.tsx` — **\*\*Куча и лучевой поиск\*\***
- \* `segment-trees` → `SegmentTreeVisualizer.tsx` — **\*\*Дерево отрезков\*\***
- \* `dynamic-programming` → `DPVisualizer.tsx` — **\*\*Динамическое программирование\*\***

## Уровень 3. Есть 2D, но НЕТ бытовой аналогии («суха

|                                        |                 |             |
|----------------------------------------|-----------------|-------------|
| Глава                                  | id              | Комментарий |
| ---                                    | ---             | ---         |
| Планарность: K5, K3,3 и формула Эйлера | `planarity-e    |             |
|                                        | т «на пальцах». |             |

## Универсальное пособие • полный исходный код

|    |                                      |   |  |       |
|----|--------------------------------------|---|--|-------|
| 18 | 18. Графы. Остовное дерево. Краскал  | 1 |  | -     |
| 19 | 19. Графы. Остовное дерево. Прима    | 1 |  | -     |
| 20 | 20. Графы. Остовное дерево. Борувка  | 1 |  | -     |
| 21 | 21. Префикс-функция ( $\pi$ ), КМП   | 4 |  | -   - |
| 22 | 22. Z-функция                        | 4 |  | -   - |
| 23 | 23. Алгоритм Ахо–Корасик             | 2 |  | -   - |
| 24 | 24. Классы сложности, сведение задач | 4 |  | -   - |
| -  | Обзор: Кратчайшие пути и Графы       | - |  | -   - |
| -  | Алгоритмы на карте                   | - |  | -   - |
| -  | Эйлер: Путь $\neq$ Цикл              | - |  | -   - |
| -  | Мосты в графах: код + визуализация   | 1 |  | -   - |

## ## Рекомендуемый порядок работ

1. **\*\*Вернуть потерянные билеты 1, 7, 11, 13\*\*** и подключить (`segment-trees`, `stack-dfs`, `queue-bfs`, `heap-be...`)  
это 5 готовых компонентов, которые сейчас просто не...
2. **\*\*Дописать аналогии\*\*** в 5 главам из «Уровня 3» (по ш...
3. **\*\*Добавить 2D\*\*** к 4 главам «Уровня 4» – минимум inli...
4. **\*\*Решить судьбу `alg-map`\*\*** – раскрыть или удалить.

ФАЙЛ 9/81: tsconfig.json

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 32 | РАЗМЕР: 6...

```
{
 "compilerOptions": {
 "target": "ES2020",
 "useDefineForClassFields": true,
 "lib": ["ES2020", "DOM", "DOM.Iterable"],
 "module": "ESNext",
 "skipLibCheck": true,
 "types": ["node"],
```



```
// https://vite.dev/config/
export default defineConfig({
 plugins: [react(), tailwindcss(), viteSingleFile()],
 resolve: {
 alias: {
 "@": path.resolve(__dirname, "src"),
 },
 },
 server: {
 host: "0.0.0.0",
 port: 5173,
 strictPort: true,
 // предпросмотр может открываться через внешний про
 allowedHosts: true,
 },
 preview: {
 host: "0.0.0.0",
 port: 4173,
 strictPort: true,
 allowedHosts: true,
 },
});
```

---

## РАЗДЕЛ: ЯДРО ПРИЛОЖЕНИЯ

---

---

ФАЙЛ 11/81: src/App.tsx

КАТЕГОРИЯ: Ядро приложения | СТРОК: 118 | РАЗМЕР: 4.9 КБ

---

```
import { useCallback, useEffect, useMemo, useState } from "react";
import { chapters } from "../data/content";
import { Navbar } from "../components/Navbar";
import { Sidebar } from "../components/Sidebar";
```

```

return (
 <div className="min-h-screen bg-slate-950 text-slate-50">
 <Navbar activeTab={activeTab} setActiveTab={setActiveTab}>
 {activeTab === "guide" ? (
 <>
 <MobileToc
 selectedChapterId={activeChapterId}
 setSelectedChapterId={goTo}
 currentTitle={activeChapter.title}
 index={index}
 total={chapters.length}
 />

 <div className="flex flex-col lg:flex-row max-w-7xl mx-auto" >
 {/* Сайдбар только на десктопе — на мобильном скрывается */}
 <div className="hidden lg:block lg:w-80 shrink-0" >
 <Sidebar selectedChapterId={activeChapterId} />
 </div>

 <main id="main-content" className="flex-1 m-0 p-4" >
 {/* Шапка главы с быстрыми переходами */}
 <div className="flex items-center justify-between" >
 <div className="min-w-0">
 <div className="text-[10px] uppercase" >
 {activeChapter.category || "Универсальное"}
 </div>
 <h2 className="text-base sm:text-xl font-medium" >
 {activeChapter.title}
 </h2>
 </div>
 <ChapterNav prev={prev} next={next} index={index} />
 </div>

 <div className="h-1 w-full bg-slate-800 rounded" >
 <div
 className="h-full bg-gradient-to-r from-slate-800 to-slate-900"
 >

```

---

ФАЙЛ 12/81: src/hooks/useTermHints.ts

КАТЕГОРИЯ: Ядро приложения | СТРОК: 124 | РАЗМЕР: 5.0 KB

---

```
import { useEffect } from "react";
import { GLOSSARY_LABELS } from "../data/glossary";

/**
 * Ограничители, чтобы текст не рябил, но и чтобы подсказки
 *
 * Считаем не «сколько раз встретился термин», а «сколько
 * написание». Иначе в главе про splay первые же «Splay
 * лимит, и слова «Zig-Zag», «вращений», «AVL-дереве» о
 */
const MAX_PER_SURFACE = 2;
const MAX_PER_TERM = 8;

const SKIP_SELECTOR = "code, pre, script, style, a, text";

const escapeRe = (s: string) => s.replace(/[\.*\+\?\^\$\{\}\(\)\|]/g, "\\$&");

/** Одна большая регулярка из всех меток: длинные раньше
function buildRegex(): RegExp {
 const alternation = GLOSSARY_LABELS.map((l) => escapeRe(l));
 // границы «не буква/цифра» – \b не работает с кириллицей
 return new RegExp(`(?<![\\p{L}\\p{N} _-])(${alternation})`);
}

let cachedRegex: RegExp | null = null;
const labelToId = new Map(GLOSSARY_LABELS.map((l) => [l, ...]));

/**
 * Проходит по тексту главы и оборачивает известные термины
 * , чтобы на них
 * повесить всплывающую карточку (как превью ссылок в Википедии)
 */
```

```

 const usedTerm = perTerm.get(id) ?? 0;
 const usedSurface = perSurface.get(surface) ?? 0;
 if (usedTerm >= MAX_PER_TERM || usedSurface >= MAX_SURFACE) {
 perTerm.set(id, usedTerm + 1);
 perSurface.set(surface, usedSurface + 1);
 }

 if (m.index > last) frag.appendChild(document.createElement("span"));
 const span = document.createElement("span");
 span.className = "term-hint";
 span.dataset.termId = id;
 span.tabIndex = 0;
 span.setAttribute("role", "button");
 span.setAttribute("aria-label", `Подсказка: ${m[1]}`);
 span.textContent = m[1];
 frag.appendChild(span);
 last = m.index + m[1].length;
 }

 if (last === 0) continue;
 if (last < text.length) frag.appendChild(document.createElement("span"));
 textNode.parentNode?.replaceChild(frag, textNode);
}

/**
 * Навешивает разметку терминов на контейнер при смене
 *
 * Дополнительно перепроверяет разметку после каждого рендера
 * (или что-то ещё) перезаписал innerHTML главы, подсказки
 * а не пропадают до перезагрузки страницы.
 */
export function useTermHints(ref: React.RefObject<HTMLDivElement>) {
 const run = () => {
 const el = ref.current;
 if (!el) return;
 try {
 annotateTerms(el);
 } catch {
 //
 }
 };
 useEffect(run, [ref]);
}

```

```
 display: none;
 }
 .no-scrollbar {
 scrollbar-width: none;
 }
}

@keyframes pulse-gold {
 0%,
 100% {
 stroke: #eab308;
 stroke-width: 5;
 filter: drop-shadow(0 0 2px rgba(234, 179, 8, 0.5))
 }
 50% {
 stroke: #fef08a;
 stroke-width: 8;
 filter: drop-shadow(0 0 12px rgba(234, 179, 8, 0.9))
 }
}

.animate-pulse-gold {
 animation: pulse-gold 1.5s infinite;
}

@keyframes tocIn {
 from {
 transform: translateX(-100%);
 }
 to {
 transform: translateX(0);
 }
}

@keyframes hintIn {
 from {
 opacity: 0;
 transform: translateY(4px);
 }
}
```

```
 overflow-wrap: anywhere;
}

.chapter-body :where(section, div, p, li) {
 min-width: 0;
}

.chapter-body :where(pre, code) {
 white-space: pre-wrap;
 word-break: break-word;
}

.chapter-body pre {
 overflow-x: auto;
 max-width: 100%;
 font-size: clamp(11px, 2.9vw, 13px);
}

.chapter-body table {
 display: block;
 width: 100%;
 overflow-x: auto;
 border-collapse: collapse;
 font-size: clamp(11px, 3vw, 14px);
}

.chapter-body summary {
 cursor: pointer;
 padding: 0.35rem 0;
}

@media (max-width: 767px) {
 .chapter-body :where(.grid) {
 grid-template-columns: minmax(0, 1fr) !important;
 }
 .chapter-body :where(h2) {
 font-size: 1.25rem !important;
 line-height: 1.3 !important;
 }
}
```

```
 width: 0%;
 }
 to {
 width: 100%;
 }
}
```

---

ФАЙЛ 14/81: src/main.tsx

КАТЕГОРИЯ: Ядро приложения | СТРОК: 11 | РАЗМЕР: 230 В

---

```
import { StrictMode } from "react";
import { createRoot } from "react-dom/client";
import "./index.css";
import App from "./App";

createRoot(document.getElementById("root")!).render(
 <StrictMode>
 <App />
 </StrictMode>
);
```

---

ФАЙЛ 15/81: src/types.ts

КАТЕГОРИЯ: Ядро приложения | СТРОК: 9 | РАЗМЕР: 153 В

---

```
export interface Chapter {
 id: string;
 title: string;
 type: "html" | "markdown";
 content: string;
 description?: string;
 category?: string;
}
```

```

export const additionalTickets: Chapter[] = [
 {
 id: "sparse-table",
 title: "2. Двумерная разреженная матрица (Sparse Table)",
 type: "html",
 description: "Разреженная таблица для идемпотентных",
 category: "Продвинутые структуры",
 content: `
<section id="sparse-table" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-blue-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-blue-300 italic">
 <p class="text-xl font-mono text-white">
 ух отрезков: $\min(ST[L][k], ST[R - 2^k + 1][k])$
 </div>
 <div class="grid grid-cols-1 xl:grid-cols-2">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
 Аналогия 1: Школьные линейки
 </div>
 <p class="text-slate-300 text-sm mb-4">
 зок длины 7. Ты берешь две заготовленные заготовки
 ь отрезок длины 7 (перекрывая друг друга пополам)
 ничего складывать, просто пересекаем куски.
 </div>
 <div class="bg-slate-900 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-rose-500 shadow-md">
 Аналогия 2: Ступени
 </div>
 <p class="text-slate-300 text-sm mb-4">

```



```
 <p class="text-xl font-mono text-white">
метода: Split и Merge.</p>
</div>
<div class="grid grid-cols-1 xl:grid-cols-2">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 l
rder border-emerald-500 shadow-md">
 Аналогия 1: Удар катаной (Sp
 </div>
 <p class="text-slate-300 text-sm mb
по значению X. Все, кто меньше X улетают в.
 </div>
 <div class="bg-slate-900 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 l
border-rose-500 shadow-md">
 Аналогия 2: Слияние капель (M
 </div>
 <p class="text-slate-300 text-sm mb
вого). Мы просто смотрим на корни: у кого Y
как потомок.</p>
 </div>
</div>
<details class="bg-slate-900/40 rounded-lg l
 <summary class="p-4 font-bold text-slate
-b border-slate-700/50">
 Код (Скрыто)
 </summary>
 <div class="p-5 text-sm text-slate-300">
 <pre class="bg-slate-950 p-4 rounde
pair<Node*, Node*> split(Node* t, int x) {
 if (!t) return {nullptr, nullptr};
 if (t->val <= x) {
 auto [L, R] = split(t->right, x);
 t->right = L;
 return {t, R};
 } else {
 auto [L, R] = split(t->left, x);
 t->left = R;
```

```

<line x1="410" y1="200" x2="554"
<line x1="194" y1="280" x2="122"
<line x1="554" y1="280" x2="482"
</g>
<g font-family="monospace" text-anc
 <circle cx="626" cy="40" r="24"
 <text x="626" y="38" font-size=
fill="#94a3b8">8</text>
 <circle cx="338" cy="120" r="24"
 <text x="338" y="118" font-size
" fill="#94a3b8">6</text>
 <circle cx="266" cy="200" r="24"
 <text x="266" y="198" font-size
" fill="#94a3b8">5</text>
 <circle cx="410" cy="200" r="24"
 <text x="410" y="198" font-size
" fill="#94a3b8">5</text>
 <circle cx="194" cy="280" r="24"
 <text x="194" y="278" font-size
" fill="#94a3b8">4</text>
 <circle cx="554" cy="280" r="24"
 <text x="554" y="278" font-size
" fill="#94a3b8">3</text>
 <circle cx="122" cy="360" r="24"
 <text x="122" y="358" font-size
" fill="#94a3b8">-4</text>
 <circle cx="482" cy="360" r="24"
 <text x="482" y="358" font-size
" fill="#94a3b8">2</text>
 </g>
</svg>
</div>
<div class="img-caption">Та же точечная кар
ая рёбрами. Без рёбер «где стоит» не отличи
по приоритету (см. граблю 2).</div>
</div>

<div class="bg-slate-700/50 p-6 rounded-xl bord

```

```

<div class="absolute -top-3 left-4 l
border-rose-500 shadow-md"> Грабля 4 • а
 <p class="text-slate-300 text-sm mb
дто дерево полное и лежит в массиве по слоям
 <p class="text-slate-300 text-sm mb
это парковка аэропорта: все ряды полные, м
разметку по туману бесполезно – детей нахо
 <p class="text-slate-400 text-xs"><
во отрезков). У treap из 8 узлов глубина 4 -
</div>
<div class="bg-slate-800 p-6 rounded-lg
 <div class="absolute -top-3 left-4 l
er border-amber-500 shadow-md"> Грабля 5 •
 <p class="text-slate-300 text-sm mb
rt, итоговая $O(n)$ (формально $O(2n)$)» – три н
 <p class="text-slate-300 text-sm mb
о разобрать весь гардероб, тронув одну полк
ационная слабость; $O(n)$ у сортировки бывает
формально $2n$ » – как «опоздал всего на два ч
 <p class="text-slate-400 text-xs"><
n вставок по высоте $\Rightarrow$ в среднем $O(n \log n)$,
сборки – но после той же сортировки, которая
 </div>
</div>

<details class="bg-slate-900/40 rounded-lg l
 <summary class="p-4 font-bold text-slate
-b border-slate-700/50">
 Музей: фрагменты сломанного решен
 </summary>
 <div class="p-5 text-sm text-slate-300
 <pre class="bg-slate-950 p-4 rounded
a.binsec(a.x) # «сортировка бинарным
tree[k*2**layout + 1] = a.(x+1) # адресация по слоям н
if a.(x+1) > a.x: ... # сравнение с сосед
 $O(n) + O(\log n) = O(n)$ # «формально $2n$ »: нал
 <p>Шаг «a.binsec(a.x)» – фирменный:
й поиск ничего не сортирует и quicksort не

```

```

 if spine:
 spine[-1].right = node
 spine.append(node)
root = spine[0]</pre>
 <p><b class="text-white">Сложность:
 n в среднем $\Rightarrow$ <b class="text-white"> $O(n \log n)$
 й сборке после сортировки по ключу хватает
 строили одно и то же дерево на схеме выше.</p>
 </div>
 </details>
 </div>
</div>
</section>`,
 },
 {
 id: "splay-tree",
 title: "4. Splay-дерево",
 type: "html",
 description: "Дерево поиска, которое чинит само себя",
 category: "Продвинутые структуры",
 content: `
<section id="splay-tree" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>

 <div class="space-y-8">

 <!-- 0. Определение -->
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-blue-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-blue-300 italic">
 <p class="text-lg sm:text-xl font-mono">
 поиска без
 : серия поворотов, которая поднимает v в корень
 <p class="text-sm text-slate-400 text-c

```

```

50</pre>
<p class="text-slate-300 text-sm mt-4">Есть
<ul class="list-disc list-inside text-slate-300">
 Есть
 каждой вставки чинят дерево, чтобы оно
 /li>
 Есть
 менно быть кривым. Но каждый раз, когда мы
 ровнее становится.

</div>

<!-- 2. Аналогии -->
<div class="grid grid-cols-1 xl:grid-cols-2 gap-4">
 <div class="bg-slate-800 p-6 rounded-lg border border-emerald-500 shadow-md">
 <div class="absolute -top-3 left-4 bg-slate-800 border border-emerald-500 shadow-md">
 Аналогия 1: стопка бумаг на столе
 </div>
 <p class="text-slate-300 text-sm">У тебя
 скиваешь его и, поработав, кладёшь Есть
 неделю самые нужные бумаги сами собой оказыва
 ет ровно это: «взял → положил наверх».</p>
 </div>
 <div class="bg-slate-900 p-6 rounded-lg border border-rose-500 shadow-md">
 <div class="absolute -top-3 left-4 bg-slate-900 border border-rose-500 shadow-md">
 Аналогия 2: тропинка через газон
 </div>
 <p class="text-slate-300 text-sm">Ты идёшь
 ода дорожка сама укорачивалась вдвое: чем чаще
 оплачивает все следующие. Пойдёшь один раз
 </div>
</div>

<!-- 3. Поворот -->
<div class="bg-slate-800/60 p-6 rounded-xl border border-emerald-500 shadow-md">
 <h3 class="text-lg font-bold text-white mb-1">

```

```

 <th class="py-2 pr-3 font-b
 <th class="py-2 font-bold">
 </tr>
 </thead>
 <tbody class="text-slate-300">
 <tr class="border-b border-slate
 <td class="py-3 pr-3 font-b
 <td class="py-3 pr-3">деда
 <td class="py-3 pr-3">сам х
 <td class="py-3">х стал кор
 </tr>
 <tr class="border-b border-slate
 <td class="py-3 pr-3 font-b
 <td class="py-3 pr-3">х и р
аво-право)</td>
 <td class="py-3 pr-3"><span
 <td class="py-3">х поднялся
 </tr>
 <tr>
 <td class="py-3 pr-3 font-b
 <td class="py-3 pr-3">х и р
 <td class="py-3 pr-3"><span
 <td class="py-3">р и г стал
 </tr>
 </tbody>
 </table>
</div>

```

```

<p class="text-slate-300 text-sm mt-4">И та
нова смотрим на тройку. Zig может случиться
<p class="text-slate-400 text-xs mt-3"> Ни
вороты можно поделать руками.</p>

```

```
</div>
```

```
<!-- 5. Главная ловушка -->
```

```

<div class="bg-rose-950/30 p-6 rounded-xl borde
 <h3 class="text-lg font-bold text-rose-300
 <p class="text-slate-300 text-sm mb-4">Это

```

```
<div class="bg-slate-800/60 p-6 rounded-xl border
 <h3 class="text-lg font-bold text-white mb-3">
 <p class="text-slate-300 text-sm mb-4">Дере
 3 шага, а потом делаем Splay(10).</p>
 <pre class="bg-slate-950 p-4 rounded-lg ove
 0 leading-relaxed">старт после Zi
 глубина 3 → 1 10 в корне
```

```

 40
 /
 30
 /
20
/
10
```

```

 30
 / \
 10 40
 \
 20
```

```

 10
 \
 30
 / \
20 40
```

итог: было 4 узла в линию (высота 4) → стало высота 3,  
а сама десятка теперь в корне: следующий запрос к

```
<p class="text-slate-300 text-sm mt-4">Обра
инили дерево по пути. Все узлы, мимо
</div>
```

<!-- 7. Амортизация -->

```
<div class="bg-slate-800/60 p-6 rounded-xl bord
 <h3 class="text-lg font-bold text-white mb-3">
 <p class="text-slate-300 text-sm mb-3">Стро
оддерева) по всем узлам, и Zig-Zig/Zig-Zag :
ах идея такая:</p>
```

```
<div class="grid grid-cols-1 md:grid-cols-3
 <div class="bg-slate-900 rounded-lg p-4
 <p class="text-white font-bold mb-1
 <p class="text-slate-400 text-xs">c
 </div>
```

```
<div class="bg-slate-900 rounded-lg p-4
 <p class="text-white font-bold mb-1
 <p class="text-slate-400 text-xs">H
</div>
```

```
<div class="bg-slate-900 rounded-lg p-4
```

```

 xed">// узел: { key, left, right, parent }

// один поворот: x встаёт на место своего родителя
function rotate(x) {
 const p = x.parent, g = p.parent;

 if (p.left === x) { // правый поворот
 p.left = x.right;
 if (x.right) x.right.parent = p;
 x.right = p;
 } else { // левый поворот (зеркало)
 p.right = x.left;
 if (x.left) x.left.parent = p;
 x.left = p;
 }

 p.parent = x; // родитель уехал вниз
 x.parent = g; // x занял его место
 if (g) { if (g.left === p) g.left = x; else g.right = x; }
}

// поднимаем x в корень
function splay(x) {
 while (x.parent) {
 const p = x.parent, g = p.parent;

 if (!g) { // ZIG: деда не было
 rotate(x);
 } else if ((g.left === p) === (p.left === x)) {
 rotate(p); // ZIG-ZIG: сноровка
 rotate(x);
 } else {
 rotate(x); // ZIG-ZAG: сноровка
 rotate(x);
 }
 }
 return x; // теперь x — корень
}

```



```

<!-- 11. Вопросы -->
<details class="bg-slate-900/40 rounded-lg border-2
 <summary class="p-4 font-bold text-slate-400"
 open:border-b border-slate-700/50">
 Вопросы, которые задают на экзамене (
 </summary>
 <div class="p-5 text-sm text-slate-300 space-y-4">
 <div>
 <strong class="text-white block mb-2">Вопрос
 <p class="text-slate-400">Последний
 енный (или преемник). Splay делают всегда, да
 бы, и амортизация сломалась бы.</p>
 </div>
 <div>
 <strong class="text-white block mb-2">Почему
 <p class="text-slate-400">Потому что
 еворачивают бамбук в другую сторону (см. бл
 </div>
 <div>
 <strong class="text-white block mb-2">Как
 <p class="text-slate-400">Чередовани
 вает один из них в корень, превращая дерево
 ступает только для одной конкретной
 </div>
 <div>
 <strong class="text-white block mb-2">Как
 <p class="text-slate-400">Треар дер
 play не хранит вообще ничего и держит балан
 <p>
 </div>
 </div>
</details>

</div>
</section>`,
 },
 {

```

```

 description: "Формула Эйлера и теорема о 4 красках.",
 category: "Графы. Теория",
 content: `
<section id="graph-planar-colors" class="mb-12 scroll-m-12">
 <div class="flex items-center mb-6 flex-wrap gap-3">
 Теория
 <h2 class="text-2xl sm:text-3xl font-bold text-slate-800">Теорема о 4 красках
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-600">
 <h3 class="text-xl font-bold text-blue-400">Формулировка
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-blue-300 italic">Любой планарный граф можно раскрасить 4 цветами.
 <p class="text-xl font-mono text-white">где V - количество вершин, E - количество ребер.
 </div>
 <div class="grid grid-cols-1 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
 Аналогия 1: Карта мира
 </div>
 <p class="text-slate-300 text-sm mb-4">Границы не пересекаются в одной точке по-
 тому что каждая граница имеет свой цвет, а цвета
 не сливались цветами, Всегда можно всего 4 цвета.
 </p>
 </div>
 </div>
 </div>
 </div>
</section>`,
 },
 {
 id: "top-sort",
 title: "9. Графы. Топологическая сортировка",
 type: "html",
 description: "Разложение задач по порядку выполнения",
 category: "Графы",
 content: `

```

```

 title: "10. Графы. Компоненты сильной связности (SCC)",
 type: "html",
 description: "Разбиение орграфа на макро-вершины. Алгоритм Косарaju.",
 category: "Графы. Продвинутые",
 content: `
<section id="scc-kosaraju" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">
 SCC
 <h2 class="text-2xl sm:text-3xl font-bold text-emerald-400">Графы. Компоненты сильной связности (SCC)
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border border-emerald-500">
 <h3 class="text-xl font-bold text-emerald-400">Макро-вершины
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-emerald-300 italic">Макро-вершины — это вершины, которые принадлежат одной и той же SCC.
 <p class="text-xl font-mono text-white">Они инвертированы в порядке top-sort.</p>
 </div>
 <div class="grid grid-cols-1 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
 Аналогия 1: Улицы с односторонним движением
 </div>
 <p class="text-slate-300 text-sm mb-4">Улицы, по которым можно двигаться только в одном направлении, называются односторонними. На карте они обозначаются стрелками. В графе это будет означать, что рёбра имеют направление. В графе с односторонними рёбрами (орграфе) можно двигаться только в определённом направлении по односторонним улицам. Как это выглядит на карте.</p>
 </div>
 </div>
 </div>
 </div>
</section>`,
 },
 {
 id: "graph-bridges",
 title: "11. Графы. Мосты",
 type: "html",
 description: "Рёбра, удаление которых расхреначивает граф. Алгоритм Бондарева."
 }

```



```

 <p class="text-xl font-mono text-white">
d-длина.</p>
</div>
<div class="grid grid-cols-1 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 b
rder border-emerald-500 shadow-md">
 Аналогия: Копаем туннель под
 </div>
 <p class="text-slate-300 text-sm mb
ней земли. Но если начать копать одновременно
в 2 РАЗА меньше работы (геометрически: площ
 </div>
 </div>
</div>
</div>
</section>`,
 },
 {
 id: "mst-kruskal",
 title: "18. Графы. Остовное дерево. Краскал",
 type: "html",
 description: "",
 category: "Графы. Остовные",
 content: `
<section id="mst-kruskal" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">
 <span class="bg-blue-600 text-white px-4 py-1 r
 <h2 class="text-2xl sm:text-3xl font-bold text-v
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl bord
 <h3 class="text-xl font-bold text-emerald-4
 <div class="bg-slate-900 p-4 rounded-lg mb-
 <p class="text-lg text-emerald-300 ital
 <p class="text-xl font-mono text-white">
 (проверяем через DSU) - берем в ответ.</p>
 </div>
 </div>
 </div>
</section>
`
 }
}

```

```

 order border-emerald-500 shadow-md">
 ✖ Аналогия: Заражение вирусом
 </div>
 <p class="text-slate-300 text-sm mb-1">
 ли вирус). Мы стартуем из одного города, и
 Сеть растет только из одного связного куска
 </div>
 </div>
</div>
</section>`,
 },
 {
 id: "mst-boruvka",
 title: "20. Графы. Остовное дерево. Борувка",
 type: "html",
 description: "",
 category: "Графы. Остовные",
 content: `
<section id="mst-boruvka" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-emerald-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-emerald-300 italic">
 <p class="text-xl font-mono text-white">
 ается с соседом.</p>
 </div>
 <div class="grid grid-cols-1 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border
 order border-emerald-500 shadow-md">
 Аналогия: Племена
 </div>
 </div>
 </div>
 </div>
 </div>
</section>
`
 }
}

```

```

 концовка "АБРА" совпадает с началом "АБРА"
 моя перепроверок!</p>
 </div>
</div>
<details class="bg-slate-900/40 rounded-lg border-1
 <summary class="p-4 font-bold text-slate-300
 -b border-slate-700/50">
 Код (Скрыто)
 </summary>
 <div class="p-5 text-sm text-slate-300
 <pre class="bg-slate-950 p-4 rounded
vector<int> pi(s.length());
for (int i = 1; i < s.length(); i++) {
 int j = pi[i-1];
 while (j > 0 && s[i] != s[j]) j = pi[j-1];
 if (s[i] == s[j]) j++;
 pi[i] = j;
}</pre>
 </div>
</details>
</div>
</div>
</section>`,
 },
 {
 id: "string-z-func",
 title: "22. Z-функция",
 type: "html",
 description: "",
 category: "Строки",
 content: `
<section id="string-z-func" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border-1

```

```

title: "24. Классы сложности, сведение задач",
type: "html",
description: "",
category: "Теория сложности",
content: `
<section id="complexity-classes" class="mb-12 scroll-mt
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-v
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border
 <h3 class="text-xl font-bold text-purple-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-purple-300 italic">
 <p class="text-xl font-mono text-white">
Сведение (Reduction) A->B: Если я умею реша
 </div>
 <div class="grid grid-cols-1 xl:grid-cols-2">
 <!-- Аналогия 1 -->
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 l
 rder border-emerald-500 shadow-md">
 Аналогия 1: Судoku (P vs NP)
 </div>
 <p class="text-slate-300 text-sm mb
пример сортировка чисел). Класс NP –
енную сетку (ответ), он БЫСТРО (за класс P)
 </div>
 <!-- Аналогия 2 -->
 <div class="bg-slate-900 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 l
 rder border-purple-500 shadow-md">
 Аналогия 2: Машина-переводчик
 </div>
 <p class="text-slate-300 text-sm mb
ь отличный переводчик на английский (Сведен
аче B, то B – это NP-Полная (сосредо

```



```
 <div class="text-left font-mono text-sm"
 <p><strong class="text-white">Эйлер
ин раз. Вершины повторять можно.</p>
 <p><strong class="text-white">Эйлер
 <div class="p-3 bg-slate-950 rounded
 <p class="text-sm text-blue-300
 <p><strong class="text-white">П
 <p><strong class="text-white">Ц
 </div>
 </div>
 </div>
</div>
```

```
<p class="text-slate-300 text-sm">
 Билет 13 (Различие пути и цикла

• <span class="text-green-400 font-
шел-вошёл-вышел).

• <span class="text-yellow-400 font
финиш).
</p>
</div>
```

```
<div class="grid grid-cols-1 xl:grid-cols-2 gap
 <div class="bg-slate-800 p-6 rounded-lg bor
 <div class="absolute -top-3 left-4 bg-s
order-green-500 shadow-md">
 Путь: Нормальная прогулка
 </div>
 <p class="text-slate-300 text-sm mb-4">
 Ты вышел из дома (нечётная)
 талый пришёл в бар (вторая нечётная
 Домой вернуться не смог, потому что
 </p>
 </div>
```

```
<div class="bg-slate-900 p-6 rounded-lg bor
 <div class="absolute -top-3 left-4 bg-r
der-rose-500 shadow-md bg-slate-950">
 Цикл: Гамильтонов синдром (болезнь
```

```

type: "html",
content: `<section id="bridges-code" class="mb-20 s
<div class="flex items-center mb-6">
 <span class="bg-emerald-600 text-white px-4 py-
 <h2 class="text-3xl font-bold text-white">Мосты
</div>

<div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl bord
 <h3 class="text-xl font-bold text-emerald-4

 <div class="bg-slate-900 p-4 rounded-lg mb-
 <p class="text-lg text-emerald-300 ital
 <div class="text-left font-mono text-sm
 <p><strong class="text-white">tin[v
 <p><strong class="text-white">low[v
 <div class="p-3 bg-slate-950 rounde
 <span class="text-rose-400 font
 <p class="text-xl font-bold tex
 <p class="text-xs text-slate-40
 </div>
 </div>
 </div>
 </div>

 <p class="text-slate-300 text-sm">
 Если из сына и и всех его потомков <str
 Единственная ниточка. Порвётся – граф р
 </p>
</div>

<div class="grid grid-cols-1 xl:grid-cols-2 gap
 <div class="bg-slate-800 p-6 rounded-lg bord
 <div class="absolute -top-3 left-4 bg-s
 border-emerald-500 shadow-md">
 Мост: Единственная веревка
 </div>
 <p class="text-slate-300 text-sm mb-4">
 Представь, что ты альпинист. Ты спу

```

```
vector<bool> visited;
vector<int> tin, low;
int timer;

void dfs(int v, int p = -1) {
 visited[v] = true;
 tin[v] = low[v] = timer++; // устанавливаем

 for (int to : adj[v]) {
 if (to == p) continue; // не идём назад

 if (visited[to]) {
 // обратное ребро: обновляем low
 low[v] = min(low[v], tin[to]);
 } else {
 // ребро дерева DFS
 dfs(to, v); // рекурсивный вызов
 low[v] = min(low[v], low[to]);

 if (low[to] > tin[v]) {
 // ЭТО МОСТ!
 // bridge_list.push_back({v, to});
 }
 }
 }
}

void find_bridges() {
 timer = 0;
 visited.assign(n, false);
 tin.assign(n, -1);
 low.assign(n, -1);

 for (int i = 0; i < n; ++i) {
 if (!visited[i]) dfs(i);
 }
}
```

```
<div class="bg-slate-800 p-6 rounded-lg border-rose-500 shadow-md">
 <div class="absolute -top-3 left-4 bg-slate-800 border-rose-500 shadow-md">
```

К5: Полный граф на 5 вершинах

```
</div>
```

```
<p class="text-slate-300 text-sm mb-4">
```

У К5: `<code class="text-white">V=5, E=10 > 9</code>` – БАХ, непланарен!

Все 5 вершин соединены со всеми. На

Теорема Куратовского: К5 – эталон не

Если внутри графа спрятан К5 – забудь

```
</p>
```

```
</div>
```

```
<div class="bg-slate-900 p-6 rounded-lg border-rose-500 shadow-md">
```

```
<div class="absolute -top-3 left-4 bg-slate-900 border-rose-500 shadow-md">
```

К3,3: 3 дома и 3 колодца

```
</div>
```

```
<p class="text-slate-300 text-sm mb-4">
```

Три дома хотят соединиться с тремя

Нарисовать без пересечений невозможно

У него `<code class="text-white">V=6, E=9 ≤ 12</code>` – проходит, но он не

Почему? Потому что у двудольного графа

Отсюда более строгое ограничение: `<code class="text-white">9 > 8</code>`

```
</p>
```

```
</div>
```

```
</div>
```

```
<details class="bg-slate-900/40 rounded-lg border-rose-500 shadow-md">
```

```
<summary class="p-4 font-bold text-slate-400 border-rose-500 shadow-md">
```

Доказательство непланарности К5 (Скрыто)

```
</summary>
```

```
<div class="p-5 text-sm text-slate-300 space-y-2">
```

```
<p class="text-blue-400">Доказательство
```

```
</div>
```

```
<p class="text-slate-300 text-sm">
```

```
 Важно: Это работает то
 еплятся к точкам сочленения. То есть,
 ег - это тупик со степенью 1).
```

```
</p>
```

```
</div>
```

```
<div class="grid grid-cols-1 xl:grid-cols-2 gap-
```

```
 <div class="bg-slate-800 p-6 rounded-lg border-
```

```
 <div class="absolute -top-3 left-4 bg-slate-800
 border-rose-500 shadow-md">
```

```
 Аналогия: Главный перекресток
```

```
 </div>
```

```
 <p class="text-slate-300 text-sm mb-4">
```

```
 Представь город, где два района соединены
 её перекроют (удалят вершину) – районы будут
 g>хрупкость системы.
```

```
 </p>
```

```
 </div>
```

```
<div class="bg-slate-900 p-6 rounded-lg border-
```

```
 <div class="absolute -top-3 left-4 bg-emerald-500
 border-emerald-500 shadow-md">
```

```
 Телеком: Центральный свитч
```

```
 </div>
```

```
 <p class="text-slate-300 text-sm mb-4">
```

```
 У тебя несколько компьютеров в одной сети
 абелем (мостом). Сами свитчи – это точки со
```

```
 </p>
```

```
</div>
```

```
</div>
```

```
<details class="bg-slate-900/40 rounded-lg border-
```

```
 <summary class="p-4 font-bold text-slate-400 border-slate-700/50">
```

```
 Душный мод: Код и корень DFS (Скрыто)
```

```

 </p>
 <ul class="list-disc list-inside text-sm text-rose-400">
 <strong class="text-rose-400">Точки
ong> и показывают физическую уязвимость свя.
 <strong class="text-emerald-400">SCC
ависимые "конгломераты".
 Как точка сочленения рушит :
Косарайю, свернув каждую SCC в одну супер-в
(сделаем неориентированным), то любая <strong
о и есть критическая SCC! Если удалить эту
. Одно слабое звено изолирует целые касты S

 </div>
</div>
</section>`,
 },
];

```

ФАЙЛ 19/81: src/data/content.ts

КАТЕГОРИЯ: Контент и данные пособия | СТРОК: 51 | РАЗМЕР: 1.5 КБ

```

import { Chapter } from "../types";
import { graphChapters } from "./graphContent";
import { additionalTickets } from "./additionalTickets";
import { tickets14to20 } from "./tickets/ticket14_20";
import { tickets21to24 } from "./tickets/ticket21_24";
import { tickets6to13 } from "./tickets/ticket6_13";
import { chapters as newChapters } from "./content_temp";

// We want to combine all of them, but overwrite 7, 11,
const merged = [
 ...graphChapters,
 ...additionalTickets,
 ...tickets6to13,
 ...tickets14to20,

```

ФАЙЛ 20/81: src/data/glossary.ts

КАТЕГОРИЯ: Контент и данные пособия | СТРОК: 636 | РАЗМ

```
import type { DemoKind } from "../components/hints/MiniI
```

```
/**
```

```
 * Глоссарий «википедийных» подсказок.
```

```
 *
```

```
 * Любое слово из `labels`, встреченное в тексте главы,
```

```
 * в подчёркнутый термин: при наведении (или тапе на мо
```

```
 * с объяснением на пальцах и мини-визуализацией.
```

```
 */
```

```
export interface GlossaryEntry {
```

```
 id: string;
```

```
 /** Варианты написания в тексте (регистр не важен). */
```

```
 labels: string[];
```

```
 title: string;
```

```
 /** Объяснение «на пальцах» – 1–2 фразы. */
```

```
 short: string;
```

```
 /** Строгая формулировка / что важно не забыть. */
```

```
 formal?: string;
```

```
 complexity?: string;
```

```
 /** Какая мини-анимация показывается в карточке. */
```

```
 demo?: DemoKind;
```

```
 /** id полного симулятора (см. vizRegistry) – кнопка
```

```
 simulator?: string;
```

```
}
```

```
export const GLOSSARY: GlossaryEntry[] = [
```

```
 {
```

```
 id: "bfs",
```

```
 labels: ["BFS", "обход в ширину", "поиск в ширину",
```

```
 title: "BFS – обход в ширину",
```

```
 short:
```

```
 "Волна от источника: сначала все соседи, потом со
```

```
 formal:
```

```

 "Полное бинарное дерево в массиве: дети узла i ле
complexity: "вставка/извлечение $O(\log n)$, «посмотре
demo: "heap",
simulator: "heap-beam-search",
 },
 {
 id: "stack",
 labels: ["стек", "стека", "стеке", "стеком", "LIFO"],
 title: "Стек (LIFO)",
 short: "Стопка тарелок: кладёшь сверху и берёшь свер
 formal: "push / pop / top за $O(1)$. На стеке живёт р
 demo: "stack",
 simulator: "stack-dfs",
 },
 {
 id: "queue",
 labels: ["очередь", "очереди", "очередью", "FIFO"],
 title: "Очередь (FIFO)",
 short: "Очередь в столовой: кто первым встал, тот п
 formal: "push_back / pop_front за $O(1)$. Основа BFS
 demo: "queue",
 simulator: "queue-bfs",
 },
 {
 id: "dsu",
 labels: ["DSU", "CHM", "система непересекающихся мн
 title: "DSU – система непересекающихся множеств",
 short:
 "«Кто твой староста?» Каждый элемент знает своего
 formal:
 "find со сжатием путей + union по рангу/размеру.
 complexity: "почти $O(1)$ – обратная функция Аккерман
 demo: "dsu",
 simulator: "mst-kruskal",
 },
 {
 id: "trie",
 labels: ["бор", "бора", "боре", "бору", "бором", "t

```



```

 строки.",
 formal: "link(v) = go(link(parent(v)), ch). Считает",
 demo: "suffix-link",
 simulator: "aho-corasick",
},
{
 id: "toposort",
 labels: ["топологическая сортировка", "топсорт", "топ"],
 title: "Топологическая сортировка",
 short:
 "Порядок «сначала штаны, потом ботинки»: каждая с
 циклов.",
 formal: "DFS + вершины в обратном порядке выхода, л",
 complexity: "O(V + E)",
 demo: "toposort",
 simulator: "top-sort",
},
{
 id: "relaxation",
 labels: ["релаксация", "релаксацию", "релаксации",
 title: "Релаксация ребра",
 short:
 "«А через соседа не быстрее?» Если известный путь
 ,
 formal: "if d[v] > d[u] + w(u,v) → d[v] = d[u] + w(u,v)",
 ан и Флойд.",
 demo: "relax",
 simulator: "dijkstra",
},
{
 id: "prefix-sum",
 labels: ["префиксные суммы", "префиксная сумма", "пре"],
 title: "Префиксные суммы",
 short:
 "Заранее посчитанные «сколько всего набежало от н",
 formal: "P[0]=0, P[i]=P[i-1]+a[i-1]. Сумма a[l..r] = P[r]-P[l-1]",
 complexity: "препроцессинг O(n), запрос O(1)",
 demo: "prefix-sum",

```

```

 simulator: "bridges-code",
},
{
 id: "articulation",
 labels: ["точка сочленения", "точки сочленения", "т"],
 title: "Точка сочленения",
 short:
 "Вершина-незаменимый-сотрудник: уволь её — и коллапс",
 formal: "Для не-корня: v — точка сочленения $\Leftrightarrow$ $\exists$ ребро",
 demo: "articulation",
 simulator: "graph-articulation",
},
{
 id: "mst",
 labels: [
 "остовное дерево", "остовного дерева", "остовное",
 "минимальное остовное дерево", "MST",
],
 title: "Минимальное остовное дерево (MST)",
 short:
 "Связать все точки проводами так, чтобы суммарная",
 formal: " $V-1$ ребро, никаких циклов. Краскал (жадно и",
 "но).",
 demo: "mst",
 simulator: "mst-kruskal",
},
{
 id: "amortized",
 labels: [
 "амортизированная", "амортизированный", "амортизи",
 "амортизированное", "амортизированного", "амортизи",
],
 title: "Амортизированная сложность",
 short:
 "Средняя цена операции «по абонементу»: одна опер",
 formal: "Метод потенциалов: дорогая операция оплачи",
 "ассив.",
 demo: "amortized",

```

```

 labels: [
 "префикс-функция", "префикс-функции", "префикс-функц",
 "префикс", "префикса", "суффиксом", "суффикс", "с",
 "подстроки", "подстроку", "подстрока",
],
 title: "Префикс-функция π (КМП)",
 short:
 "Для каждой позиции запоминаем: какой кусок начался",
 "от кусок.",
 formal:
 " $\pi[i]$ — длина наибольшего собственного префикса s ",
 "возвращая указатель по тексту.",
 complexity: "O(n + m)",
 demo: "prefix-function",
 simulator: "string-kmp",
},
{
 id: "z-function",
 labels: ["Z-функция", "Z-функции", "Z-функцию", "z-функ",
 title: "Z-функция",
 short:
 "Для каждой позиции: сколько символов подряд, начинающихся",
 "с каждой буквы.",
 formal:
 " $z[i]$ = длина наибольшего общего префикса s и $s[i..n]$ ",
 complexity: "O(n)",
 demo: "prefix-function",
 simulator: "string-z-func",
},
{
 id: "dp",
 labels: [
 "динамическое программирование", "динамики", "динамическое",
 "мемоизация", "мемоизацию", "подзадач", "подзадачу",
],
 title: "Динамическое программирование",
 short:
 "Решаем задачу через ответы на её меньшие копии и"
```

```

 formal:
 "Корректность обычно доказывается «леммой о разре
demo: "mst",
simulator: "mst-kruskal",
},
{
 id: "euler",
 labels: ["эйлеров цикл", "эйлеров путь", "эйлерова
title: "Эйлеров путь и цикл",
short:
 "Пройти по каждому ребру ровно один раз, не отрыв
formal:
 "Связный граф: эйлеров цикл все степени чётные;
simulator: "euler-path-vs-cycle",
},
{
 id: "planarity",
 labels: ["планарный", "планарность", "планарного",
title: "Планарность",
short:
 "Граф планарен, если его можно нарисовать на листе
 ,
formal:
 " $V - E + F = 2$ для связного планарного графа. Кур
 ,
simulator: "planarity-euler-formula",
},
{
 id: "splay",
 labels: [
 "splay", "splay-дерево", "AVL", "AVL-дерево", "вс
],
title: "Splay и повороты",
short:
 "Каждый раз, когда к узлу обратились, его «вытягив
formal:
 "Три случая: zig (родитель – корень), zig-zig (уз
 рацию.",

```

```

 },
 {
 id: "floyd",
 labels: [
 "Флойд", "Флойда", "Флойд-Уоршелл", "Флойда-Уорше",
 "промежуточная вершина", "промежуточную вершину",
],
 title: "Флойд-Уоршелл: разрешаем вершину k",
 short:
 "Внешний цикл – это не «шаг», а «какие пересадки

 шить пересадку в k?",
 formal:
 " $D[i][j] = \min(D[i][j], D[i][k] + D[k][j])$. Порядок

 ами.",
 complexity: "O(V³) времени, O(V²) памяти",
 demo: "floyd",
 simulator: "floyd",
 },
 {
 id: "prim",
 labels: ["Прима", "Прим", "Prim"],
 title: "Алгоритм Прима",
 short:
 "Остов растёт как плесень из одной точки: на кажды
 formal:
 "Множество посещённых + куча рёбер на границе. До
 усок.",
 complexity: "O(E log V)",
 demo: "mst",
 simulator: "mst-prima",
 },
 {
 id: "boruvka",
 labels: ["Борувка", "Борувки", "Борувку", "Boruvka"],
 title: "Алгоритм Борувки",
 short:
 "Все компоненты одновременно шлют «гонца» к ближай
 formal:

```

```

],
 title: "Граф",
 short:
 "Точки (вершины) и связи между ними (рёбра). Города",
 formal:
 "G = (V, E). Ребро бывает ориентированным (улицы)",
 demo: "graph",
 simulator: "graph-dfs-bfs",
},
{
 id: "coord-compress",
 labels: ["сжатие координат", "сжатия координат", "сжатие координат"],
 title: "Сжатие координат",
 short:
 "Координаты бывают гигантские и дробные, а нам нужны",
 "порядковым номером.",
 formal:
 "sort + unique + binary search: значение → индекс",
 complexity: "O(n log n)",
 demo: "coord-compress",
},
{
 id: "treap",
 labels: ["Treap", "декартово дерево", "декартова деревья"],
 title: "Treap = Tree + Heap",
 short:
 "Одно дерево живёт по двум правилам сразу: по ключу",
 "и по приоритету. Случайность и держит его сбалансированным.",
 formal:
 "Базис — split(t, x) (разрезать по ключу) и merge(t1, t2)",
 "через них. Ожидаемая высота O(log n).",
 complexity: "ожидаемо O(log n)",
 demo: "heap",
},
{
 id: "bst",
 labels: ["бинарное дерево поиска", "дерево поиска", "дерево"],

```

```

 title: "Запрос на отрезке",
 short:
 "Спрашиваем не про один элемент, а про кусок массива и
 предподсчёты.",
 formal:
 "Суммы — префиксные суммы $O(1)$. Минимум без изменений.",
 demo: "prefix-sum",
 simulator: "prefix-sums-2d",
},
{
 id: "string",
 labels: ["строки", "строке", "строку", "строка", "строки"],
 title: "Строка как массив символов",
 short:
 "Строковые алгоритмы держатся на трёх вопросах: где встречается
 (Z-функция) и где встречаются слова словаря (бор).",
 formal:
 "Наивный поиск подстроки — $O(N \cdot M)$. Префикс-функция.",
 demo: "prefix-function",
 simulator: "string-kmp",
},
{
 id: "rotation",
 labels: ["поворот", "повороты", "поворота", "поворот"],
 title: "Поворот (rotation) в дереве",
 short:
 "Локальная перевеска двух узлов: ребёнок встаёт на место
 дившемся месту. Порядок ключей при этом не ломается.",
 formal:
 "Правый поворот вокруг p: $x = p.left; p.left = x$.
 g и Zig-Zag.",
 complexity: " $O(1)$ ",
 demo: "zig",
 simulator: "splay-rotations",
},
{
 id: "zig",
 labels: ["Zig", "Zag", "зиг"],

```

```
e.labels.map((label) => ({ label, id: e.id })))
).sort((a, b) => b.label.length - a.label.length);
```

---

ФАЙЛ 21/81: src/data/graphContent.ts

КАТЕГОРИЯ: Контент и данные пособия | СТРОК: 275 | РАЗМ

---

```
import { Chapter } from "../types";
```

```
export const graphChapters: Chapter[] = [
 {
 id: "intro",
 title: "Обзор: Кратчайшие пути и Графы",
 type: "html",
 category: "Графы",
 content: `<section id="intro-content" class="mb-12">
<div class="flex items-center mb-6">

 <h2 class="text-3xl font-bold text-white">Обзор
 </div>
<div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-blue-400">
 <p class="text-slate-300 text-sm mb-4">Пред
 оезда). Алгоритмы поиска кратчайшего пути п
 <div class="mt-8 mb-4">
 <div id="slot-mnemonic-cards"></div>
 </div>
 </div>
</div>
</section>`
 },
 {
 id: "dijkstra",
 title: "Билет 14. Дейкстра",
 type: "html",
```



```

 <details class="bg-slate-900/40 rounded-lg border
 <summary class="p-4 font-bold text-slate-300
 -b border-slate-700/50">
 Строгое доказательство (Скрыто)
 </summary>
 <div class="p-5 text-sm text-slate-300
 <p>Алгоритм использует приоритетную
 g V). Гарантирует корректность только для н
 </div>
 </details>

 <div id="slot-dijkstra-viz" class="mt-8"></div>
 </div>
</section>`
 },
 {
 id: "bellman-ford",
 title: "Билет 15. Форд-Беллман",
 type: "html",
 category: "Графы",
 content: `<section id="bellman-ford-content" class=
 <div class="flex items-center mb-6">
 <span class="bg-blue-600 text-white px-4 py-1 r
 <h2 class="text-3xl font-bold text-white">Форд-Б
 </div>

 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border
 <h3 class="text-xl font-bold text-rose-400 m

 <div class="bg-slate-900 p-4 rounded-lg mb-6
 <p class="text-lg text-rose-300 italic m
 <p class="text-xl font-mono text-white"
 </div>

 <div class="grid grid-cols-1 xl:grid-cols-2

```

```

<div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border"
 <h3 class="text-xl font-bold text-indigo-400">

 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-indigo-300 italic">
 <p class="text-xl font-mono text-white">
 </div>

 <div class="grid grid-cols-1 xl:grid-cols-2">
 <!-- Аналогия 1 -->
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
 Аналогия 1: Автомагистрали
 </div>
 <p class="text-slate-300 text-sm mb-4">
 евле долететь из Москвы в Париж, если мы сд
 <div id="slot-chapter-image-floyd"
 </div>

 <!-- Аналогия 2 -->
 <div class="bg-slate-900 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-rose-500 shadow-md">
 Время работы
 </div>
 <p class="text-slate-300 text-sm mb-4">
 три вложенных цикла. Сложность $O(V^3)$. Если
 </div>
 </div>

 <div id="slot-floyd-viz" class="mt-8"></div>
 </div>
</div>
</section>`
},
{

```

## Аналогия 2: Матёшка (Терминальные)

```

</div>
<p class="text-slate-300 text-sm mb-4">Представ
нашли и "he", потому что оно спрятано внутри
минальные ссылки (term_link) – прямые мос
p>
</div>
</div>

<details class="bg-slate-900/40 rounded-lg border b
<summary class="p-4 font-bold text-slate-400 outl
pen:border-slate-700/50">
 Внутренняя структура автомата и код BFS (Скры
</summary>
<div class="p-5 text-sm text-slate-300 space-y-4">
 <p>Каждая вершина имеет таблицу переходов <code>
 ссылку <code>term_link</code> (ближайшая мен
 <div class="bg-black/50 p-4 rounded-lg font-mon
struct Node {
 map<char, int> go; // Предвычисленные пер
 int link = 0; // Суффиксная ссылка: куда
 int term_link = 0; // Терминальная ссылка: матр
 bool is_terminal = false;
 vector<string> words;
};

// Расчет ссылок идет через BFS (level-order traversal)
// так как для вычисления link вершины v на глубине d,
// её предки на глубине d-1 уже должны иметь вычисленные
void buildLinks() {
 queue<int> q;
 // Корни и их дети
 for (auto const& [ch, child] : nodes[0].trie) {
 nodes[child].link = 0;
 q.push(child);
 }

 while (!q.empty()) {

```

```

 ут быть большими дробными числами (долгота
 ву от 0 до N, и уже по ним строим графы.</p>
 </div>
</div>
</section>`
 }
};

```

ФАЙЛ 22/81: src/data/quizzes.ts

КАТЕГОРИЯ: Контент и данные пособия | СТРОК: 106 | РАЗМ:

```
export interface QuizQuestion {
 question: string;
 options: string[];
 correctIndex: number;
 explanation: string;
}

export const quizzes: Record<string, QuizQuestion[]> =
 "euler-path-vs-cycle": [
 {
 question: "В связном графе ровно 2 вершины с нечётными степенями",
 options: [
 "Есть эйлеров цикл (вернусь домой!)",
 "Есть эйлеров путь (из одной нечётной в другую)",
 "Ни пути, ни цикла нет — это полный провал"
],
 correctIndex: 1,
 explanation: "Если нечётных вершин ровно 2 — это эйлеров путь. Если нечётных больше, то ни пути, ни цикла нет. Если нечётных 0, то есть цикл."
 },
 {
 question: "Почему гамильтонов цикл — это NP-полная задача?",
 options: [
 "Потому что Гамильтон — болезнь, и лечить её тяжело",
 "Потому что это задача на перебор",
 "Потому что это задача на динамическое программирование"
],
 correctIndex: 0,
 explanation: "Задача нахождения гамильтонова цикла является NP-полной, так как она принадлежит к классу NP и не имеет известного полиномиального алгоритма решения."
 }
]
```

```

 explanation: "low[v] хранит минимальное время входа в вершину v при поиске мостов и точек сочленения.",
 },
 {
 question: "Какова временная сложность алгоритма поиска мостов?",
 options: [
 "O(V^2) – ищем мосты в квадрате",
 "O(V + E) – каждый элемент один раз",
 "O(2^V) – полный перебор"
],
 correctIndex: 1,
 explanation: "DFS обходит каждую вершину и каждое ребро.",
 },
],
"planarity-euler-formula": [
 {
 question: "Планарный граф имеет V=6, E=12. Возможно ли такое?",
 options: [
 "Да, если это двудольный граф",
 "Нет, нарушается неравенство E ≤ 3V - 6",
 "Да, если нет треугольных граней"
],
 correctIndex: 1,
 explanation: "E ≤ 3V - 6 ⇒ 12 ≤ 12 – на грани простоя. Но для планарности нужно E ≤ 3V - 6. Так что такой граф непланарен.",
 },
 {
 question: "Сколько граней у планарного графа с V=7, E=10?",
 options: [
 "3 грани",
 "5 граней",
 "10 граней"
],
 correctIndex: 1,
 explanation: "F = E - V + 2 = 10 - 7 + 2 = 5. Не 10.",
 },
 {
 question: "Почему K5 непланарен?",
 },

```

```
<div id="intro" class="bg-slate-700/50 p-6 rounded-
 <h3 class="text-xl font-bold text-blue-400 m-
```

```
<div class="bg-slate-900 p-4 rounded-lg mb-10
 <p class="text-lg text-blue-300 italic m-0
 <p class="text-xl font-mono text-white" m-0
omise (lazy).</p>
</div>
```

```
<div class="grid grid-cols-1 xl:grid-cols-2
 <div class="bg-slate-800 p-6 rounded-lg
 <div class="absolute -top-3 left-4 right-4
rder border-emerald-500 shadow-md">
```

Аналогия 1: Корпоративная презентация

```
</div>
 <p class="text-slate-300 text-sm mb-4" m-0
ы рассылать 10 000 писем, он пишет одно письмо
счетах сотрудников только тогда, когда сотру
женное обновление).</p>
</div>
```

```
<div class="bg-slate-900 p-6 rounded-lg
 <div class="absolute -top-3 left-4 right-4
border-rose-500 shadow-md">
```

Аналогия 2: Матрешки-коробки

```
</div>
 <p class="text-slate-300 text-sm mb-4" m-0
ше, и так далее. Если нам нужно покрасить по
Внутри всё синее!" на большую коробку. Опус
</div>
</div>
```

```
<details class="bg-slate-900/40 rounded-lg
 <summary class="p-4 font-bold text-slate-300
-b border-slate-700/50">
```

Строгое доказательство / Код (Скрытие)

```
</summary>
```

```
<div class="p-5 text-sm text-slate-300 m-0"
```

```

<div class="bg-slate-800 p-6 rounded-lg" style="border: 1px solid #000; border-radius: 10px; padding: 10px;">
 <div class="absolute -top-3 left-4 right-4 bottom-4 border border-emerald-500 shadow-md" style="border: 1px solid #000; border-radius: 10px; padding: 10px;">
 Аналогия 1: Школьные линейки
 </div>
 <p class="text-slate-300 text-sm mb-4" style="color: #a0c0ff; font-size: 0.9em; margin-bottom: 10px;">
 отрезок длины 7. Ты берешь две заготовленные заготовки, берешь отрезок длины 7 (перекрывая друг друга по 1 см), ничего складывать, просто пересекаем куски.
 </p>
 <div class="bg-slate-900 p-6 rounded-lg" style="border: 1px solid #000; border-radius: 10px; padding: 10px;">
 <div class="absolute -top-3 left-4 right-4 bottom-4 border border-rose-500 shadow-md" style="border: 1px solid #000; border-radius: 10px; padding: 10px;">
 Аналогия 2: Ступени
 </div>
 <p class="text-slate-300 text-sm mb-4" style="color: #a0c0ff; font-size: 0.9em; margin-bottom: 10px;">
 чтобы покрыть отрезок, мы берем две самые большие ступени.
 </p>
 </div>
 <details class="bg-slate-900/40 rounded-lg" style="border: 1px solid #000; border-radius: 10px; padding: 10px;">
 <summary class="p-4 font-bold text-slate-300" style="color: #a0c0ff; font-weight: bold; padding: 5px 10px;">
 -b border-slate-700/50" style="border: 1px solid #000; border-radius: 10px; padding: 10px;">
 Код (Скрыто)
 </summary>
 <div class="p-5 text-sm text-slate-300" style="padding: 5px 10px;">
 <pre class="bg-slate-950 p-4 rounded" style="background-color: #000; color: #a0c0ff; font-family: monospace; padding: 10px;">
int k = log2(R - L + 1);
int minimum = min(st[L][k], st[R - (1 << k) + 1][k]);</pre>
 </div>
 </details>
 </div>
 </div>
</section>`
},
{
 id: "treap",
 title: "3. Декартово дерево (Treap)",
 type: "html",

```

Код (Скрыто)

```

</summary>
<div class="p-5 text-sm text-slate-300">
 <pre class="bg-slate-950 p-4 rounded">
pair<Node*, Node*> split(Node* t, int x) {
 if (!t) return {nullptr, nullptr};
 if (t->val <= x) {
 auto [L, R] = split(t->right, x);
 t->right = L;
 return {t, R};
 } else {
 auto [L, R] = split(t->left, x);
 t->left = R;
 return {L, t};
 }
}</pre>
</div>
</details>
</div>
</section>`
},
{
 id: "splay-tree",
 title: "4. Splay-дерево",
 type: "html",
 description: "Дерево поиска, которое самобалансирует",
 category: "Продвинутые структуры",
 content: `
<section id="splay-tree" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-blue-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">

```



```

 p.right = x.left;
 x.left = p;
 }
} </pre>

</div>
</details>

<div class="bg-indigo-950/60 p-6 rounded-xl">
 <h4 class="text-lg font-bold text-indigo-400">
 <div class="space-y-4 text-sm text-slate-300">
 <div>
 <strong class="text-white block">
 корень?
 <p>Оно поднимет узел, на котором
 ска. За счет сдвига этого листа в корень, д
 /p>
 </div>

 <div>
 <strong class="text-white block">
 <p>Если агент постоянно переключ
 находящимися на разных концах дерева, его
 ащая дерево в вытянутый бамбук для другого.
 . Постоянное свинг-переключение превратит п
 </div>

 <div>
 <strong class="text-white block">
 <p>Хотя один поиск может стоять
 емах обладают прекрасным свойством: они не
 по пути. "Палочное" дерево прессуется в сб
 осов.</p>
 </div>
 </div>
 </div>
 </div>
 </div>
 </div>
 </div>
</div>

```

---

ФАЙЛ 24/81: src/data/tickets/ticket14\_20.ts

КАТЕГОРИЯ: Билеты (учебный контент) | СТРОК: 248 | РАЗМ

---

```
import { Chapter } from '../..types';
```

```
export const tickets14to20: Chapter[] = [
```

```
{
```

```
 id: "dijkstra",
```

```
 title: "14. Графы. Поиск кратчайшего пути. Дейкстра
```

```
 type: "html",
```

```
 category: "Графы. Пути",
```

```
 content: `
```

```
<section id="dijkstra-content" class="mb-12 scroll-mt-12">
```

```
 <div class="flex items-center mb-6 flex-wrap gap-3">
```

```

```

```
 <h2 class="text-2xl sm:text-3xl font-bold text-white">
```

```
 </div>
```

```
 <div class="space-y-8">
```

```
 <div class="bg-slate-700/50 p-6 rounded-xl border">
```

```
 <h3 class="text-xl font-bold text-emerald-400">
```

```
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
```

```
 <p class="text-lg text-emerald-300 italic">
```

```
 <p class="text-xl font-mono text-white">
```

```
 </div>
```

```
 <div class="grid grid-cols-1 xl:grid-cols-2">
```

```
 <!-- Аналогия 1 -->
```

```
 <div class="bg-slate-800 p-6 rounded-lg">
```

```
 <div class="absolute -top-3 left-4 l
```

```
 <div class="border-emerald-500 shadow-md">
```

```
 Аналогия 1: Позитивное плани
```

```
 </div>
```

```
 <p class="text-slate-300 text-sm mb
```

```
 (нельзя поехать и заработать). Он всегда в
```

```
 </div>
```

```

 </div>
 <p class="text-slate-300 text-sm mb-4">
 еряет ВСЕ возможные дороги V-1 раз подряд.
 </div>
 <div class="bg-slate-900 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 right-4 border-rose-500 shadow-md">
 Отрицательный цикл
 </div>
 <p class="text-slate-300 text-sm mb-4">
 опали во временную петлю.</p>
 </div>
 </div>
 <div id="slot-bellman-viz" class="mt-8"></div>
</div>
</div>
</section>`
 },
 {
 id: "floyd",
 title: "16. Графы. Поиск кратчайшего пути. Флойд",
 type: "html",
 category: "Графы. Пути",
 content: `
<section id="floyd-content" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-indigo-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-indigo-300 italic">
 <p class="text-xl font-mono text-white">
 </div>
 </div>
 <div class="bg-slate-800 p-6 rounded-lg border">

```

```

<div class="bg-slate-900 p-4 rounded-lg mb-6">
 <p class="text-xl font-mono text-white">
</div>
<div class="grid grid-cols-1 gap-6 mb-6">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-amber-500 shadow-md">
 ⚡ Суть перевзвешивания
 </div>
 <p class="text-slate-300 text-sm mb-6">
 Дейкстра (Билет 14) быстрая, но работает очень медленно. Мы хотим сделать Джонсона пути остались теми же.
 </p>
 <ol class="text-sm text-slate-300 space-y-2">
 Добавляем фиктивную вершину
 Запускаем от неё медленный поиск «потенциалы» <code class="text-pink-400">
 Меняем старые веса: новое расстояние = старый вес + потенциал
 Магия! Все веса теперь минимальные (потенциалы сокращаются).
 Теперь смело запускаем быстрый поиск

 </div>
</div>
<div id="slot-johnson-viz" class="mt-8"></div>
</div>
</div>
</section>`
},
{
 id: "mst-kruskal",
 title: "18. Графы. Остовное дерево. Краскал",
 type: "html",
 category: "Графы. Остовные",
 content: `
<section id="mst-kruskal" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

```

```

<div class="bg-slate-700/50 p-6 rounded-xl border-1 border-emerald-500" >
 <h3 class="text-xl font-bold text-emerald-400" >
 <div class="bg-slate-900 p-4 rounded-lg mb-6" >
 <p class="text-lg text-emerald-300 italic" >
 <p class="text-xl font-mono text-white" >
 которое торчит из посещенных в непосещенных
 </p>
 </div>
 <div class="grid grid-cols-1 gap-6" >
 <div class="bg-slate-800 p-6 rounded-lg" >
 <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md" >
 ✖ Аналогия: Заражение вирусом
 </div>
 <p class="text-slate-300 text-sm mb-6" >
 ли вирус). Мы стартуем из одного города, и
 Сеть растет только из одного связного куска
 </p>
 </div>
 </div>
 </div>
 </div>
</section>`
 },
 {
 id: "mst-boruvka",
 title: "20. Графы. Остовное дерево. Борувка",
 type: "html",
 category: "Графы. Остовные",
 content: `
<section id="mst-boruvka" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white" >
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border-1 border-emerald-500" >
 <h3 class="text-xl font-bold text-emerald-400" >
 <div class="bg-slate-900 p-4 rounded-lg mb-6" >
 <p class="text-lg text-emerald-300 italic" >

```

```
<h2 class="text-2xl sm:text-3xl font-bold text-slate-800">
</div>
<div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-800">
 <h3 class="text-xl font-bold text-blue-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-blue-300 italic">
 <p class="text-xl font-mono text-white">
 сом, оканчивающимся в позиции i.</p>
 </div>

 <div class="grid grid-cols-1 xl:grid-cols-2">
 <!-- Аналогия 1 -->
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
 ♂ Аналогия 1: Поиск без отка
 </div>
 <p class="text-slate-300 text-sm mb-4">
 Мы идём по строке слева направо. Если строка
 строки СЕЙЧАС закончился под моим пальцем?»
 Представь, ты ищешь в тексте слово
 х</code>. Тупой алгоритм начнет искать заголовок
 </code>, а это начало нашего слова! Не надо
 </p>
 </div>

 <!-- Аналогия 2 -->
 <div class="bg-slate-900 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-rose-500 shadow-md">
 Аналогия 2: LLM стриминг
 </div>
 <p class="text-slate-300 text-sm mb-4">
 Для LLM, которая стримит токены
 каждом шаге. Если мы частично совпали с заголовком
 какого места этого слова мы можем продолжить
 </p>
 </div>
 </div>
 </h3>
 </div>
 </div>
</div>
```

```

 pi[i] = j;
 }</pre>
 </div>
 </details>
</div>
</div>
</section>`
 },
 {
 id: "string-z-func",
 title: "22. Z-функция — Сдвиг и наложение",
 type: "html",
 category: "Строки",
 content: `
<section id="string-z-func" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-emerald-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-emerald-300 italic">
 <p class="text-xl font-mono text-white">
 ca 0) и её суффиксом, начинающимся с i.</p>
 </div>

 <div class="grid grid-cols-1 xl:grid-cols-2">
 <!-- Аналогия 1 -->
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 l
rder border-emerald-500 shadow-md">
 Аналогия 1: Линейка-шаблон
 </div>
 <p class="text-slate-300 text-sm mb">
 Тут мы берём всю строку и «прик
 я начну читать строку отсюда, насколько дли

```

```

 <details class="bg-slate-900/40 rounded-lg
 <summary class="p-4 font-bold text-slate-300
 -b border-slate-700/50">
 Код C++ (Скрыто)
 </summary>
 <div class="p-5 text-sm text-slate-300
 <pre class="bg-slate-950 p-4 rounded
int l = 0, r = 0;
for (int i = 1; i < n; i++) {
 if (i <= r) z[i] = min(r - i + 1, z[i - 1]);
 while (i + z[i] < n && s[z[i]] == s[i + z[i]]) z[i]++;
 if (i + z[i] - 1 > r) { l = i; r = i + z[i] - 1; }
}</pre>
 </div>
 </details>
 </div>
</div>
</section>`
 },
 {
 id: "aho-corasick",
 title: "23. Алгоритм Ахо-Корасик",
 type: "html",
 category: "Строки",
 content: `
<section id="aho-corasick" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>

 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border-l">
 <h3 class="text-xl font-bold text-blue-400 mb-4">

 <div class="bg-slate-900 p-4 rounded-lg mb-6 text
 <p class="text-sm text-blue-300 italic mb-2">0c
 <p class="text-lg font-mono text-white">Бор (Tr
```



```

 return t[v].link;
 }

 int go(int v, char c) {
 if (t[v].go[c] == -1) {
 if (t[v].next[c] != -1) t[v].go[c] = t[v].next[c];
 else t[v].go[c] = v == 0 ? 0 : go(get_link(v), c);
 }
 return t[v].go[c];
 }
}

</div>
</details>
</div>
</section>`
},
{
 id: "complexity-classes",
 title: "24. Классы сложности, сведение задач",
 type: "html",
 category: "Теория сложности",
 content: `
<section id="complexity-classes" class="mb-12 scroll-mt-12">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-purple-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-purple-300 italic">
 <p class="text-xl font-mono text-white">
 Сведение (Reduction) A->B: Если я умею реша
 </div>
 <div class="grid grid-cols-1 xl:grid-cols-2">
 <!-- Аналогия 1 -->
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4">

```

```

category: "Графы. База",
content: `
<section id="graph-dfs-bfs" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-slate-300">
 </div>
 <div class="space-y-8">
 {/* База представлений */}
 <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-600">
 <h3 class="text-xl font-bold text-slate-300">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-slate-300 italic">
 <p class="text-xl font-mono text-white">
 (V + E).</p>
 </div>
 <div class="grid grid-cols-1 xl:grid-cols-2">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
 Аналогия: Матрица = Таблица
 </div>
 <p class="text-slate-300 text-sm mb-4">
 глупо, но проверка "знает ли Вася Петю" мгно
 </div>
 <div class="bg-slate-900 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-rose-500 shadow-md">
 Аналогия: Списки = Контакты
 </div>
 <p class="text-slate-300 text-sm mb-4">
 т. Оптимально для почти всех задач.</p>
 </div>
 </div>
 </div>
 </div>
 {/* DFS vs BFS */}
 <div class="bg-slate-700/50 p-6 rounded-xl border

```

<li>Двунаправленный поиск для у

</ul>

</div>

</div>

<details class="bg-slate-900/40 rounded-lg l

<summary class="p-4 font-bold text-slate

-b border-slate-700/50">

Код обходов C++ (Основа)

</summary>

<div class="grid grid-cols-1 md:grid-co

<pre class="bg-slate-950 p-3 rounde

// DFS (Рекурсия)

vector<bool> vis;

vector<vector<int>>> adj;

void dfs(int v) {

vis[v] = true;

for (int u : adj[v]) {

if (!vis[u]) dfs(u);

}

}</pre>

<pre class="bg-slate-950 p-3 rounde

// BFS (Очередь)

queue<int> q;

q.push(start\_node);

vis[start\_node] = true;

while(!q.empty()) {

int v = q.front(); q.pop();

for (int u : adj[v]) {

if (!vis[u]) {

vis[u] = true;

q.push(u);

}

}

}</pre>

</div>

```

</section>`,
 },
 {
 id: "graph-components",
 title: "8. Графы. Компоненты связности",
 type: "html",
 description: "Нахождение кусков графа",
 category: "Графы. База",
 content: `
<section id="graph-components" class="mb-12 scroll-mt-12">
 <div class="flex items-center mb-6 flex-wrap gap-3">
 Графы
 <h2 class="text-2xl sm:text-3xl font-bold text-emerald-400">8. Графы. Компоненты связности
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-600">
 <h3 class="text-xl font-bold text-emerald-400">Нахождение кусков графа
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-emerald-300 italic">Граф — это совокупность вершин и ребер.
 <p class="text-xl font-mono text-white">Граф может состоять из нескольких
 ество компонент.</p>
 </div>
 <div class="grid grid-cols-1 xl:grid-cols-2">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
 Аналогия 1: Острова
 </div>
 <p class="text-slate-300 text-sm mb-4">Если мы посмотрим на карту — остались ли серые (неисследованные)
 ь через океан — столько и компонент.</p>
 </div>
 </div>
 </div>
 </div>
</section>`,
 },
 {

```

```

 <details class="bg-slate-900/40 rounded-lg"
 <summary class="p-4 font-bold text-slate-300"
 -b border-slate-700/50">
 Душный мод: Код на C++ (Скрыто)
 </summary>
 <div class="p-5 text-sm text-slate-300"
 <p class="text-indigo-400 font-bold"
 <div class="bg-slate-950 p-4 rounded"
 <pre class="text-xs font-mono text-slate-300"
vector<int> adj[MAXN];
bool visited[MAXN];
vector<int> ans;

void dfs(int v) {
 visited[v] = true;
 for (int to : adj[v]) {
 if (!visited[to]) {
 dfs(to);
 }
 }
 ans.push_back(v); // Добавляем в момент ВЫХОДА (pos
}

void topological_sort(int n) {
 for (int i = 0; i < n; ++i) {
 if (!visited[i]) dfs(i);
 }
 reverse(ans.begin(), ans.end()); // Разворачиваем с
}</pre>
 </div>
 </div>
 </div>
</section>`,
 },
 {

```

### Аналогия: Интриги в офисе

```
</div>
<p class="text-slate-300 text-sm mb-4">
 Сильная связность — это "клуб слесарей"
 ни в одной SCC. Добавить курьера Гошу, который может
 йти наружу (к директору), но обратно в клуб не может.
</p>
</div>
```

```
<div class="bg-slate-900 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 right-4 border border-rose-500 shadow-md">
 Связь с Билетом 12
 </div>
 <p class="text-slate-300 text-sm mb-4">
 SCC (Билет 10) склеивает
 </p>
 <p class="text-slate-300 text-sm mb-4">
 Точки сочленения (Билет 12) — это слабая связность
 ость монолита. Вырвешь точку сочленения — получишь
 </p>
 </div>
</div>
```

```
<details class="bg-slate-900/40 rounded-lg">
 <summary class="p-4 font-bold text-slate-300">
 Душный мод: Код Алгоритм Косарайю
 </summary>
 <div class="p-5 text-sm text-slate-300">
 <p class="text-emerald-400 font-bold">Алгоритм Косарайю</p>
 <ol class="list-decimal list-inside">
 Запускаем DFS на исходном графе
 Разворачиваем этот список (только вершины)
 Переворачиваем все ребра в графе
 Идем по <code>order</code>: посещаем вершины в порядке
 !

```

```

</section>`,
 },
 {
 id: "graph-articulation",
 title: "12. Графы. Точки сочленения",
 type: "html",
 description: "Вершины, удаление которых убивает свя",
 category: "Графы. Продвинутые",
 content: `
<section id="graph-articulation" class="mb-12 scroll-mt
 <div class="flex items-center mb-6 flex-wrap gap-3">
 <span class="bg-blue-600 text-white px-4 py-1 r
 <h2 class="text-2xl sm:text-3xl font-bold text-r
 </div>

 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl bord
 <h3 class="text-xl font-bold text-rose-400 m

 <div class="bg-slate-900 p-4 rounded-lg mb-4
 <p class="text-lg text-rose-300 italic m
 <div class="text-left font-mono text-sm
 <p><strong class="text-white">Точка
рой увеличивает число компонент связности.<
 <div class="p-3 bg-slate-950 rounde
 <span class="text-rose-400 font
 <p class="text-xl font-bold tex
 <p class="text-xs text-slate-400
 </div>
 </div>
 </div>
 </div>

 <p class="text-slate-300 text-sm">
 Важно: Это работает то
еются к точкам сочленения. То есть,
ег - это тупик со степенью 1).
 </p>
</div>

```

```

 <div class="bg-slate-950 p-4 rounded-lg" style="background-color: #2d3748; color: white; padding: 10px; border-radius: 0.5em;">
 <pre class="text-xs font-mono text-white" style="font-family: monospace; font-size: 0.8em; color: white;">
void dfs(int v, int p = -1) {
 visited[v] = true;
 tin[v] = low[v] = timer++;
 int children = 0;

 for (int to : adj[v]) {
 if (to == p) continue;
 if (visited[to]) {
 low[v] = min(low[v], tin[to]);
 } else {
 dfs(to, v);
 low[v] = min(low[v], low[to]);
 if (low[to] >= tin[v] && p != -1)
 IS_CUTPOINT(v);
 ++children;
 }
 }
 if (p == -1 && children > 1)
 IS_CUTPOINT(v);
}
}
</pre>
 </div>
 </div>
</details>

```

```

<div class="bg-indigo-950/60 p-6 rounded-xl border" style="background-color: #2d3748; color: white; padding: 10px; border-radius: 1em; border: 1px solid #4a5568;">
 <h4 class="text-lg font-bold text-indigo-400" style="margin: 0; color: #818cf8; font-size: 1.2em;">
 <p class="text-slate-300 text-sm mb-4" style="margin: 0; color: #a6b8c7; font-size: 0.9em;">
 Это глубокий дуальный мост между ориентированными графами и неориентированными графами.
 </p>
 <ul class="list-disc list-inside text-sm text-slate-300" style="margin: 0; color: #a6b8c7; font-size: 0.9em;">
 <strong class="text-rose-400" style="color: #f44336; font-weight: bold; font-size: 0.9em;">Точкиarticulation points и показывают физическую уязвимость связности графа.
 <strong class="text-emerald-400" style="color: #2e8b57; font-weight: bold; font-size: 0.9em;">SCCs или "сильно связные компоненты" являются фундаментальными строительными блоками графов.
 Как точка сочленения рушит связность? Косарайю, свернув каждую SCC в одну супер-вершину, мы можем увидеть, как удаление одной вершины может разбить граф на несколько частей.


```



```
 </div>
 </div>
</section>`,
 },
];
```

---

## РАЗДЕЛ: ИНТЕРАКТИВНЫЕ КОМПОНЕНТЫ И СИМУЛЯТОРЫ

---

---

ФАЙЛ 27/81: src/components/ArticulationPointsViz.tsx  
КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

---

```
import React, { useState, useEffect } from "react";
import { Play, Pause, RotateCcw, Info } from "lucide-react";

interface Node {
 id: number;
 x: number;
 y: number;
 label: string;
}

interface Edge {
 u: number;
 v: number;
}

const nodes: Node[] = [
 { id: 0, x: 250, y: 50, label: "0" },
 { id: 1, x: 100, y: 150, label: "1" },
 { id: 2, x: 250, y: 250, label: "2" },
 { id: 3, x: 400, y: 150, label: "3" },
 { id: 4, x: 550, y: 150, label: "4" },
 { id: 5, x: 700, y: 50, label: "5" },
```

```

useEffect(() => {
 const newSteps: StepInfo[] = [];
 let timer = 0;
 const visited = new Set<number>();
 const ap = new Set<number>();
 const currentTin = Array(nodes.length).fill(-1);
 const currentLow = Array(nodes.length).fill(-1);

 const recordStep = (desc: string, u: number | null) => {
 newSteps.push({
 desc,
 visited: new Set(visited),
 ap: new Set(ap),
 currentNode: u,
 tin: [...currentTin],
 low: [...currentLow],
 });
 };

 recordStep("Начало DFS (Тарьян) для поиска точек со

 const dfs = (v: number, p: number = -1) => {
 visited.add(v);
 currentTin[v] = currentLow[v] = timer++;
 let children = 0;

 recordStep(`Заходим в вершину ${v}. tin[${v}]=${c

 for (const to of adj[v]) {
 if (to === p) continue;

 if (visited.has(to)) {
 currentLow[v] = Math.min(currentLow[v], curren
 recordStep(
 `Обратное ребро ${v}-${to}. Обновляем low[$
 v,
);
 } else {

```

```

 recordStep("Алгоритм завершен. Найдены все точки со
 setSteps(newSteps);
}, []));

const currentStep = steps[currentStepIndex] || {
 desc: "",
 visited: new Set(),
 ap: new Set(),
 currentNode: null,
 tin: [],
 low: [],
};

useEffect(() => {
 let interval: NodeJS.Timeout;
 if (isPlaying && currentStepIndex < steps.length -
 interval = setInterval(() => {
 setCurrentStepIndex((prev) => prev + 1);
 }, 1500);
 } else if (currentStepIndex >= steps.length - 1) {
 setIsPlaying(false);
 }
 return () => clearInterval(interval);
}, [isPlaying, currentStepIndex, steps.length]);

const togglePlay = () => setIsPlaying(!isPlaying);
const reset = () => {
 setIsPlaying(false);
 setCurrentStepIndex(0);
};

return (
 <div className="bg-slate-900 rounded-xl border bord
 <div className="bg-slate-800 p-4 border-b border-
 <h3 className="font-bold text-white flex items-
 <Info className="w-5 h-5 text-rose-400" />
 Моделирование поиска точек сочленения

```

```

 return (
 <line
 key={"edge-" + i}
 x1={u.x}
 y1={u.y}
 x2={v.x}
 y2={v.y}
 stroke={isBridge ? "#f43f5e" : "#475568"}
 strokeWidth={isBridge ? "4" : "2"}
 strokeDasharray={isBridge ? "8,4" : ""}
 />
);
 }}}

 {/* Nodes */}
 {nodes.map((node) => {
 const isCurrent = currentStep.currentNode === node.id;
 const isVisited = currentStep.visited.has(node.id);
 const isAp = currentStep.ap.has(node.id);

 const fill = isCurrent
 ? "#3b82f6"
 : isAp
 ? "#e11d48"
 : isVisited
 ? "#10b981"
 : "#1e293b";
 const stroke = isCurrent
 ? "#60a5fa"
 : isAp
 ? "#f43f5e"
 : isVisited
 ? "#34d399"
 : "#334155";
 const r = isAp ? 24 : 20;

 return (
 <g key={node.id}>

```

```

 y={node.y + r + 20}
 textAnchor="middle"
 fill="#94a3b8"
 fontSize="10px"
 >
 low:{" "}
 {currentStep.low[node.id] >= 0
 ? currentStep.low[node.id]
 : "-"}
 </text>
 </>
 })
</g>
);
}}
</svg>
</div>

<div className="bg-slate-950 p-6 border-t lg:bo
 <h4 className="text-sm font-bold text-slate-4
 Статус алгоритма
 </h4>

 <div className="bg-slate-900 border border-sl
 <p className="text-white font-medium min-h-
 {currentStep.desc}
 </p>
 </div>

 <div className="space-y-4 flex-1 overflow-y-a
 <div className="mb-4">
 <h5 className="text-xs text-slate-500 fon
 <div className="flex items-center gap-2 t
 <div className="w-3 h-3 rounded-full bg
 Точка сочленения
 </div>
 <div className="flex items-center gap-2 t
 <div className="w-3 h-3 rounded-full bg

```

```
 }}
 />
 </div>
 </div>
</div>
</div>
</div>
);
};
```

---

ФАЙЛ 28/81: src/components/BellmanFordViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

---

```
import { useState, useEffect } from 'react';
```

```
interface Node { id: string; x: number; y: number; name: string; }
```

```
interface Edge { u: string; v: string; w: number; }
```

```
interface Step {
```

```
 iteration: number;
```

```
 checkingEdge: { u: string; v: string; w: number } | null;
```

```
 distances: { [key: string]: number };
```

```
 previous: { [key: string]: string | null };
```

```
 log: string;
```

```
 hasNegativeCycle?: boolean;
```

```
 activeCodeLine: number;
```

```
}
```

```
export default function BellmanFordViz() {
```

```
 const [hasCycle, setHasCycle] = useState(false);
```

```
 const [steps, setSteps] = useState<Step[]>([]);
```

```
 const [currentStepIndex, setCurrentStepIndex] = useState(0);
```

```
 const [isPlaying, setIsPlaying] = useState(false);
```

```
 const nodes: Node[] = [
```

```
 { id: 'S', x: 60, y: 150, name: 'База (S)' },
```

```

{ line: 2, text: ' dist = {v: ∞}; dist[start] = 0;
{ line: 3, text: ' for i from 1 to V - 1: // |V|
{ line: 4, text: ' for (u, v, weight) in E:
{ line: 5, text: ' if dist[u] + weight < dist[v]:
{ line: 6, text: ' dist[v] = dist[u] + weight;
{ line: 7, text: ' for (u, v, weight) in E: // Проверка
{ line: 8, text: ' if dist[u] + weight < dist[v]:
{ line: 9, text: ' return "ОБНАРУЖЕН ОТРИЦАТЕЛЬНЫЙ ЦИКЛ";
{ line: 10, text: ' return dist' },
];

```

```

useEffect(() => {
 const simulationSteps: Step[] = [];
 let dist: { [key: string]: number } = { S: 0, A: 999 };
 let prev: { [key: string]: string | null } = { S: null };

 simulationSteps.push({
 iteration: 0, checkingEdge: null,
 distances: { ...dist }, previous: { ...prev },
 log: ' Фура заведена. Начинаем с Базы (S). Расстояние до А: 999',
 activeCodeLine: 2,
 });

 const vCount = nodes.length;
 for (let iter = 1; iter < vCount; iter++) {
 let anyUpdate = false;
 for (const edge of edges) {
 const uDist = dist[edge.u];
 const vDist = dist[edge.v];

 let updated = false;
 if (uDist !== 999 && uDist + edge.w < vDist) {
 dist = { ...dist, [edge.v]: uDist + edge.w };
 prev = { ...prev, [edge.v]: edge.u };
 updated = true;
 anyUpdate = true;
 }
 }
 }
}

```

```

 distances: { ...dist }, previous: { ...prev }
 log: ' Проверка завершена. Ни одно ребро не
 activeCodeLine: 10,
 });
 }
}

setSteps(simulationSteps);
setCurrentStepIndex(0);
setIsPlaying(false);
}, [hasCycle]));

useEffect(() => {
 let timer: any = null;
 if (isPlaying && currentStepIndex < steps.length -
 timer = setTimeout(() => setCurrentStepIndex((p)
 } else if (currentStepIndex >= steps.length - 1) {
 setIsPlaying(false);
 }
 return () => clearTimeout(timer);
}, [isPlaying, currentStepIndex, steps.length]);

const currentStep = steps[currentStepIndex] || null;
if (!currentStep) return <div>Загрузка...</div>;

return (
 <div className="bg-slate-800/90 p-6 rounded-2xl border
 <div className="flex flex-wrap items-center justify
 <div className="flex items-center gap-3">
 <div className="p-2.5 bg-blue-600 text-white

 </div>
 <div>
 <h3 className="text-2xl font-bold text-white
 <p className="text-sm text-blue-300">Полный
 </div>
 </div>
 </div>
)

```



```

<path d="M0,0 L6,3 L0,6 z" fill="#f59e0b" />
 <marker id="arrow-bf-cycle" markerWidth="1"
path d="M0,0 L6,3 L0,6 z" fill="#e11d48" />
</defs>

```

```

(edges.map((edge, idx) => {
 const uNode = nodes.find((n) => n.id === u);
 const vNode = nodes.find((n) => n.id === v);
 const isChecking = currentStep.checkingEdges;
 const isNeg = edge.w < 0;
 const isCyclePart = hasCycle && ((edge.u === u && edge.v === 'D'));

```

```

 let strokeColor = '#475569';
 let marker = 'url(#arrow-bf-normal)';
 if (isChecking) { strokeColor = '#f59e0b';
 else if (currentStep.hasNegativeCycle && isCyclePart) {

```

```

 return (
 <g key={idx}>
 <line x1={uNode.x} y1={uNode.y} x2={vNode.x} y2={vNode.y} stroke={strokeColor} stroke-width={currentStep.hasNegativeCycle && isCyclePart ? 4 : 2} marker={marker} />
 <circle cx={({uNode.x + vNode.x} / 2)} cy={({uNode.y + vNode.y} / 2)} r={1} fill={isChecking ? '#f59e0b' : isNeg ? '#f43f5e' : '#e11d48'} stroke={strokeColor} />
 <text x={({uNode.x + vNode.x} / 2)} y={({uNode.y + vNode.y} / 2)} text={isChecking ? 'Checking' : isNeg ? 'Neg' : 'Cycle'} fontSize="11" fontWeight="bold" />
 </g>
);
 });
}

```

```

(nodes.map((node) => {
 const dist = currentStep.distances[node.id];
 const isCheckingNode = currentStep.checkingNodes[node.id];

```

```

 return (

```

```

 </div>
 </div>
 </div>

 <div className="flex flex-col lg:flex-row gap-6">
 {/* Таблица расстояний */}
 <div className="w-full lg:w-1/2 bg-rose-950/10">
 <div>
 <h4 className="text-lg font-bold text-rose-950">
 Таблица расстояний
 </h4>
 <div className="grid grid-cols-4 gap-2 text-slate-400">
 {nodes.map(n => {
 const d = currentStep.distances[n.id];
 const isTgt = currentStep.checkingEdge === n.id;
 return (
 <div key={n.id} className={`p-2 rounded-lg
 ${isTgt ? 'border-slate-700/60 bg-slate-800/40' : ''}`}>
 <div className="text-xs text-slate-500">
 {n.id}
 </div>
 <div className={`font-mono font-bold text-sm`}
 >
 {d}
 </div>
 </div>
);
 })}
 </div>
 </div>
 </div>
 </div>

 {/* Код алгоритма */}
 <div className="w-full lg:w-1/2 bg-slate-900/60">
 <div>
 <h4 className="text-lg font-bold text-slate-400">
 Код алгоритма
 </h4>
 <div className="bg-slate-950/80 rounded-lg p-4">
 {codeLines.map(item => (
 <div key={item.line} className={`py-1.5 px-4`}
 >
 {item.line}

 {item.indent}

 {item.code}

 </div>
))}
 </div>
 </div>
 </div>
 </div>
</div>

```

```
 activeLine: number;
 queue: number[];
 visited: number[];
 currentNode: number | null;
 logs: string;
}

const generateBfsSteps = (): Step[] => {
 const steps: Step[] = [];
 const queue: number[] = [];
 const visited = new Set<number>();

 // Init
 steps.push({
 activeLine: 1,
 queue: [],
 visited: [],
 currentNode: null,
 logs: "Инициализация: начинаем с корня (0).",
 });

 queue.push(0);
 visited.add(0);
 steps.push({
 activeLine: 2,
 queue: [...queue],
 visited: Array.from(visited),
 currentNode: null,
 logs: "Добавляем стартовую вершину 0 в очередь и по",
 });

 while (queue.length > 0) {
 steps.push({
 activeLine: 4,
 queue: [...queue],
 visited: Array.from(visited),
 currentNode: null,
 logs: `Очередь не пуста, элементов: ${queue.length}
```

```

 queue: [...queue],
 visited: Array.from(visited),
 currentNode: curr,
 logs: `Соседей нет или все уже посещены.`
 }));
 }
}

steps.push({
 activeLine: 12,
 queue: [],
 visited: Array.from(visited),
 currentNode: null,
 logs: "Очередь пуста. Алгоритм завершен!"
});

return steps;
};

const STEPS = generateBfsSteps();

export default function BfsViz() {
 const [stepIdx, setStepIdx] = useState(0);
 const [isPlaying, setIsPlaying] = useState(false);

 const step = STEPS[stepIdx];

 const handleNext = useCallback(() => {
 setStepIdx(prev => Math.min(prev + 1, STEPS.length - 1));
 }, []);

 // @ts-expect-error зарезервировано под кнопку «назад»
 const handlePrev = useCallback(() => {
 setStepIdx(prev => Math.max(prev - 1, 0));
 }, []);

 useEffect(() => {
 if (!isPlaying) return;

```

```

 >
 <StepForward className="w-5 h-5" />
 </button>
 </div>
 </div>

 <div className="grid grid-cols-1 lg:grid-cols-2 gap-4">
 { /* Визуал графа */ }
 <div className="bg-slate-800 border border-slate-600 border-radius-50px">
 <svg className="w-full h-full min-h-[350px]" >
 { /* Рёбра */ }
 { EDGES.map((e, idx) => {
 const n1 = NODES.find(n => n.id === e.from)
 const n2 = NODES.find(n => n.id === e.to)
 const isVisited = step.visited.includes(n1.id)
 return (
 <line
 key={idx}
 x1={n1.x} y1={n1.y} x2={n2.x} y2={n2.y}
 className={`transition-all duration-500 ${
 isVisited ? 'stroke-slate-600' : 'stroke-slate-400'
 }`}
 />
)
 }) }
 </svg>
 </div>
 { /* Вершины */ }
 { NODES.map(n => {
 const isCurrent = step.currentNode === n.id
 const isInQueue = step.queue.includes(n.id)
 const isVisited = step.visited.includes(n.id)

 let fill = '#1e293b'; // slate-800
 let stroke = '#475569'; // slate-600
 let scale = 1;
 void scale;

 if (isCurrent) {

```

```

 { /* Код и Очередь */ }
 <div className="flex flex-col gap-4">
 <div className="bg-slate-900 border border-slate-800 p-4">
 <div className="bg-slate-800 text-slate-400 p-4 font-mono se tracking-wider">
 bfs(graph, start)
 Mar {step}
 </div>
 <div className="p-4">
 {[
 { line: 1, code: 'queue = Queue()' },
 { line: 2, code: 'queue.push(start); visited.add(start)' },
 { line: 3, code: '' },
 { line: 4, code: 'while not queue.isEmpty():' },
 { line: 5, code: ' node = queue.pop()' },
 { line: 6, code: ' for neighbor in graph[node]:' },
 { line: 7, code: ' if neighbor not in visited:' },
 { line: 8, code: ' visited.add(neighbor)' },
 { line: 9, code: ' queue.push(neighbor)' },
]}.map((cl) => {
 const isActive = step.activeLine === cl.line
 return (
 <div key={cl.line} className={`px-2 py-2 border-l-2 border-blue-400 -ml-0.5` : `text-${isActive ? 'blue' : 'slate'}-400`} >

 {cl.code}
 </div>
)
 })
 </div>
 </div>

 <div className="bg-slate-800/80 border border-slate-700 p-4">
 <div className="text-sm font-bold text-emerald-400">
 Состояние Очереди (Queue):


```

```

 caption: ' Добрый — только позитив!',
 borderColor: 'border-blue-500/30',
 captionColor: 'text-blue-400',
 },
 'bellman-ford': {
 src: bellmanImg,
 alt: 'Фура по болоту',
 caption: ' Тяжёлая, но ей пофиг на ямы!',
 borderColor: 'border-rose-500/30',
 captionColor: 'text-rose-400',
 },
 floyd: {
 src: floydImg,
 alt: 'Все ко Всем Флойд',
 caption: ' Все связаны со всеми!',
 borderColor: 'border-emerald-500/30',
 captionColor: 'text-emerald-400',
 },
};

export default function ChapterImage({ vizType }: { vizType: string }) {
 const info = imageMap[vizType];
 if (!info) return null;

 return (
 <div>
 <div className={`rounded-2xl overflow-hidden border-2`} >

 </div>
 <p className={`text-center text-xs mt-2 font-bold`} >{info.caption}</p>
 </div>
);
}
```

```

 onClick={() => next && onGo(next.id)}
 title={next ? next.title : "Это последняя тема"}
 className="flex items-center gap-1 px-2.5 py-1
 :bg-slate-700 enabled:hover:text-white disabled:opacity-50"
 >
 Вперёд
 <ChevronRight className="w-4 h-4" />
 </button>
 </div>
);
}

return (
 <nav className="grid grid-cols-1 sm:grid-cols-2 gap-4" >
 {prev ? (
 <button
 onClick={() => onGo(prev.id)}
 className="group text-left p-4 rounded-xl bg-white
 transition-all"
 >
 <div className="flex items-center gap-1.5 text-slate-700 mb-1">
 <ChevronLeft className="w-3.5 h-3.5" /> Назад
 </div>
 <div className="text-sm font-bold text-slate-700">
 {prev.title}
 </div>
 </button>
) : (
 <div className="hidden sm:block" />
)}
 {next && (
 <button
 onClick={() => onGo(next.id)}
 className="group text-right p-4 rounded-xl bg-white
 transition-all sm:col-start-2"
 >
 <div className="flex items-center justify-end text-indigo-400 mb-1">

```



```

const [anchor, setAnchor] = useState<HintAnchor | null>(null);
const hideTimer = useRef<number | null>(null);
const [saving, setSaving] = useState<"idle" | "work">("idle");

useTermHints(contentRef, [chapter.id]);

const cancelHide = useCallback(() => {
 if (hideTimer.current) {
 window.clearTimeout(hideTimer.current);
 hideTimer.current = null;
 }
}, []);

const scheduleHide = useCallback(() => {
 cancelHide();
 hideTimer.current = window.setTimeout(() => setAnchor(anchor), 2500);
}, [cancelHide]);

useEffect(() => {
 setAnchor(null);
 setSaving("idle");
}, [chapter.id]);

const saveHtml = useCallback(async () => {
 setSaving("work");
 try {
 await downloadChapterHtml(chapter);
 setSaving("done");
 window.setTimeout(() => setSaving("idle"), 2500);
 } catch {
 setSaving("idle");
 }
}, [chapter]);

const open = useCallback(
 (el: HTMLElement) => {
 const termId = el.dataset.termId;
 if (!termId) return;
 },
);

```

```

<button
 onClick={saveHtml}
 disabled={saving === "work"}
 title="Сохранить эту тему одним автономным файлом"
 className="flex items-center gap-1.5 text-x-300 hover:text-white hover:border-emerald-500"
>
 {saving === "work" ? (
 <Loader2 className="w-3.5 h-3.5 animate-spin" />
) : saving === "done" ? (
 <Check className="w-3.5 h-3.5 text-emerald-500" />
) : (
 <Download className="w-3.5 h-3.5" />
)}
 {saving === "done" ? "Файл сохранён" : "Скачать"}
</button>
<button
 onClick={() => window.print()}
 title="Распечатать или сохранить в PDF средствами браузера"
 className="flex items-center gap-1.5 text-x-300 hover:text-white hover:border-indigo-500"
>
 <Printer className="w-3.5 h-3.5" /> Печать
</button>

 </div>

<div
 ref={contentRef}
 className="prose prose-invert max-w-none"
 onMouseOver={handlePointerOver}
 onMouseOut={handlePointerOut}
 onClick={handleClick}
 onFocus={handleFocus}
 dangerouslySetInnerHTML={{ __html: chapter.content }}
/>

<p className="mt-6 flex items-start gap-2 text-x-300">

```

```

 onCardEnter={cancelHide}
 onCardLeave={scheduleHide}
 />
 </>
);
};

```

ФАЙЛ 33/81: src/components/DfsBridgesSimulator.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

```

import { useState, useEffect, useRef } from "react";
import { Undo, Play, Pause, SkipForward, RefreshCw } from "lucide-react";

```

```

interface CodeLine {
 line: string;
 highlight: string;
}

```

```

const CODE_LINES: CodeLine[] = [
 { line: "void dfs(int v, int p = -1) {", highlight: "normal" },
 { line: " visited[v] = true;", highlight: "normal" },
 { line: " tin[v] = low[v] = timer++;", highlight: "normal" },
 { line: " for (int to : adj[v]) {", highlight: "normal" },
 { line: " if (to == p) continue;", highlight: "normal" },
 { line: " if (visited[to]) {", highlight: "normal" },
 { line: " low[v] = min(low[v], tin[to]);", highlight: "normal" },
 { line: " } else {", highlight: "normal" },
 { line: " dfs(to, v);", highlight: "normal" },
 { line: " low[v] = min(low[v], low[to]);", highlight: "normal" },
 { line: " }", highlight: "normal" },
 { line: " if (low[to] > tin[v]) {", highlight: "normal" },
 { line: " // ЭТО МОСТ!", highlight: "normal" },
 { line: " }", highlight: "normal" },

```

```

const DFS_STEPS: DfsStep[] = [
 {
 currentNode: 0, visitedIndices: [0],
 tin: {0:1}, low: {0:1}, stack: [0], bridges: [],
 activeEdge: null, backEdge: null,
 log: "Входим в A. tin[A]=1, low[A]=1.",
 activeCodeLine: [0,1,2]
 },
 {
 currentNode: 1, visitedIndices: [0,1],
 tin: {0:1,1:2}, low: {0:1,1:2}, stack: [0,1], bridges: [],
 activeEdge: [0,1], backEdge: null,
 log: "Идём (A,B). B впервые. tin[B]=2, low[B]=2.",
 activeCodeLine: [4,5,7,10,11]
 },
 {
 currentNode: 6, visitedIndices: [0,1,6],
 tin: {0:1,1:2,6:3}, low: {0:1,1:2,6:3}, stack: [0,1,6],
 activeEdge: [1,6], backEdge: null,
 log: "Из B идём в G. tin[G]=3, low[G]=3.",
 activeCodeLine: [4,5,7,10,11]
 },
 {
 currentNode: 1, visitedIndices: [0,1,6],
 tin: {0:1,1:2,6:3}, low: {0:1,1:2,6:3}, stack: [0,1,6],
 activeEdge: null, backEdge: null,
 log: "У G нет соседей (кроме B). Возврат. low[G]=3 : ",
 activeCodeLine: [13,14,15]
 },
 {
 currentNode: 2, visitedIndices: [0,1,6,2],
 tin: {0:1,1:2,6:3,2:4}, low: {0:1,1:2,6:3,2:4}, stack: [0,1,6,2],
 activeEdge: [1,2], backEdge: null,
 log: "Из B идём в C. tin[C]=4, low[C]=4.",
 activeCodeLine: [4,5,7,10,11]
 },
 {

```

```

 log: "Возврат F→E. low[E]=min(6,5)=5. (E,F) – не мост
 activeCodeLine: [11,13,16,17]
},
{
 currentNode: 3, visitedIndices: [0,1,6,2,3,4,5],
 tin: {0:1,1:2,6:3,2:4,3:5,4:6,5:7}, low: {0:1,1:2,6:3,2:4,3:5,4:6,5:7},
 activeEdge: null, backEdge: null,
 log: "Возврат E→D. low[D]=min(6,5)=5. (D,E) – не мост
 activeCodeLine: [11,13,16,17]
},
{
 currentNode: 2, visitedIndices: [0,1,6,2,3,4,5],
 tin: {0:1,1:2,6:3,2:4,3:5,4:6,5:7}, low: {0:1,1:2,6:3,2:4,3:5,4:6,5:7},
 activeEdge: null, backEdge: null,
 log: "Возврат D→C. low[5] = 5 > tin[2] = 4! (C,D) – мост
 activeCodeLine: [13,14,15,16,17]
},
{
 currentNode: 1, visitedIndices: [0,1,6,2,3,4,5],
 tin: {0:1,1:2,6:3,2:4,3:5,4:6,5:7}, low: {0:1,1:1,6:3,2:4,3:5,4:6,5:7},
 activeEdge: null, backEdge: null,
 log: "Возврат C→B. low[B]=min(low[B],low[C])=1. (B,C) – мост
 activeCodeLine: [11,13,16,17]
},
{
 currentNode: 0, visitedIndices: [0,1,6,2,3,4,5],
 tin: {0:1,1:2,6:3,2:4,3:5,4:6,5:7}, low: {0:1,1:1,6:3,2:4,3:5,4:6,5:7},
 activeEdge: null, backEdge: null,
 log: "Возврат B→A. DFS завершён! Мосты: (B,G) и (C,D). Алгоритм
 activeCodeLine: [11,13,16,17,18]
},
{
 currentNode: null, visitedIndices: [0,1,6,2,3,4,5],
 tin: {0:1,1:2,6:3,2:4,3:5,4:6,5:7}, low: {0:1,1:1,6:3,2:4,3:5,4:6,5:7},
 activeEdge: null, backEdge: null,
 log: "Готово! Найдены мосты: (B,G) и (C,D). Алгоритм
 activeCodeLine: []
},

```

```

disabled={dfsIdx === DFS_STEPS.length-1}
className="p-1.5 hover:bg-slate-700 rounded
<SkipForward className="h-3.5 w-3.5" />
</button>
<button onClick={() => { setDfsAuto(false); s
 className="p-1.5 hover:bg-slate-700 rounded
 <RefreshCw className="h-3.5 w-3.5" />
</button>
</div>
</div>

<div className="grid grid-cols-1 md:grid-cols-5 g
 {/* Graph visualization */}
 <div className="md:col-span-3 bg-slate-950 roun
 <div className="absolute inset-0 bg-[radial-g
 >
 <svg className="w-full h-[280px] z-10 relative
 {GRAPH_EDGES.map(e => {
 const u = GRAPH_NODES.find(n => n.id ===
 const v = GRAPH_NODES.find(n => n.id ===
 const bridge = step.bridges.includes(e.key
 const activeTree = step.activeEdge && (
 (step.activeEdge[0]===e.u && step.active
 (step.activeEdge[0]===e.v && step.active
));
 const activeBack = step.backEdge && (
 (step.backEdge[0]===e.u && step.backEdge
 (step.backEdge[0]===e.v && step.backEdge
));
 let sc = "#475569", sw = 2, dash = "none"
 if (bridge) { sc = "#eab308"; sw = 4; }
 else if (activeTree) { sc = "#10b981"; sw
 else if (activeBack) { sc = "#f43f5e"; sw
 else if (step.visitedIndices.includes(e.u
 return <line key={e.key} x1={u.x} y1={u.y
 stroke={sc} strokeWidth={sw} strokeDash
 className="transition-all duration-500"
 }}}

```

```

 { /* Code panel + stack */ }
 <div className="md:col-span-2 space-y-3">
 { /* Code panel */ }
 <div className="bg-slate-950 rounded-lg border" style={{ border: '1px solid #333' }}>
 <div className="bg-slate-900 px-3 py-1.5 text-slate-400">
 >
 <div className="p-2 overflow-x-auto">
 <pre className="text-[10px] font-mono leading-relaxed" style={{ margin: 0, padding: 0 }}>
 {CODE_LINES.map((cl, i) => {
 const hl = step.activeCodeLine.includes(cl.line)
 return <div key={i}
 className={`px-1 transition-all duration-200 ${
 hl ? "bg-indigo-600/20 text-indigo-400" : "text-slate-400"
 }`} >

 {cl.line} || ' '
 </div>
)}
 </pre>
 </div>
 </div>
 </div>

 { /* Stack */ }
 <div className="bg-slate-900/50 rounded-xl border" style={{ border: '1px solid #333' }}>
 <div className="text-[10px] font-bold text-slate-400" style={{ padding: 5px 10px 0 10px }}>
 Стек DFS
 {step.stack.join(' ')}
 </div>
 <div className="flex flex-col-reverse gap-1" style={{ padding: 0 10px 10px 10px }}>
 {step.stack.length === 0
 ? <div className="text-[10px] text-slate-400" style={{ padding: 5px 0 0 0 }}>
 : step.stack.map((id, i) => {
 <div key={id} className={`px-2 py-0.5 text-[10px] transition-all duration-200 ${
 i === step.stack.length - 1
 ? "bg-emerald-600/20 text-emerald-400"
 : "bg-slate-900 text-slate-400"
 }`} >Вершина {GRAPH_NODES.find(n=>n.id === id).name}</div>

 </div>
)}
 </div>
 </div>
 </div>
)

```

```

 log: string;
 activeCodeLine: number;
}

export default function DijkstraViz() {
 const nodes: Node[] = [
 { id: 'A', x: 60, y: 150, name: 'Пиццерия (A)' },
 { id: 'B', x: 160, y: 60, name: 'Дом 1 (B)' },
 { id: 'C', x: 160, y: 240, name: 'Парк (C)' },
 { id: 'D', x: 340, y: 60, name: 'Офис (D)' },
 { id: 'E', x: 340, y: 240, name: 'Дом 2 (E)' },
 { id: 'F', x: 450, y: 150, name: 'ТЦ (F)' },
 { id: 'G', x: 250, y: 150, name: 'Метро (G)' },
];

 const edges: Edge[] = [
 { u: 'A', v: 'B', w: 4 },
 { u: 'A', v: 'C', w: 2 },
 { u: 'C', v: 'B', w: 1 },
 { u: 'C', v: 'G', w: 3 },
 { u: 'C', v: 'E', w: 8 },
 { u: 'B', v: 'G', w: 4 },
 { u: 'B', v: 'D', w: 5 },
 { u: 'G', v: 'D', w: 1 },
 { u: 'G', v: 'E', w: 2 },
 { u: 'D', v: 'F', w: 3 },
 { u: 'E', v: 'F', w: 1 },
];

 const adjList: { [key: string]: { v: string; w: number } } =
 nodes.forEach(n => (adjList[n.id] = []));
 edges.forEach(e => {
 adjList[e.u].push({ v: e.v, w: e.w });
 });

 const codeLines = [
 { line: 1, text: 'function dijkstra(graph, start):' },
 { line: 2, text: ' dist = {v: ∞}; dist[start] = 0' },
];
}

```



```

 if (!u) break;

 visited[u] = true;
 simulationSteps.push({
 currentNode: u, checkingEdge: null,
 distances: { ...dist }, visited: { ...visited },
 log: ` Выбираем вершину ${u} с мин. непосещённой
 activeCodeLine: 5,
 });

 const neighbors = edges.filter(e => e.u === u);
 for (const edge of neighbors) {
 const v = edge.v;
 if (visited[v]) continue;

 simulationSteps.push({
 currentNode: u, checkingEdge: { u, v },
 distances: { ...dist }, visited: { ...visited },
 log: ` Смотрим соседа ${v}. Текущее расстояние
 ${dist[u] + edge.w}.`,
 activeCodeLine: 8,
 });

 if (dist[u] + edge.w < dist[v]) {
 dist[v] = dist[u] + edge.w;
 prev[v] = u;
 simulationSteps.push({
 currentNode: u, checkingEdge: { u, v },
 distances: { ...dist }, visited: { ...visited },
 log: ` Улучшение (Релаксация)! Новое кратчайшее
 activeCodeLine: 9,
 });
 }
 }
 }
}

simulationSteps.push({
 currentNode: null, checkingEdge: null,

```

```

<div className="flex items-center gap-2">
 <button
 onClick={() => { setIsPlaying(false); setCu
 className="flex items-center gap-1 bg-slate
 ion-colors"
 >
 Сброс
 </button>
 <button
 onClick={() => setIsPlaying(!isPlaying)}
 className={`flex items-center gap-1 px-4 py
 isPlaying ? 'bg-amber-600 hover:bg-amber-!
 }`}
 >
 {isPlaying ? ' Пауза' : '▶ Пуск'}
 </button>
 <button
 onClick={() => { setIsPlaying(false); if (c
 disabled={currentStepIndex >= steps.length
 className="flex items-center gap-1 bg-indigo
 px-3 py-2 rounded-lg text-sm font-medium tr
 >
 ⏮
 </button>
</div>
</div>

<div className="flex flex-col lg:flex-row gap-6 m
 {/* Граф */}
 <div className="w-full lg:w-2/3 bg-indigo-950/2
 stify-center min-h-[400px]">
 <div className="absolute top-3 left-3 bg-indi
 ont-bold shadow-sm">
 ⏮ {currentStepIndex + 1} из {steps.length
 </div>

 <svg className="w-full h-auto max-h-[500px]"
 <defs>

```

```

 stroke={isChecking ? '#f59e0b' : '#f8
 />
 <text
 x={({uNode.x + vNode.x} / 2} y={({uNode
 fill={isChecking ? '#f59e0b' : '#f8
 fontSize="11" fontWeight="bold" text
 >
 {edge.w}
 </text>
 </g>
);
}})

{/* Вершины */}
{nodes.map((node) => {
 const isCurrent = currentStep.currentNode
 const isVisited = currentStep.visited[node.id]
 const dist = currentStep.distances[node.id]

 return (
 <g key={node.id} className="transition-
 <circle
 cx={node.x} cy={node.y}
 r={isCurrent ? 22 : 18}
 fill={isCurrent ? '#4f46e5' : isVisited ? '#f59e0b' : '#f8
 stroke={isCurrent ? '#a5b4fc' : isVisited ? '#f59e0b' : '#f8
 strokeWidth={isCurrent ? 4 : 2}
 className="transition-all duration-300"
 />
 <text x={node.x} y={node.y + 5} fill="white"
 {node.id}
 </text>
 <text x={node.x} y={node.y - 28} fill="white"
 "middle" className="bg-indigo-950 px-1 py-0.5"
 {dist === 999 ? '∞' : `d=${dist}`}
 </text>
 <text x={node.x} y={node.y + 35} fill="white"
 {node.name.split(' ')[0]}

```

```

 <span className={isCheckingTh
an>

);
 }}}
</div>
</div>
);
}}}
</div>
</div>
</div>

<div className="flex flex-col lg:flex-row gap-6">
 {/* Таблица расстояний */}
 <div className="w-full lg:w-1/2 bg-rose-950/10">
 <div>
 <h4 className="text-lg font-bold text-rose-950">
 Таблица расстояний
 </h4>
 <div className="overflow-x-auto">
 <table className="w-full text-left border-collapse">
 <thead>
 <tr className="border-b border-rose-950">
 <th className="py-2 px-3">Вершина</th>
 <th className="py-2 px-3">Расстояние</th>
 <th className="py-2 px-3">Из (пред. шаг)</th>
 </tr>
 </thead>
 <tbody className="text-sm">
 {nodes.map((n) => {
 const dist = currentStep.distances[n.id];
 const prev = currentStep.previous[n.id];
 const vis = currentStep.visited[n.id];
 const isCurr = currentStep.currentNode === n.id;

 return (
 <tr key={n.id} className={`border-b border-rose-950
 isCurr ? 'bg-indigo-950/60 for
 `}>

```

```
 </div>
 </div>
 </div>
 </div>
);
}
```

---

ФАЙЛ 35/81: src/components/DPVisualizer.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

---

```
import { useState, useEffect } from 'react';
import { Play, Pause, RotateCcw, ChevronLeft, ChevronRight } from 'lucide-react';
```

```
interface DPStep {
 id: number;
 n: number;
 action: 'call' | 'memo_hit' | 'base_case' | 'calc' | 'return';
 desc: string;
 codeLine: number;
 memoState: { [key: number]: number };
}
```

```
const DP_CODE_LINES = [
 /* 1 */ "function fibMemo(n, memo = {}) {",
 /* 2 */ " if (n in memo) return memo[n];",
 /* 3 */ " if (n <= 2) return 1;",
 /* 4 */ " memo[n] = fibMemo(n - 1, memo) + fibMemo(n - 2, memo);",
 /* 5 */ " return memo[n];",
 /* 6 */ "}"
];
```

```
export function DPVisualizer() {
 const [targetN, setTargetN] = useState<number>(5);
 const [steps, setSteps] = useState<DPStep[]>([]);
 const [currentStepIndex, setCurrentStepIndex] = useState(0);
```

```

 memoState: { ...memoMap }
 });
 return 1;
}

generatedSteps.push({
 id: stepId++,
 n,
 action: 'call',
 desc: `Вычисляем рекурсивно: fibMemo(${n - 1})`,
 codeLine: 4,
 memoState: { ...memoMap }
});

const res1 = simulateFib(n - 1);
const res2 = simulateFib(n - 2);
memoMap[n] = res1 + res2;

generatedSteps.push({
 id: stepId++,
 n,
 action: 'calc',
 desc: `Сохраняем в кэш: memo[${n}] = ${res1} + ${res2}`,
 codeLine: 5,
 memoState: { ...memoMap }
});

return memoMap[n];
}

const finalRes = simulateFib(targetN);

generatedSteps.push({
 id: stepId++,
 n: targetN,
 action: 'done',
 desc: `Расчет завершен! fibMemo(${targetN}) = ${finalRes}`,
 codeLine: 6,

```

```

<div className="flex flex-wrap items-center gap-2" style={{width: 1000px; height: 100px; background-color: #f0f0f0; border: 1px solid #ccc; padding: 10px; margin: 10px auto;}}
 <div className="flex items-center gap-2 bg-slate-500 text-white" style={{width: 100px; height: 40px; border: 1px solid #333; margin-bottom: 5px;}}
 N:
 <select
 value={targetN}
 onChange={(e) => setTargetN(Number(e.target.value))}
 disabled={isPlaying}
 className="bg-transparent text-white font-mono"
 >
 <option value={4}>N = 4</option>
 <option value={5}>N = 5</option>
 <option value={6}>N = 6</option>
 <option value={7}>N = 7</option>
 </select>
 </div>

 <div className="flex items-center bg-slate-950 text-white" style={{width: 100px; height: 40px; border: 1px solid #333; margin-bottom: 5px;}}
 <button
 onClick={() => { setIsPlaying(false); setTargetN(4); }}
 className="p-2 text-slate-400 hover:text-white"
 title="Сбросить"
 >
 <RotateCcw className="w-4 h-4" />
 </button>
 <button
 onClick={() => { setIsPlaying(false); setTargetN(4); }}
 disabled={currentStepIndex === 0}
 className="p-2 text-slate-400 hover:text-white"
 title="Шаг назад"
 >
 <ChevronLeft className="w-5 h-5" />
 </button>
 <button
 onClick={() => setIsPlaying(!isPlaying)}
 className={`flex items-center gap-1.5 px-2 py-1 ${isPlaying ? 'bg-amber-500 text-slate-950' : 'bg-slate-950 text-white'} hover:opacity-80`}
 >
 {isPlaying ? 'Пауза' : 'Воспроизвести'}
 </button>
 </div>
</div>

```

```


 <span className={`text-xl md:text-2xl font-medium`}
 {isCached ? val : '0'}

 {num <= 2 &&
 </div>
);
 }}}
</div>

{currentStep && (
 <div className="mt-6 pt-4 border-t border-slate-200">
 <div className="text-sm font-medium text-slate-500">

 {currentStep.desc}
 </div>
 <div className="text-xs font-mono bg-slate-100 p-2">
 Текущий N: <strong className="text-emerald-500">{currentStep.n}
 </div>
 </div>
)}
</div>

{/* Tabs */}
<div className="flex items-center gap-2 border-b border-slate-200">
 <button
 onClick={() => setActiveTab('table')}
 className={`flex items-center gap-2 px-6 py-3 font-medium`}
 activeTab === 'table'
 ? 'border-emerald-500 text-emerald-400 bg-emerald-50'
 : 'border-transparent text-slate-400 hover:text-slate-500'
 >
 <Eye className="w-4 h-4" />
 Степ-бай-степ состояние
 </button>
 <button
 onClick={() => setActiveTab('code')}

```



```

 const lineNum = idx + 1;
 const isActive = currentStep?.codeLine === lineNum;
 return (
 <div
 key={lineNum}
 className={`flex items-center px-4 py-2
 ${isActive ? 'bg-emerald-600/20 border-l-4 border-emerald-600' : 'text-slate-300 hover:bg-slate-800'}
 `}
 >
 {lineNum}
 {line}
 </div>
);
 }
}
</pre>
<div className="p-6 bg-slate-900/50 border-l-4 border-emerald-600">
 {currentStep?.desc}
</div>
</div>
)}
</div>

{/* Progress Bar */}
<div className="mt-8 pt-6 border-t border-slate-800">
 <div className="flex items-center justify-between">
 Прогресс выполнения
 Шаг {currentStepIndex + 1} из {steps.length}
 </div>
 <div className="h-2 bg-slate-950 rounded-full overflow-hidden">
 <div
 className="h-full bg-gradient-to-r from-emerald-500 to-emerald-600"
 style={{ width: `${((currentStepIndex + 1) / steps.length) * 100}%` }}
 ></div>
 </div>
</div>
</div>

```

```

const clickC1 = (vId: number) => {
 if (c1Done && c1Path.length > 0) return;
 if (c1Path.length === 0) {
 setC1Path([vId]);
 setC1Msg("Старт! Кликай соседние вершины, чтобы пр
 return;
 }
 const last = c1Path[c1Path.length - 1];
 const edge = C1_EDGES.find(e => (e.u === last && e.v === vId));
 if (!edge) {
 setC1Msg("Нет ребра между этими вершинами! Выбери
 return;
 }
 const key = `${Math.min(last, vId)}-${Math.max(last, vId)}`;
 if (c1VisitedEdges.includes(key)) {
 setC1Msg("Это ребро уже пройдено! Эйлер не прощае
 setC1Done(true); return;
 }
 const nextEdges = [...c1VisitedEdges, key];
 const nextPath = [...c1Path, vId];
 setC1VisitedEdges(nextEdges);
 setC1Path(nextPath);

 if (nextEdges.length === C1_EDGES.length) {
 const startDegree = C1_EDGES.filter(e => e.u === last).length;
 const endDegree = C1_EDGES.filter(e => e.v === vId).length;
 const isCycle = startDegree % 2 === 0 && endDegree % 2 === 0;
 if (isCycle && nextPath[0] === vId) {
 setC1Msg(" ЭЙЛЕРОВ ЦИКЛ! Все рёбра пройдены, ф
 } else {
 setC1Msg(` ЭЙЛЕРОВ ПУТЬ! Все рёбра пройдены! С
 }
 setC1Done(true);
 } else {
 setC1Msg(`Пройдено рёбер: ${nextEdges.length} / $
 }
};

```

```

 </svg>
 <div className="absolute top-2 left-2 bg-slate-900
 Pebep: {c1VisitedEdges.length}/{C1_EDGES.length}
 </div>
 </div>

 <div className={`p-3 rounded-xl border text-sm ${
 c1Done && c1VisitedEdges.length === C1_EDGES.length
 ? "bg-emerald-950/40 border-emerald-500/30 text-emerald-500"
 : c1Done
 ? "bg-rose-950/40 border-rose-500/30 text-rose-500"
 : "bg-slate-900/50 border-slate-700 text-slate-500"
 }`}>
 {c1Msg}
 </div>
 </div>
);
}

```

ФАЙЛ 37/81: src/components/ExpressQuiz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```

import React, { useState } from 'react';
import { CheckCircle2, XCircle, Award, RotateCcw, AlertTriangle } from 'lucide-react';

interface Question {
 id: number;
 question: string;
 options: string[];
 correctAnswer: number;
 explanation: string;
}

const quizQuestions: Question[] = [
 {

```

```

 делить вычисления на GPU или многопроцессорных кл
 },
 {
 id: 4,
 question: 'В чем главная суть Динамического програм
 options: [
 'Писать код очень быстро, динамично нажимая на кл
 'Запоминать (кешировать) результаты уже решенных
 'Использовать динамическую типизацию как в Python
 'Постоянно менять требования к проекту в процессе
],
 correctAnswer: 1,
 explanation: 'Точно! Главный принцип ДП – мемоизаци
 },
 {
 id: 5,
 question: 'Какая структура данных идеально помогает
 options: [
 'СНМ (Система непересекающихся множеств / Disjoin
 'Стек (LIFO).',
 'Красно-черное дерево.',
 'Хэш-таблица со списками коллизий.'
],
 correctAnswer: 0,
 explanation: 'Блестяще! СНМ (DSU) позволяет за прак
 '
 }
];

```

```

export const ExpressQuiz: React.FC = () => {
 const [currentQIndex, setCurrentQIndex] = useState<num
 const [selectedOption, setSelectedOption] = useState<
 const [isAnswered, setIsAnswered] = useState<boolean>
 const [score, setScore] = useState<number>(0);
 const [showResults, setShowResults] = useState<boolean>

 const currentQ = quizQuestions[currentQIndex];

```

```

 <div className="bg-slate-950 border border-slate-900" >
 <p className="text-slate-400 text-sm uppercase">Результат теста</p>
 <p className="text-5xl font-black text-indigo-500">{score}</p>
 <p className="text-slate-300 text-sm">
 {score === quizQuestions.length
 ? ' Идеально! Ты настоящий сеньор-раскра
 : score >= 3
 ? ' Отличный результат! Главные концепции
 : ' Стоит повторить главы пособия и еще р
 }
 </p>
 </div>

 <button
 onClick={handleRestart}
 className="inline-flex items-center gap-2 px-4 py-2 rounded-xl shadow-lg shadow-indigo-600/30"
 >
 <RotateCcw className="w-5 h-5" />
 Пройти тест заново
 </button>
 </div>
);
}

```

```

return (
 <div className="bg-slate-900/90 border border-slate-900" >
 <div className="flex items-center justify-between" >
 <div className="flex items-center gap-3">

 Экспресс-тест

 <h3 className="text-xl font-bold text-white">
 Вопрос {currentQIndex + 1} / {quizQuestions.length}
 </h3>
 </div>

 Вопрос {currentQIndex + 1} / {quizQuestions.length}

 </div>
 </div>
);

```

```

 }}
 </div>
 <div className="flex-1 text-sm md:text-l
 {option}
 </div>
 </div>
);
 }}}
</div>
</div>

{isAnswered && (
 <div className="bg-slate-950 border border-slate
 <div className="flex items-center gap-2 mb-2
 <AlertCircle className="w-4 h-4" />
 Объяснение:
 </div>
 <p className="text-slate-300 text-sm leading-
 {currentQ.explanation}
 </p>
 </div>
)}

<div className="flex justify-end pt-4 border-t bo
 <button
 onClick={handleNext}
 disabled={!isAnswered}
 className={
 "px-8 py-3.5 font-bold rounded-xl transition
 (!isAnswered
 ? "bg-slate-800 text-slate-500 border bor
 : "bg-indigo-600 hover:bg-indigo-500 activ
)
 }
 >
 {currentQIndex + 1 < quizQuestions.length ? '
 </button>
</div>
</div>

```

```

 [999, 999, 999, 999, 0, 999, 1],
 [999, 999, 999, 999, 999, 0, 5],
 [999, 1, 999, 999, 999, 999, 0]
];

```

```

const adjList: { [key: number]: { v: number; w: number }[] } = {
 0: [{ v: 1, w: 4 }, { v: 3, w: 2 }],
 1: [{ v: 2, w: 3 }, { v: 5, w: 3 }],
 2: [{ v: 4, w: 2 }],
 3: [{ v: 4, w: 4 }, { v: 5, w: 2 }],
 4: [{ v: 6, w: 1 }],
 5: [{ v: 6, w: 5 }],
 6: [{ v: 1, w: 1 }],
};

```

```

const codeLines = [
 { line: 1, text: 'function floyd_warshall(matrix, V' },
 { line: 2, text: ' dist = copy(matrix)' },
 { line: 3, text: ' for k from 0 to V - 1: // Пос' },
 { line: 4, text: ' for i from 0 to V - 1:' },
 { line: 5, text: ' for j from 0 to V - 1' },
 { line: 6, text: ' if dist[i][k] + d' },
 { line: 7, text: ' dist[i][j] = ' },
 { line: 8, text: ' return dist' },
];

```

```

useEffect(() => {
 const simulationSteps: Step[] = [];
 const dist = initialMatrix.map(row => [...row]);

 simulationSteps.push({
 k: -1, i: -1, j: -1, matrix: dist.map(r => [...r]),
 log: ' Инициализация матрицы прямых путей, ∞ (999)',
 activeCodeLine: 2,
 });
});

```

```

const N = 7;
for (let k = 0; k < N; k++) {

```





```

const isI = currentStep.i === node.id;
const isJ = currentStep.j === node.id;
let fill = '#1e293b'; let stroke = '#64748f';
if (isK) { fill = '#4f46e5'; stroke = '#a6c1ee'; }
else if (isI) { fill = '#b45309'; stroke = '#d9a72f'; }
else if (isJ) { fill = '#047857'; stroke = '#2e8b57'; }
return (
 <g key={node.id} className="transition-colors">
 <circle cx={node.x} cy={node.y} r={isI ? 4 : 2} className="transition-colors duration-200">
 <text x={node.x} y={node.y + 5} fill="white" font-size="10px">{node.i}</text>
 <text x={node.x} y={node.y + 32} fill="white" font-size="10px">{node.j}</text>
 </g>
);
}}
</svg>

```

```

<div className="w-full bg-slate-950/80 p-4 rounded">
 <p className="font-semibold text-amber-400">Смежность</p>
 <p className="min-h-[40px] flex items-center">
 <div>
 </div>
 </div>

```

```

{/* Список смежности */}
<div className="w-full lg:w-1/3 bg-emerald-950/40 p-4 rounded">
 <h4 className="text-lg font-bold text-emerald-400">Смежность</h4>
 <div className="space-y-2 text-sm font-mono">
 {Object.keys(adjList).map((uStr) => {
 const u = parseInt(uStr);
 const isI = currentStep.i === u;
 const isK = currentStep.k === u;
 return (
 <div key={u} className={`flex flex-wrap items-center justify-between p-2 border ${
 isI ? 'bg-amber-900/40 border-amber-400' :
 isK ? 'bg-indigo-900/40 border-indigo-400' : 'bg-white border-gray-200'
 }`}>
 {u}
 </div>
);
 })}
 </div>

```

```

 </td>
 {row.map((val, j) => {
 const isUpd = currentStep.i === i
 const isVia = currentStep.k !== i
 tep.j === j));
 let bg = 'bg-slate-900/40 text-slate-50';
 if (isUpd) bg = 'bg-emerald-900/80 text-white';
 else if (isVia) bg = 'bg-indigo-900/80 text-white';
 else if (i === j) bg = 'bg-slate-900/40 text-slate-50';
 return (<td key={j} className={bg}
 /td>));
 }}}
 </tr>
 }}}
 </tbody>
</table>
</div>
</div>

 { /* Код алгоритма */
 <div className="w-full lg:w-1/2 bg-slate-900/60" >
 <div>
 <h4 className="text-lg font-bold text-slate-50">
 <div className="bg-slate-950/80 rounded-lg p-4">
 {codeLines.map(item => {
 <div key={item.line} className={`py-1.5 px-2`} >
 e === item.line ? 'bg-blue-900/80 text-amber-50' : 'text-slate-50';

 {item.value}

 </div>
 })}
 </div>
 </div>
 </div>
 </div>
 </div>
</div>
</div>
);
}

```

```

type Action = {
 type: 'visit' | 'process' | 'backtrack';
 node: string;
 desc: string;
 queueOrStack?: string[];
 visitedNodes?: string[];
};

function generateDFSSteps(start: string) {
 const steps: Action[] = [];
 const vis = new Set<string>();
 const stack: string[] = []; // for visualization, just for visualization

 function dfs(v: string) {
 vis.add(v);
 stack.push(v);
 steps.push({ type: 'visit', node: v, desc: `Зашли в узел ${v}`,
 visitedNodes: Array.from(vis) });

 for (const u of adj[v]) {
 if (!vis.has(u)) {
 steps.push({ type: 'process', node: v, desc: `Переходим к узлу ${u}`,
 queueOrStack: [...stack], visitedNodes: Array.from(vis) });
 dfs(u);
 }
 }

 stack.pop();
 steps.push({ type: 'backtrack', node: v, desc: `Вышли из узла ${v}`,
 queueOrStack: [...stack], visitedNodes: Array.from(vis) });
 }

 dfs(start);
 return steps;
}

function generateBFSSteps(start: string) {

```

```

useEffect(() => {
 setStepIdx(0);
 setAutoPlay(false);
}, [mode]);

useEffect(() => {
 if (autoPlay) {
 const t = setInterval(() => {
 setStepIdx(p => {
 if (p >= steps.length - 1) {
 setAutoPlay(false);
 return p;
 }
 return p + 1;
 });
 }, 1200);
 return () => clearInterval(t);
 }
}, [autoPlay, steps.length]);

const step = steps[stepIdx] || steps[0];

const isVisited = (nodeId: string) => step.visitedNodes.includes(nodeId);
const isCurrent = (nodeId: string) => step.node === nodeId;

return (
 <div className="space-y-4">
 <div className="flex bg-slate-900 border border-slate-700 p-4">
 <button onClick={() => setMode("dfs")}
 className={`px-4 py-2 text-xs font-medium text-slate-400 hover:text-slate-300`} `>
 DFS (В глубину)
 </button>
 <button onClick={() => setMode("bfs")}
 className={`px-4 py-2 text-xs font-medium text-slate-400 hover:text-slate-300`} `>
 BFS (В ширину / Волна)
 </button>
 </div>
 </div>
);

```

```

 if (current && step.type !== 'edge') {
 strokeColor = "stroke-amber-500";
 textColor = "fill-amber-500";
 radius = 22;
 }

 return (
 <g key={n.id} className="graph-node">
 <circle cx={n.x} cy={n.y}
 className={`stroke-${strokeColor} fill-${fillColor} stroke-width=2`}
 r={radius} />
 <text x={n.x} y={n.y}
 className={`text-${textColor} font-size=14`}
 {n.label} />
 </g>

 {
 /* Ripple for BFS */
 current && mode === 'bfs' && n === current
 ? (
 <circle cx={n.x} cy={n.y}
 className="stroke-amber-500 fill-transparent stroke-width=2"
 r={22} />
)
 : null
)
)
)
)
 </svg>
</div>

{
 /* Structure View (Stack / Queue) */
 <div className="bg-slate-900 border border-slate-700 p-10" >
 <div className={`absolute -top-3 left-3 right-3 bottom-3
 ${mode === 'dfs' ? 'border border-emerald-500' : 'border border-slate-700'}
 flex flex-col gap-2`}>
 {step.queueOrStack && step.queueOrStack.length > 0 ? (
 <div key={idx} className={`w-40 h-20 flex items-center justify-center
 border border-slate-700 rounded`}>
 {step.queueOrStack[idx]}
 </div>
) : null}
 </div>
 </div>
}

```

```
 </button>
 </div>

 {/* Description Log */}
 <div className="flex-1 bg-slate-900/50 h-100px overflow-hidden">
 <div className="text-[10px] text-slate-100">
 {steps.length}</div>
 <div className="text-sm text-slate-100">
 {step.desc}
 </div>
 </div>
 </div>
 </div>
);
}
```

---

ФАЙЛ 40/81: src/components/HeapViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

---

```
import React, { useState, useCallback } from 'react';
import { Heart } from 'lucide-react';
```

```
interface Patient {
 id: string;
 name: string;
 emoji: string;
 symptom: string;
 priority: number; // 1 to 99 (Severity)
 animState: 'in' | 'idle' | 'winner' | 'out';
}
```

```
// Max-heap helpers for Patients
function siftUp(arr: Patient[]): Patient[] {
 const a = arr.map(x => ({ ...x }));
```

```

 const x = ((posInLevel + 0.5) / count) * 100;
 const y = 10 + level * 24;
 return { x, y };
}

// Priority presets for realistic ER scenarios
const PATIENT_PRESETS = [
 { name: 'Иван (Инфаркт)', symptom: ' Болиит сердце', emoji: '👤🔴' },
 { name: 'Маша (Перелом)', symptom: ' Открытый перелом', emoji: '👤🔴' },
 { name: 'Кот Барсик (Укус)', symptom: ' Укусил шмеля', emoji: '🐈🔴' },
 { name: 'Семён (Температура)', symptom: ' Жар 39.5', emoji: '👤🔴' },
 { name: 'Оля (Порез)', symptom: ' Глубокий порез', emoji: '👤🔴' },
 { name: 'Пётр (Кашель)', symptom: ' Простуда', emoji: '👤🔴' },
];

const INITIAL_PATIENTS: Patient[] = (() => {
 let arr: Patient[] = [];
 const start = [
 { name: 'Иван (Инфаркт)', symptom: ' Болиит сердце', emoji: '👤🔴' },
 { name: 'Маша (Перелом)', symptom: ' Открытый перелом', emoji: '👤🔴' },
 { name: 'Семён (Температура)', symptom: ' Жар 39.5', emoji: '👤🔴' },
 { name: 'Кот Барсик (Укус)', symptom: ' Укусил шмеля', emoji: '🐈🔴' },
 { name: 'Оля (Порез)', symptom: ' Глубокий порез', emoji: '👤🔴' },
 { name: 'Пётр (Кашель)', symptom: ' Простуда', emoji: '👤🔴' },
];
 start.forEach((p, i) => {
 arr = heapInsert(arr, { id: `init${i}`, ...p, animS: 0 });
 });
 return arr;
})();

let uid = 300;

export const HeapViz: React.FC = () => {
 const [heap, setHeap] = useState<Patient[]>([]);
 const [customName, setCustomName] = useState('');
 const [customPriority, setCustomPriority] = useState(0);
 const [log, setLog] = useState<string[]>([]);

```

```

 setTimeout(() => {
 setHeap(nextHeap);
 setBusy(false);
 }, 500);
 }, [busy, heap]);

const handleRandomInsert = () => {
 const preset = PATIENT_PRESETS[Math.floor(Math.random() * PATIENT_PRESETS.length)];
 const randomShift = Math.floor(Math.random() * 10);
 const prio = Math.max(10, Math.min(99, preset.priority + randomShift));
 insertPatient(preset.name, prio);
};

const handleCustomInsert = (e: React.FormEvent) => {
 e.preventDefault();
 if (!customName.trim()) return;
 insertPatient(customName, customPriority);
 setCustomName('');
};

// --- EXTRACT MAX: Направить самого тяжелого в реанимацию
const extractMax = useCallback(() => {
 if (busy || heap.length === 0) {
 setShake(true);
 addLog('⚠ Нет пациентов для экстренной госпитализации');
 setTimeout(() => setShake(false), 400);
 return;
 }
 setBusy(true);
 const topPatient = heap[0];
 setHealing(topPatient);

 addLog(`  СРОЧНО! Забираем самого тяжелого: ${topPatient.name}`);

 // Step 1: Root lights up as winner
 setHeap(prev => prev.map((x, i) => i === 0 ? { ...x, status: 'winning' } : x));

 setTimeout(() => {

```



```

const l = 2 * idx + 1;
const r = 2 * idx + 2;
if (l < heap.length) {
 const lp = nodePos(l);
 res.push(
 <line key={`el${idx}`}
 x1={` ${p.x}%`} y1={` ${p.y + 4}%`}
 x2={` ${lp.x}%`} y2={` ${lp.y - 4}%`}
 stroke="#475569" strokeWidth="2"
 />
);
}
if (r < heap.length) {
 const rp = nodePos(r);
 res.push(
 <line key={`er${idx}`}
 x1={` ${p.x}%`} y1={` ${p.y + 4}%`}
 x2={` ${rp.x}%`} y2={` ${rp.y - 4}%`}
 stroke="#475569" strokeWidth="2"
 />
);
}
return res;
});

return (
 <div className="flex flex-col gap-6">
 {/* МНЕМОНИКА / ПРАВИЛО */}
 <div className="bg-emerald-950/40 border border-e
 <div className="text-3xl"> </div>
 <div>
 <h3 className="font-black text-emerald-400 te
 ty Queue)</h3>
 <p className="text-xs text-slate-300 mt-1 lea
 В реанимации не важно, кто пришел раньше, а
 остояние (максимальный приоритет).
 Куча (Heap) автоматически перестраивает пац
 оказывался на самом верху дерева (в корне).
```

```

 />
 </div>
 <button
 type="submit"
 disabled={busy || !customName.trim() || heap.length > 0}
 className="bg-teal-600 hover:bg-teal-500 disabled:opacity-50 text-white px-4 rounded-xl shadow-lg active:scale-95 transition-all cursor-pointer"
 >
 Вести
 </button>
 </form>

 { /* Забрать самого тяжелого */ }
 <button
 onClick={extractMax}
 disabled={busy || heap.length === 0}
 className="flex-1 min-w-[170px] bg-gradient-to-r from-teal-600 to-teal-500 disabled:cursor-not-allowed text-white font-medium rounded-xl shadow-lg transition-all cursor-pointer flex items-center justify-center"
 >
 В реанимацию (EXTRACT MAX)
 </button>

 <button
 onClick={reset}
 className="px-5 py-3.5 rounded-2xl bg-slate-800 text-white font-medium cursor-pointer border border-slate-700"
 >
 Сброс
 </button>
 </div>

 { /* ИНТЕРАКТИВНОЕ ОТДЕЛЕНИЕ */ }
 <div className="grid grid-cols-1 lg:grid-cols-12 gap-4" >

 { /* ЛЕВАЯ КОЛОНКА: ДЕРЕВО ПАЦИЕНТОВ */ }
 <div className={`lg:col-span-8 bg-slate-900 rounded-2xl shadow-2xl p-4 min-h-[380px] overflow-hidden ${shakeEmpty ? 'animate-pulse' : ''}`}>

```

```

 0 shadow-xl">
 <div className="font-bold text-white">
 <div className="text-emerald-400">
 </div>
 </div>
 </div>
 </div>
 </div>
 </div>
 </div>

 <div className="w-full h-3 bg-gradient-to-r from-emerald-400 to-emerald-600">
 </div>

 { /* ПРАВАЯ КОЛОНКА: ОПЕРАЦИОННЫЙ СТОЛ */ }
 <div className="lg:col-span-4 bg-slate-900 rounded-2xl p-4">
 <div className="absolute top-4 left-4 text-white">
 Операционная койка
 </div>

 <div className="flex-1 flex flex-col justify-between p-4">
 {healingPatient ? (
 <div className="flex flex-col items-center justify-between h-full">
 <div className="relative w-full h-full">

 </div>
 </div>
 <div>
 <h4 className="text-white font-black">
 Пациент: {healingPatient.name}
 </h4>
 <p className="text-emerald-400 text-xl">
 Состояние: Стабилизация ({healingPatient.status})
 </p>
 <div className="flex justify-center gap-4">

 <Heart className="w-4 h-4 text-rose-500">
 </div>
 </div>
 </div>
 </div>
 </div>
) : (
 <div>
 <h4>Пациент не найден</h4>
 <p>Пожалуйста, добавьте пациента в базу данных.</p>
 </div>
)}
 </div>
 </div>

```

```
import React, { createContext, useContext, useEffect, u

/** Общие примитивы для мини-визуализаций подсказок. */
export const W = 260;
export const H = 130;

/**
 * Общий множитель скорости. Кадры были слишком короткими
 * заметить, что именно переехало, и анимация читалась
 */
export const SPEED = 2.1;

/** Длительность переезда узлов. Должна быть заметно ме
export const MOVE_MS = 800;
export const EASE = "cubic-bezier(.34,.01,.2,1)";
export const MOVE = `all ${MOVE_MS}ms ${EASE}`;

/** Наведение на карточку ставит анимацию на паузу – мо
export const DemoPauseContext = createContext(false);

export function useLoop(steps: number, ms = 900) {
 const [step, setStep] = useState(0);
 const paused = useContext(DemoPauseContext);
 const period = Math.round(ms * SPEED);

 useEffect(() => {
 if (steps <= 1 || paused) return;
 const t = setInterval(() => setStep((s) => (s + 1) %
 return () => clearInterval(t);
 }, [steps, period, paused]);

 return step;
}

export const C = {
 idle: "#334155",
 idleStroke: "#475569",
```

```

 {label !== undefined && {
 <text x={x} y={y} textAnchor="middle" dominantBaseline="middle">
 {label}
 </text>
 }}
 </g>
);

export const Edge: React.FC<{
 a: [number, number];
 b: [number, number];
 color?: string;
 width?: number;
 dashed?: boolean;
}> = ({ a, b, color = C.edge, width = 2, dashed }) => (
 <line
 x1={a[0]}
 y1={a[1]}
 x2={b[0]}
 y2={b[1]}
 stroke={color}
 strokeWidth={width}
 strokeDasharray={dashed ? "4 3" : undefined}
 style={{ transition: MOVE }}
 />
);

```

ФАЙЛ 42/81: src/components/hints/demosExtra.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```

import React from "react";
import { C, Edge, MOVE, Node, Svg, useLoop } from "../demo";

/** Флойд: внешний цикл по «разрешённой» промежуточной вершине */
export const FloydDemo: React.FC = () => {

```

```

const comps: [string, string][][] = [
 ["A", "B"], ["B", "C"],
 ["D", "E"],
 ["F", "G"], ["G", "H"], ["F", "H"],
];
const pos: Record<string, [number, number]> = {
 A: [26, 40], B: [62, 92], C: [26, 92],
 D: [120, 40], E: [120, 92],
 F: [190, 34], G: [232, 78], H: [178, 96],
};
const colors = [C.active, C.warn, C.done];
const step = useLoop(comps.length + 1, 900);
const compOf = (v: string) => comps.findIndex((e) =>

return (
 <Svg caption={step === 0 ? "Граф распался на острова"
 {comps.flatMap((edges, ci) =>
 edges.map(([u, v], i) => {
 <Edge key={` ${ci} - ${i}`} a={pos[u]} b={pos[v]}
 })
 })
 {Object.entries(pos).map(([k, [x, y]]) => {
 const ci = compOf(k);
 return <Node key={k} x={x} y={y} r={12} label={
 }}}
 </Svg>
);
};

```

/\*\* Что такое граф: вершины, рёбра, направление и вес.

```

export const GraphBasicsDemo: React.FC = () => {
 const step = useLoop(3, 1300);
 const pos: Record<string, [number, number]> = { A: [50, 40], B: [150, 40],
 const edges: [string, string, number][] = [
 ["A", "B", 1],
 ["B", "C", 1],
 ["C", "D", 1],
 ["D", "E", 1],
 ["E", "F", 1],
 ["F", "G", 1],
 ["G", "H", 1],
 ["H", "I", 1],
 ["I", "J", 1],
 ["J", "K", 1],
 ["K", "L", 1],
 ["L", "M", 1],
 ["M", "N", 1],
 ["N", "O", 1],
 ["O", "P", 1],
 ["P", "Q", 1],
 ["Q", "R", 1],
 ["R", "S", 1],
 ["S", "T", 1],
 ["T", "U", 1],
 ["U", "V", 1],
 ["V", "W", 1],
 ["W", "X", 1],
 ["X", "Y", 1],
 ["Y", "Z", 1],
 ["Z", "A", 1],
];
 return (
 <Svg
 caption={
 step === 0

```

```

const step = useLoop(3, 1200);
const boxW = 46;
const startX = (260 - raw.length * (boxW + 4)) / 2;

return (
 <Svg
 caption={
 step === 0
 ? "Исходные координаты – огромные и с дырами"
 : step === 1
 ? "Сортируем уникальные значения"
 : "Каждое число заменяем его номером → плотн
 }
 >
 {raw.map((v, i) => {
 <g key={i}>
 <rect x={startX + i * (boxW + 4)} y={20} width
 <text x={startX + i * (boxW + 4) + boxW / 2}
 {v}
 </text>
 </g>
 })}

 {step >= 1 &&
 sorted.map((v, i) => {
 <g key={v}>
 <rect x={30 + i * 52} y={62} width={46} hei
 <text x={53 + i * 52} y={77} textAnchor="mi
 {v}
 </text>
 </g>
 })}

 {step >= 2 &&
 raw.map((v, i) => {
 <g key={`c${i}`}>
 <rect x={startX + i * (boxW + 4)} y={98} wi
 <text x={startX + i * (boxW + 4) + boxW / 2

```

```

 <circle r={13} fill={color(id)} stroke={C.idl
 <text textAnchor="middle" dominantBaseline="c
 {id}
 </text>
 </g>
 }}}
</Svg>
);
};

```

```

/** Zig – один поворот, когда родитель уже корень. */
export const ZigDemo: React.FC = () => {
 <SplayFrames
 captions=[["p – корень, x под ним", "Один поворот:
 frames=[
 { pos: { p: [130, 32], x: [78, 92] }, edges: [{"p
 { pos: { x: [130, 32], p: [182, 92] }, edges: [{"
]}
 />
};

```

```

/** Zig-Zig – x и p с одной стороны, крутим сначала вер
export const ZigZigDemo: React.FC = () => {
 <SplayFrames
 captions=[["Линия g → p → x («бамбук»)", "Поворот В
 frames=[
 { pos: { g: [176, 24], p: [126, 70], x: [76, 112]
 { pos: { p: [130, 26], x: [78, 78], g: [186, 78]
 { pos: { x: [102, 24], p: [152, 70], g: [202, 112
]}
 />
};

```

```

/** Zig-Zag – x и p с разных сторон, крутим сначала ниж
export const ZigZagDemo: React.FC = () => {
 <SplayFrames
 captions=[["Зигзаг: g влево, x вправо", "Поворот НИ
 frames=[

```



```

 </Svg>
);
};

export const DfsDemo: React.FC = () => {
 const path = ["A", "B", "D", "E", "F", "C"];
 const step = useLoop(path.length + 1, 750);
 const visited = path.slice(0, step);
 const head = visited[visited.length - 1];
 return (
 <Svg caption={head ? `Идём вглубь: ${visited.join(" ")}` : ""}
 {GRAPH_EDGES.map(([u, v], i) => {
 const iu = visited.indexOf(u);
 const iv = visited.indexOf(v);
 const onPath = iu >= 0 && iv >= 0 && Math.abs(iu - iv) <= 1;
 return <Edge key={i} a={GRAPH_POS[u]} b={GRAPH_POS[v]} color={onPath ? "red" : "gray"} />
 })}
 {Object.entries(GRAPH_POS).map([k, [x, y]] => (
 <Node key={k} x={x} y={y} label={k} fill={k} />
))}
 </Svg>
);
};

```

```

export const ToposortDemo: React.FC = () => {
 const nodes = [
 { id: "труссы", x: 52, y: 32 }, { id: "штаны", x: 148, y: 32 },
 { id: "носки", x: 52, y: 98 }, { id: "боты", x: 148, y: 98 }
];
 const edges: [string, string][] = [
 ["труссы", "штаны"],
 ["штаны", "носки"],
 ["штаны", "боты"]
];
 const order = ["труссы", "носки", "штаны", "боты"];
 const step = useLoop(order.length + 1, 800);
 const placed = order.slice(0, step);
 const pos = Object.fromEntries(nodes.map((n) => [n.id, n]));
 return (
 <Svg caption={`Порядок: ${placed.join(" → ") || "..."}`
 {edges.map(([u, v], i) => (
 <Edge key={i} a={pos[u]} b={pos[v]} color={placed.indexOf(v) < placed.indexOf(u) ? "red" : "gray"} />
))}
 </Svg>
);
};

```

```
export const BridgeDemo: React.FC = () => {
 const step = useLoop(2, 1300);
 const pos: Record<string, [number, number]> = {
 A: [40, 40], B: [40, 95], C: [95, 68], D: [170, 68]
 };
 const edges: [string, string][] = [
 ["A", "B"], ["A", "C"], ["B", "C"], ["C", "D"], ["D", "A"]
];
 const cut = step === 1;
 return (
 <Svg caption={cut ? "Убрали C-D → граф развалился на 2 компонента" : "Граф связен"}>
 {edges.map(([u, v], i) => {
 const isBridge = u === "C" && v === "D";
 if (isBridge && cut) return null;
 return <Edge key={i} a={pos[u]} b={pos[v]} color={isBridge ? "red" : "black"} />
 })}
 {Object.entries(pos).map(([k, [x, y]]) => (
 <Node key={k} x={x} y={y} label={k} fill={cut ? "white" : "black"} />
))}
 </Svg>
);
};
```

```
export const ArticulationDemo: React.FC = () => {
 const step = useLoop(2, 1300);
 const pos: Record<string, [number, number]> = {
 A: [36, 40], B: [36, 96], C: [130, 68], D: [224, 40]
 };
 const edges: [string, string][] = [
 ["A", "B"], ["A", "C"], ["B", "C"], ["C", "D"], ["D", "A"]
];
 const cut = step === 1;
 return (
 <Svg caption={cut ? "Убрали вершину C → две отдельные компоненты" : "Граф связен"}>
 {edges.map(([u, v], i) => (cut && (u === "C" || v === "C") ? null : <Edge key={i} a={pos[u]} b={pos[v]} color="black" />))}
 {Object.entries(pos).map(([k, [x, y]]) => (
 cut && k === "C" ? (
 <circle key={k} cx={x} cy={y} r={12} fill="none" />
) : (
 <Node key={k} x={x} y={y} label={k} fill={cut ? "white" : "black"} />
)
))}
 </Svg>
);
};
```

```

A: [45, 35], B: [110, 35], C: [77, 95], D: [185, 45]
};
const edges: [string, string][] = [["A", "B"], ["B",
const step = useLoop(2, 1400);
const scc = ["A", "B", "C"];
const inScc = (u: string, v: string) => step === 1 &&
return (
 <Svg caption={step === 1 ? "{A,B,C} – цикл: из кажд
 {edges.map(([u, v], i) => {
 <Edge key={i} a={pos[u]} b={pos[v]} color={inSc
 })}
 {Object.entries(pos).map(([k, [x, y]]) => {
 <Node key={k} x={x} y={y} label={k} fill={step
 })}
 </Svg>
);
};

```

ФАЙЛ 44/81: src/components/hints/demosStruct.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```

import React from "react";
import { C, Edge, Node, Svg, useLoop } from "../demoKit"

export const HeapDemo: React.FC = () => {
 const states = [[4, 8, 9, 12, 2], [4, 2, 9, 12, 8], [
 const step = useLoop(states.length, 1000);
 const heap = states[step];
 const pos: [number, number][] = [[130, 24], [80, 68],
 const moving = step === 0 ? 4 : step === 1 ? 1 : 0;
 return (
 <Svg caption="Новый элемент всплывает наверх, пока
 <Edge a={pos[0]} b={pos[1]} /><Edge a={pos[0]} b=
 <Edge a={pos[1]} b={pos[3]} /><Edge a={pos[1]} b=
 {heap.map((v, i) => {

```

```

<text x={206} y={44} fontSize={9} fill={C.dim}>vx
{items.map((v, i) => (
 <g key={v} style={{ transition: "all .3s" }}>
 <rect x={30 + i * 54} y={56} width={46} height={16}>
 <text x={53 + i * 54} y={71} textAnchor="middle">{v}</text>
 </g>
)}}
<line x1={24} y1={71} x2={10} y2={71} stroke={C.dim}>
</Svg>
);
};

```

```

export const DsuDemo: React.FC = () => {
 const parents = [[0, 1, 2, 3], [0, 0, 2, 3], [0, 0, 2, 3], [0, 0, 2, 3]];
 const step = useLoop(parents.length, 950);
 const p = parents[step];
 const pos: [number, number][] = [[50, 95], [105, 95], [160, 95], [215, 95]];
 const rootColor = ["#6366f1", "#10b981", "#f59e0b", "#f59e0b"];
 const root = (i: number): number => (p[i] === i ? i : root(p[i]));
 return (
 <Svg caption="find = «кто твой староста», union = 0" >
 {p.map((par, i) =>
 par !== i ? (
 <path key={i} d={`M ${pos[i][0]} ${pos[i][1]} L ${pos[root(i)][0]} ${pos[root(i)][1]}`
 fill="none" stroke={rootColor[root(i)]} strokeDash={[]}>
) : null
)}
)}
 {pos.map(([x, y], i) => (<Node key={i} x={x} y={y}>{p[i]}</Node>))}
 </Svg>
);
};

```

```

const TRIE_NODES = [
 { id: 0, x: 130, y: 22, ch: "*", parent: null as number },
 { id: 1, x: 72, y: 62, ch: "a", parent: 0 },
 { id: 2, x: 188, y: 62, ch: "b", parent: 0 },
 { id: 3, x: 42, y: 104, ch: "b", parent: 1 },

```

```

const links: [number, number][] = [[1, 0], [2, 0], [3
const step = useLoop(links.length + 1, 900);
return (
 <Svg caption="Суффиксная ссылка: «куда откатиться,
 {TRIE_NODES.filter((n) => n.parent !== null).map(
 const p = TRIE_NODES[n.parent as number];
 return <Edge key={n.id} a={[p.x, p.y]} b={[n.x,
]}}
 {links.slice(0, step).map(([from, to], i) => {
 const a = TRIE_NODES[from];
 const b = TRIE_NODES[to];
 return (
 <path key={i} d={`M ${a.x} ${a.y} Q ${(a.x +
 fill="none" stroke={C.warn} strokeWidth={1.
)};
)}}
 {TRIE_NODES.map((n) => (
 <Node key={n.id} x={n.x} y={n.y} label={n.ch} f
)}}
 </Svg>
);
};

```

```

export const SegmentTreeDemo: React.FC = () => {
 const step = useLoop(3, 1000);
 const nodes = [
 { x: 130, y: 24, w: 226, label: "[0..7]", lvl: 0 },
 { x: 72, y: 60, w: 110, label: "[0..3]", lvl: 1 },
 { x: 188, y: 60, w: 110, label: "[4..7]", lvl: 1 },
 { x: 44, y: 96, w: 52, label: "[0..1]", lvl: 2 },
 { x: 100, y: 96, w: 52, label: "[2..3]", lvl: 2 },
 { x: 160, y: 96, w: 52, label: "[4..5]", lvl: 2 },
 { x: 216, y: 96, w: 52, label: "[6..7]", lvl: 2 },
];
 return (
 <Svg caption="Запрос собирается из $O(\log n)$ готовых
 {nodes.map((n, i) => (
 <g key={i}>

```

```

 });
};

export const SparseTableDemo: React.FC = () => {
 const step = useLoop(3, 1000);
 const len = [1, 2, 4][step];
 const count = 8 - len + 1;
 return (
 <Svg caption={`Уровень j=${step}: готовые ответы для`}
 {Array.from({ length: 8 }).map((_, i) => {
 <g key={i}>
 <rect x={12 + i * 30} y={18} width={26} height={10} />
 <text x={25 + i * 30} y={29} textAnchor="middle" />
 </g>
 })}
 {Array.from({ length: count }).map((_, i) => {
 <rect key={i} x={12 + i * 30} y={52 + (i % 4) * 10} width={26} height={10}
 fill={C.active} opacity={i === 0 ? 1 : 0.4} stroke={C.done} />
 })}
 </Svg>
);
};

```

```

export const AmortizedDemo: React.FC = () => {
 const costs = [1, 1, 1, 8, 1, 1, 1, 1];
 const step = useLoop(costs.length + 1, 550);
 const total = costs.slice(0, step).reduce((a, b) => a + b, 0);
 const avg = step ? (total / step).toFixed(2) : "-";
 return (
 <Svg caption={`Сделано ${step} операций, суммарно $`}
 {costs.map((c, i) => {
 <rect key={i} x={14 + i * 30} y={104 - c * 10} width={26} height={10}
 fill={i < step ? (c > 2 ? C.bad : C.done) : C.done} stroke={C.done} />
 })}
 <line x1={8} y1={106} x2={252} y2={106} stroke={C.done} />
 <text x={130} y={20} textAnchor="middle" fontSize={14} />
 </Svg>
);
};

```

```

 <text x={4} y={70} fontSize={8} fill={C.dim}>π</text>
 <text x={130} y={106} textAnchor="middle" fontSize=
 π[i] – длина самого длинного «префикс = суффикс»
 </text>
 </Svg>
);
};

export const DpDemo: React.FC = () => {
 const rows = 3, cols = 5;
 const step = useLoop(rows * cols + 1, 320);
 return (
 <Svg caption={`Заполняем таблицу подзадач: готово $
 {Array.from({ length: rows }).map((_, r) =>
 Array.from({ length: cols }).map((_, c) => {
 const idx = r * cols + c;
 const filled = idx < step;
 return (
 <g key={idx}>
 <rect x={40 + c * 38} y={22 + r * 32} wid
 fill={idx === step - 1 ? C.active : fil
 {filled && (
 <text x={57 + c * 38} y={36 + r * 32} t
 fontSize={10} fontWeight="700" fill="
)}
 </g>
);
 })
)}
 <text x={130} y={122} textAnchor="middle" fontSiz
 каждая клетка считается один раз и переиспользоу
 </text>
 </Svg>
);
};

```

```

 bridge: BridgeDemo,
 articulation: ArticulationDemo,
 mst: MstDemo,
 amortized: AmortizedDemo,
 np: NpDemo,
 scc: SccDemo,
 "prefix-function": PrefixFunctionDemo,
 dp: DpDemo,
 floyd: FloydDemo,
 components: ComponentsDemo,
 graph: GraphBasicsDemo,
 "coord-compress": CoordCompressDemo,
 zig: ZigDemo,
 "zig-zig": ZigZigDemo,
 "zig-zag": ZigZagDemo,
 };

export const MiniDemo: React.FC<{ kind: DemoKind }> = (
 const [paused, setPaused] = useState(false);
 const Cmp = REGISTRY[kind];
 if (!Cmp) return null;
 return (
 <DemoPauseContext.Provider value={paused}>
 <div
 className="bg-slate-950/70 rounded-lg border bo
 onMouseEnter={() => setPaused(true)}
 onMouseLeave={() => setPaused(false)}
 >
 <Cmp />
 </div>
 </DemoPauseContext.Provider>
);
};

```



```
 <h3 className="font-bold text-white text-sm
 {viz.hint && <p className="text-[11px] text
</div>
<button
 onClick={onClose}
 className="p-2 rounded-lg bg-slate-800 hove
 aria-label="Заккрыть симулятор"
>
 <X className="w-5 h-5" />
</button>
</div>
<div className="overflow-y-auto overscroll-cont
 <Component />
</div>
</div>
</div>,
document.body
);
};
```

---

ФАЙЛ 47/81: src/components/hints/TermPopover.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРО

---

```
import React, { useEffect, useLayoutEffect, useRef, use
import { createPortal } from "react-dom";
import { Play, X } from "lucide-react";
import { GLOSSARY_BY_ID } from "../../data/glossary";
import { MiniDemo } from "../MiniDemo";
import { getViz } from "../vizRegistry";

export interface HintAnchor {
 termId: string;
 rect: DOMRect;
}
```

```

 };
 window.addEventListener("keydown", onKey);
 window.addEventListener("scroll", onClose, true);
 return () => {
 window.removeEventListener("keydown", onKey);
 window.removeEventListener("scroll", onClose, true);
 };
}, [anchor, onClose]);

if (!anchor) return null;
const entry = GLOSSARY_BY_ID.get(anchor.termId);
if (!entry) return null;
const viz = getViz(entry.simulator);
const simulatorId = entry.simulator;

return createPortal(
 <div
 ref={cardRef}
 onMouseEnter={onCardEnter}
 onMouseLeave={onCardLeave}
 className="fixed z-[100] term-popover"
 style={{
 top: pos ? pos.top : -9999,
 left: pos ? pos.left : -9999,
 width: Math.min(CARD_W, typeof window !== "undefined" ? window.innerWidth : 1000),
 visibility: pos ? "visible" : "hidden",
 }}
 role="dialog"
 >
 <div className="bg-slate-900 border border-indigo-500"
 <div className="flex items-start gap-2 px-3 pt-3"
 <h4 className="text-sm font-bold text-indigo-400">
 <button
 onClick={onClose}
 className="text-slate-500 hover:text-white"
 aria-label="Заккрыть подсказку"
 >
 <X className="w-4 h-4" />

```

ФАЙЛ 48/81: src/components/JohnsonViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

```
import { useState } from 'react';
import { RotateCcw, SkipForward, Undo } from 'lucide-react';

export function JohnsonViz() {
 const [step, setStep] = useState(0);
 const MAX_STEP = 7;

 const nodes = [
 { id: 'S', x: 200, y: 150, label: 'S' },
 { id: 'A', x: 200, y: 50, label: 'A' },
 { id: 'B', x: 320, y: 220, label: 'B' },
 { id: 'C', x: 80, y: 220, label: 'C' },
];

 const edges = [
 { id: 'SA', u: 'S', v: 'A', weight: 0, labelDx: 0, labelDy: 0 },
 { id: 'SB', u: 'S', v: 'B', weight: 0, labelDx: 0, labelDy: 0 },
 { id: 'SC', u: 'S', v: 'C', weight: 0, labelDx: 0, labelDy: 0 },
 { id: 'AB', u: 'A', v: 'B', weight: -2, newWeight: 0, labelDx: 0, labelDy: 0 },
 { id: 'BC', u: 'B', v: 'C', weight: 1, newWeight: 0, labelDx: 0, labelDy: 0 },
 { id: 'CA', u: 'C', v: 'A', weight: 2, newWeight: 0, labelDx: 0, labelDy: 0 },
];

 const isNodeVisible = (id: string) => {
 if (id === 'S' && (step === 0 || step === 7)) return true;
 return false;
 };

 const getPotential = (id: string) => {
 if (step >= 2 && step < 7) {
 if (id === 'S') return 0;
 if (id === 'A') return 0;
 if (id === 'B') return -2;
 if (id === 'C') return -1;
 }
 return 0;
 };
}
```

```

 if (e.id === 'BC' && step === 5) return "url(#a
 if (e.id === 'BC' && step > 5) return "url(#arr
 if (e.id === 'CA' && step === 6) return "url(#a
 if (e.id === 'CA' && step > 6) return "url(#arr

 if (step >= 4) return "url(#arrow-slate-dim)";
 return "url(#arrow-slate)";
};

const getEdgeContent = (e: any) => {
 if (e.u === 'S') return e.weight;

 if (step < 4) return e.weight;

 if (e.id === 'AB') {
 if (step === 4) return `${e.weight} + (${0}
 return e.newWeight;
 }
 if (e.id === 'BC') {
 if (step < 5) return e.weight;
 if (step === 5) return `${e.weight} + (${-2}
 return e.newWeight;
 }
 if (e.id === 'CA') {
 if (step < 6) return e.weight;
 if (step === 6) return `${e.weight} + (${-1}
 return e.newWeight;
 }
 return e.weight;
};

const isEdgeTextHighlighted = (e: any) => {
 if (e.id === 'AB' && step === 4) return true;
 if (e.id === 'BC' && step === 5) return true;
 if (e.id === 'CA' && step === 6) return true;
 return false;
};

```

```

);
 case 4: return (
 <div>
 Шаг 3.1: Пересчет ребра A → B.
 Старый вес: 1
 Вычисляем: $-2 + 0 - (-2) =$ 0
 </div>
);
 case 5: return (
 <div>
 Шаг 3.2: Пересчет ребра B → C.
 Старый вес: 1.

 Вычисляем: $1 + (-2) - (-1) =$ 0
 </div>
);
 case 6: return (
 <div>
 Шаг 3.3: Пересчет ребра C → A.
 Старый вес: 2.

 Вычисляем: $2 + (-1) - 0 =$ 1
 </div>
);
 case 7: return (
 <div className="text-emerald-300">
 Готово! Фиктивную вершину S
 Все новые веса ребер в графе стали
 При этом старые кратчайшие пути оста
 </div>
);
 default: return "";
 }
};

return (
 <div className="bg-slate-800 p-4 rounded-xl border">
 <h4 className="text-sm md:text-base font-bold">


```

```

const midX = (u.x + v.x) / 2
const midY = (u.y + v.y) / 2

const edgeColorClass = getEdgeColorClass(u, v)
const textHigh = isEdgeTextHigh(u, v)
const textEmer = isEdgeTextEmer(u, v)

return (
 <g key={i}>
 <line
 x1={u.x} y1={u.y}
 className={`stroke-${edgeColorClass}`}
 markerEnd={getMarkerEnd(u, v)}
 />

 <text
 x={midX} y={midY}
 textAnchor="middle"
 dominantBaseline="bottom"
 className={`text-${edgeColorClass}`}
 style={{textHeight: textHigh, textEmer: textEmer}}
 isS ? 'font-size: 1.2em' : ''
 >{getEdgeContent(u, v)}
 </text>
 </g>
)
)
})

```

```

{/* Nodes */}
nodes.map(n => {
 if (!isNodeVisible(n.id)) return
 const pot = getPotential(n)

 return (
 <g key={n.id} className={`node-${pot}>
 <circle
 cx={n.x} cy={n.y}
 r={n.r}
 fill={n.fill}
 stroke={n.stroke}
 stroke-width={n.strokeWidth}
 />
 >

```

```

 className="p-3 bg-slate-800
-slate-400 transition-colors" title="Сброси
 <RotateCcw strokeWidth={3}
 </button>
 <button onClick={() => setStep(1)
 className="p-3 bg-slate-800
-slate-400 transition-colors" title="Шаг на
 <Undo strokeWidth={3} size=
 </button>
 <button onClick={() => setStep(
 className={`flex-1 font-bol
 ${step === MAX_STEP
 ? 'bg-emerald-600/50'
 : 'bg-indigo-600 ho
 {step === MAX_STEP ? "Готов
 {step !== MAX_STEP && <Skip
 </button>
 </div>
 </div>

 {/* Progress Dots */}
 <div className="flex justify-center
 {Array.from({length: MAX_STEP +
 <div key={i} className={`w-1
125' : i < step ? 'bg-indigo-900' : 'bg-sla
)}}
 </div>
</div>
</div>
</div>
</div>
);
}

```

```

phase: "init" | "dfs1" | "transpose" | "dfs2" | "fini
desc: string;
visited: Set<number>;
currentNode: number | null;
order: number[];
transposed: boolean;
sccMap: Record<number, number>; // node -> sccId
currentSccId: number;
activeEdges: GEdge[];
}

```

```

export const KosarajuViz: React.FC = () => {
 const [steps, setSteps] = useState<AlgoStep[]>([]);
 const [currentStepIdx, setCurrentStepIdx] = useState(0);
 const [isPlaying, setIsPlaying] = useState(false);

```

```

 useEffect(() => {
 // Generate step-by-step Kosaraju steps
 const generatedSteps: AlgoStep[] = [];
 const visited = new Set<number>();
 const order: number[] = [];
 const sccMap: Record<number, number> = {};

```

```

 const recordStep = (
 phase: AlgoStep["phase"],
 desc: string,
 currentNode: number | null,
 transposed: boolean,
 currentSccId: number,
) => {
 generatedSteps.push({
 phase,
 desc,
 visited: new Set(visited),
 currentNode,
 order: [...order],
 transposed,
 sccMap: { ...sccMap },

```



```

 "dfs1",
 `DFS 1: вершина ${u} полностью обработана. Запи
 u,
 false,
 -1,
);
};

// Run DFS1 on all unvisited
for (let i = 0; i < 5; i++) {
 if (!visited.has(i)) {
 dfs1(i);
 }
}

const topoOrder = [...order].reverse();
recordStep(
 "transpose",
 `Первый проход завершен! Стек выхода в топологиче
 ебра.`,
 null,
 true,
 -1,
);

// Transpose adj
const adjT: number[][] = Array.from({ length: 5 },
initialEdges.forEach((e) => adjT[e.v].push(e.u));

// Reset visited for phase 2
visited.clear();
let sccId = 0;

const dfs2 = (u: number, currentScc: number) => {
 visited.add(u);
 sccMap[u] = currentScc;
 recordStep(
 "dfs2",

```

```

 "finished",
 `Алгоритм успешно завершен! Найдено ${sccId} компонент,
 null,
 true,
 sccId,
);

 setSteps(generatedSteps);
 setCurrentStepIdx(0);
}, []));

const step = steps[currentStepIdx] || {
 phase: "init",
 desc: "Загрузка...",
 visited: new Set(),
 currentNode: null,
 order: [],
 transposed: false,
 sccMap: {},
 currentSccId: -1,
 activeEdges: initialEdges,
};

useEffect(() => {
 let timer: NodeJS.Timeout;
 if (isPlaying && currentStepIdx < steps.length - 1) {
 timer = setInterval(() => {
 setCurrentStepIdx((prev) => prev + 1);
 }, 1500);
 } else {
 setIsPlaying(false);
 }
 return () => clearInterval(timer);
}, [isPlaying, currentStepIdx, steps.length]);

const togglePlay = () => setIsPlaying(!isPlaying);
const reset = () => {
 setIsPlaying(false);

```

```

 </div>
</div>

<div className="grid grid-cols-1 lg:grid-cols-12" >
 { /* Playfield SVG */}
 <div className="lg:col-span-8 bg-[#0B1120] relative" >
 <svg className="w-full h-full max-w-[650px]" >
 { /* Markers for directed arrows */}
 <defs>
 <marker
 id="arrow"
 viewBox="0 0 10 10"
 refX="22"
 refY="5"
 markerWidth="6"
 markerHeight="6"
 orient="auto-start-reverse"
 >
 <path d="M 0 1 L 10 5 L 0 9 z" fill="#66BB6A" />
 </marker>
 <marker
 id="arrow-active"
 viewBox="0 0 10 10"
 refX="22"
 refY="5"
 markerWidth="6"
 markerHeight="6"
 orient="auto-start-reverse"
 >
 <path d="M 0 1 L 10 5 L 0 9 z" fill="#FFEB3B" />
 </marker>
 <marker
 id="arrow-transposed"
 viewBox="0 0 10 10"
 refX="22"
 refY="5"
 markerWidth="6"
 markerHeight="6"

```

```

const isBidi =
 (edge.u === 3 && edge.v === 4) ||
 (edge.u === 4 && edge.v === 3);
const dy = isBidi ? (edge.u === 3 ? -10 : 10) : 0;

return (
 <line
 key={`edge-${idx}`}
 x1={x1}
 y1={y1 + dy}
 x2={x2}
 y2={y2 + dy}
 stroke={stroke}
 strokeWidth="2.5"
 markerEnd={`url(#${markerId})`}
 className="transition-all duration-300"
 />
);
}}}

{/* Nodes */}
{initialNodes.map((node) => {
 const isCurrent = step.currentNode === node.id;
 const isVisited =
 step.visited.has(node.id) ||
 (step.phase === "dfs1" && step.currentNode === node.id);
 const sccId = step.sccMap[node.id];

 let classes = "fill-slate-800 stroke-slate-800";
 if (sccId) {
 classes = getSccColor(sccId);
 } else if (isCurrent) {
 classes = "fill-amber-600 stroke-amber-600";
 } else if (isVisited && step.phase === "dfs2") {
 classes = "fill-emerald-700 stroke-emerald-700";
 }

 return (
 <div
 key={node.id}
 className={classes}
 style={{
 width: node.w,
 height: node.h,
 position: 'relative',
 margin: 10,
 }}
 >
 {node.label}
 </div>
);
})}

```

```

 {step.transposed && (
 <div className="absolute top-4 left-4 bg-indigo-950
 1 rounded">
 Граф транспонирован (Ребра обратные)
 </div>
)}
 </div>

```

```

 {/* Logs & Stacks */}
 <div className="lg:col-span-4 bg-slate-950 p-5
 00 overflow-y-auto">
 <h4 className="text-xs font-bold text-slate-400">
 Порядок обхода и стек
 </h4>

```

```

 <div className="bg-slate-900 border border-slate-800
 <p className="text-xs text-white leading-relaxed">
 {step.desc}
 </p>
 </div>

```

```

 <div className="flex-1 space-y-4 overflow-y-auto">
 {/* Top-Sort output / Order stack representation */}
 <div>

 СТЕК ВЫХОДА (DFS 1 ORDER):

 <div className="flex flex-wrap gap-1 bg-slate-900
 {step.order.length === 0 ? (

 Нет элементов в стеке
) : (
 [...step.order].reverse().map((val, idx) => {
 <div
 key={idx}
 className="bg-indigo-950 border border-indigo-800
 px-2 py-1 text-xs text-white
 >
 {val}

```

```
 Назад
 </button>
 <button
 disabled={currentStepIdx >= steps.length}
 onClick={() => setCurrentStepIdx((prev) => prev - 1)}
 className="bg-slate-900 border border-slate-500"
 >
 Вперед
 </button>
 </div>
 </div>
</div>
</div>
</div>
);
};
```

---

ФАЙЛ 50/81: src/components/KruskalSimulator.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

---

```
import React, { useState } from 'react';
import { Play, RotateCcw, SkipForward, HelpCircle, Check } from 'lucide-react';

interface Vertex {
 id: number;
 label: string;
 x: number;
 y: number;
 color: string; // 'none', 'red', 'blue', 'purple', 'magenta'
}

interface Edge {
 id: string;
 u: number;
 v: number;
 weight: number;
 type: 'solid' | 'dashed';
}
```



```

 type: 'success'
 }, ...prev]));
 },
 // Step 7: Inspect A-D (5)
 () => {
 setEdges(prev => prev.map(e => e.id === '0-3' ? {
 setLogs(prev => [{
 title: 'Шаг 4: Берем ребро A-D (вес 5)',
 description: 'ВНИМАНИЕ! Ребро соединяет КРАСНУЮ
 ю – мы перекрашиваем всё в один цвет!',
 type: 'warning'
 }, ...prev]));
 },
 // Step 8: Include A-D (Merge to purple)
 () => {
 setVertices(prev => prev.map(v => v.color === 'blue' ? {
 setEdges(prev => prev.map(e => e.status === 'included' ? {
 setLogs(prev => [{
 title: 'Слияние кланов: Ребро A-D добавлено',
 description: 'Мы объединили красный и синий кланы',
 type: 'success'
 }, ...prev]));
 },
 // Step 9: Inspect B-E (6) - Cycle!
 () => {
 setEdges(prev => prev.map(e => e.id === '1-4' ? {
 setLogs(prev => [{
 title: 'Шаг 5: Берем ребро B-E (вес 6)',
 description: 'НО: это ребро соединяет вершины B и E, что образует цикл!',
 type: 'warning'
 }, ...prev]));
 },
 // Step 10: Reject B-E
 () => {
 setEdges(prev => prev.map(e => e.id === '1-4' ? {
 setLogs(prev => [{
 title: 'Отказ: Цикл обнаружен!',
 description: 'Мы выбрасываем ребро B-E. Иначе мы бы получили цикл B-A-D-E-B'
 }, ...prev]));
 },
 // Step 11: Add B-E (6)
 () => {
 setEdges(prev => prev.map(e => e.id === '1-4' ? {
 setLogs(prev => [{
 title: 'Шаг 6: Добавляем ребро B-E (вес 6)',
 description: 'Поскольку ребро B-E не образует цикла, мы можем его добавить. Теперь все вершины имеют один цвет (фиолетовый).',
 type: 'success'
 }, ...prev]));
 },
 // Step 12: Done!
 () => {
 setEdges(prev => prev.map(e => e.id === '1-4' ? {
 setLogs(prev => [{
 title: 'Готово!',
 description: 'Алгоритм Крускала завершён. Мы построили минимальное остовное дерево.',
 type: 'success'
 }, ...prev]));
 }
], []);
}

```



```

 type: 'success'
 }, ...prev]);
},
// Step 15: Inspect C-F (9) - Cycle!
() => {
 setEdges(prev => prev.map(e => e.id === '2-5' ? {
 setLogs(prev => [{
 title: 'Шаг 8: Берем ребро C-F (вес 9)',
 description: 'Оно соединяет вершины C и F, кото
 type: 'warning'
 }, ...prev]);
},
// Step 16: Reject C-F
() => {
 setEdges(prev => prev.map(e => e.id === '2-5' ? {
 setLogs(prev => [{
 title: 'Отказ: Цикл обнаружен!',
 description: 'Выбрасываем ребро C-F.',
 type: 'error'
 }, ...prev]);
},
// Step 17: Inspect G-H (10)
() => {
 setEdges(prev => prev.map(e => e.id === '6-7' ? {
 setLogs(prev => [{
 title: 'Шаг 9: Берем ребро G-H (вес 10)',
 description: 'Соединяет фиолетовую G и бесцветн
 type: 'info'
 }, ...prev]);
},
// Step 18: Include G-H
() => {
 setVertices(prev => prev.map(v => v.id === 7 ? {
 setEdges(prev => prev.map(e => e.id === '6-7' ? {
 setLogs(prev => [{
 title: 'Успех: Ребро G-H в остовете',
 description: 'Вершина H перекрашена в фиолетовы
 type: 'success'
 }

```

```

 }, ...prev]);
 },
];

// --- PRIM STEPS ---
const primSteps = [
 // Step 1: Start at E
 () => {
 setVertices(prev => prev.map(v => v.id === 4 ? {
 setHeapQueue(['D-E (3)', 'B-E (6)', 'E-F (7)', 'E-
 setEdges(prev => prev.map(e => ['3-4', '1-4', '4-
 setLogs(prev => [{
 title: 'Шаг 1: Плесень зарождается в Токио (Вер
 description: 'Мы выбрали вершину E как стартовую
 ритетную очередь (Heap): D-E(3), B-E(6), E-F(
 type: 'info'
 }, ...prev]);
 },
 // Step 2: Pop D-E (3)
 () => {
 setEdges(prev => prev.map(e => e.id === '3-4' ? {
 setLogs(prev => [{
 title: 'Шаг 2: Куча выдает самое дешевое ребро
 description: 'Плесень тянется по ребру D-E к вер
 type: 'info'
 }, ...prev]);
 },
 // Step 3: Include D-E, add D's edges to heap
 () => {
 setVertices(prev => prev.map(v => v.id === 3 ? {
 setEdges(prev => prev.map(e => e.id === '3-4' ? {
 ..e, status: 'in_heap' } : e));
 setHeapQueue(['A-D (5)', 'B-E (6)', 'E-F (7)', 'D
 setLogs(prev => [{
 title: 'Успех: Плесень захватила вершину D',
 description: 'В кучу добавлены новые пути из D:
 type: 'success'
 }, ...prev]);
 }, ...prev]);

```

```

 title: 'Успех: Вершина В захвачена плесенью',
 description: 'В кучу добавлено В-С(4). Ребро В-С',
 type: 'success'
 }, ...prev));
},
// Step 8: Pop В-С (4)
() => {
 setEdges(prev => prev.map(e => e.id === '1-2' ? {
 setLogs(prev => [{
 title: 'Шаг 5: Куча выдает ребро В-С (вес 4)',
 description: 'Плесень тянется к вершине С.',
 type: 'info'
 }, ...prev]));
},
// Step 9: Include В-С, add С's edges
() => {
 setVertices(prev => prev.map(v => v.id === 2 ? {
 setEdges(prev => prev.map(e => e.id === '1-2' ? {
 eap' } : e));
 setHeapQueue(['В-Е (6 - цикл)', 'Е-Е (7)', 'D-G (4)']);
 setLogs(prev => [{
 title: 'Успех: Вершина С захвачена',
 description: 'Ребро С-Е(9) добавлено в кучу.',
 type: 'success'
 }, ...prev]));
},
// Step 10: Pop В-Е (6) - Cycle!
() => {
 setEdges(prev => prev.map(e => e.id === '1-4' ? {
 setLogs(prev => [{
 title: 'Шаг 6: Куча выдает ребро В-Е (вес 6)',
 description: 'ВНИМАНИЕ! Плесень проверяет вершину В',
 type: 'warning'
 }, ...prev]));
},
// Step 11: Reject В-Е
() => {
 setEdges(prev => prev.map(e => e.id === '1-4' ? {

```

```

() => {
 setVertices(prev => prev.map(v => v.id === 6 ? {
 setEdges(prev => prev.map(e => e.id === '3-6' ? {
 eap' } : e));
 setHeapQueue(['C-F (9 - цикл)', 'G-H (10)', 'E-H
 setLogs(prev => [{
 title: 'Успех: Вершина G захвачена',
 description: 'В кучу добавлено G-H(10).',
 type: 'success'
 }, ...prev]));
},
// Step 16: Pop C-F (9) - Cycle!
() => {
 setEdges(prev => prev.map(e => e.id === '2-5' ? {
 setLogs(prev => [{
 title: 'Шаг 9: Куча выдает ребро C-F (вес 9)',
 description: 'Вершины C и F уже заражены плесенью',
 type: 'warning'
 }, ...prev]));
},
// Step 17: Reject C-F
() => {
 setEdges(prev => prev.map(e => e.id === '2-5' ? {
 setHeapQueue(['G-H (10)', 'E-H (11)', 'F-I (13)'])
 setLogs(prev => [{
 title: 'Отказ: Игнорируем ребро C-F',
 description: 'Выбрасываем из кучи.',
 type: 'error'
 }, ...prev]));
},
// Step 18: Pop G-H (10)
() => {
 setEdges(prev => prev.map(e => e.id === '6-7' ? {
 setLogs(prev => [{
 title: 'Шаг 10: Куча выдает ребро G-H (вес 10)',
 description: 'Плесень ползет к вершине H.',
 type: 'info'
 }, ...prev]));
}

```

```

 type: 'info'
 }, ...prev]);
 },
 // Step 23: Include H-I
 () => {
 setVertices(prev => prev.map(v => v.id === 8 ? {
 setEdges(prev => prev.map(e => e.id === '7-8' ? {
 setHeapQueue(['F-I (13 - цикл)']));
 setLogs(prev => [{
 title: 'Успех: Плесень захватила весь граф
 description: 'Все 9 вершин поглощены. Итоговый
 иной – мы разрастались из одной точки с помо
 type: 'success'
 }, ...prev]));
 }, ...prev]);
 },
];

// --- BORUVKA STEPS ---
const boruvkaSteps = [
 // Step 1: Five-Year Plan 1 (Inspecting outgoing)
 () => {
 // Highlight the cheapest outgoing edges for EACH
 // Node 0(A) -> A-B(2)
 // Node 1(B) -> A-B(2)
 // Node 2(C) -> B-C(4)
 // Node 3(D) -> D-E(3)
 // Node 4(E) -> D-E(3)
 // Node 5(F) -> E-F(7)
 // Node 6(G) -> D-G(8)
 // Node 7(H) -> G-H(10)
 // Node 8(I) -> H-I(12)
 setEdges(prev => prev.map(e =>
 ['0-1', '3-4', '1-2', '4-5', '3-6', '6-7', '7-8
));
 setLogs(prev => [{
 title: 'Первая пятилетка: Каждая деревня выбира
 description: 'Каждая из 9 деревень одновременно
 А выбирает A-B(2), D выбирает D-E(3), а G вы

```

```

 -D с весом 5. Остальные мосты B-E(6) и C-F(9)
 type: 'warning'
 }, ...prev]);
},
// Step 4: Concurrently merge into single Kolkhoz
() => {
 setVertices(prev => prev.map(v => ({ ...v, color:
 setEdges(prev => prev.map(e =>
 e.status === 'included' || e.id === '0-3' ? { .
 }));
 setLogs(prev => [{
 title: 'Объединение завершено: Один гигантский
 description: 'Оба колхоза объединились по мосту
 type: 'success'
 }, ...prev]);
},
// Step 5: Check remaining edges & mark as rejected
() => {
 setEdges(prev => prev.map(e => e.status === 'pend
 setLogs(prev => [{
 title: ' Успех: MST построен Борувкой за 2 шаг
 description: 'Все лишние ребра (B-E, E-H, C-F,
 вес: 51. Благодаря параллельности, мы потрати
 type: 'success'
 }, ...prev]);
}
];

const currentSteps =
 activeAlgo === 'kruskal' ? kruskalSteps :
 activeAlgo === 'prim' ? primSteps :
 boruvkaSteps;

const handleNextStep = () => {
 if (stepIndex < currentSteps.length) {
 currentSteps[stepIndex]();
 setStepIndex(stepIndex + 1);
 }
}

```

```

switch (color) {
 case 'red': return 'bg-red-500 border-red-300 text-white';
 case 'blue': return 'bg-blue-500 border-blue-300 text-white';
 case 'purple': return 'bg-purple-600 border-purple-300 text-white';
 case 'mold': return 'bg-emerald-500 border-emerald-300 text-white';
 default:
 if (activeAlgo === 'prim' && id === 4 && stepIndex === 0)
 return 'bg-slate-700 border-emerald-500 text-white';
 return 'bg-slate-700 border-slate-500 text-slate-200';
}
};

```

```

const getEdgeStroke = (status: string, color: string) => {
 if (status === 'inspecting') return '#f59e0b';
 if (status === 'rejected') return '#ef4444';
 if (status === 'in_heap') return '#0284c7';
 if (status === 'included') {
 if (color === 'red') return '#ef4444';
 if (color === 'blue') return '#3b82f6';
 if (color === 'purple') return '#a855f7';
 if (color === 'mold') return '#10b981';
 }
 return '#475569';
};

```

```

return (
 <div className="space-y-12">

 {/* 3-in-1 MST Simulator Box */}
 <div className="bg-slate-900/90 border border-slate-800 p-4">
 <div className="flex flex-col lg:flex-row justify-between">
 <div>
 <div className="flex items-center gap-2 mb-2">

 Сунер-Интерактив 3 в 1


```

```

 className={
 "flex items-center gap-1.5 px-3 py-2
+
 (activeAlgo === 'boruvka'
 ? 'bg-rose-600 text-white shadow-md
 : 'text-slate-400 hover:text-slate-400
)
 }
 >
 <Users className="w-3.5 h-3.5" />
 Борувка (Билет 20)
 </button>
 </div>

 <button
 onClick={() => handleReset(activeAlgo)}
 className="flex items-center justify-center gap-2
 -200 font-semibold text-xs rounded-xl border-1
 >
 <RotateCcw className="w-3.5 h-3.5" />
 Сбросить
 </button>

 <button
 onClick={handleNextStep}
 disabled={stepIndex >= currentSteps.length}
 className={
 "flex items-center justify-center gap-2
initial cursor-pointer " +
 (stepIndex >= currentSteps.length
 ? "bg-slate-800 text-slate-500 border-1
 : activeAlgo === 'kruskal'
 ? "bg-indigo-600 hover:bg-indigo-500
 : activeAlgo === 'prim'
 ? "bg-emerald-600 hover:bg-emerald-500
 : "bg-rose-600 hover:bg-rose-500 text-white
)
 }
 >
 <SkipForward className="w-4 h-4" />

```





```

 </div>
)})
 </div>

 {stepIndex >= currentSteps.length && (
 <div className="absolute inset-x-6 bottom
 shadow-xl z-20">
 <p className={"font-bold text-base " +
 'ld-300' : 'text-rose-300'}>
 MST построено с помощью {activeAlgo}
 </p>
 <p className="text-slate-300 text-xs mt
 Общий вес минимального остова: 51. Вс
 </p>
 </div>
)}
 </div>

 {/* Logs / Heap */}
 <div className="bg-slate-950 rounded-2xl border
 <div className="p-4 bg-slate-900 border-b b
 <h4 className="font-bold text-slate-200 t
 Ход выполнения
 </h4>
 </div>

 {activeAlgo === 'prim' && (
 <div className="p-3 bg-slate-900/60 borde
 <p className="text-[11px] font-bold tex
 ⚡ Приоритетная очередь (Min-Heap):
 </p>
 <div className="flex flex-wrap gap-1.5"
 {heapQueue.length === 0 ? (
 <span className="text-xs text-slate
) : (
 heapQueue.map(({item, idx}) => (
 <span key={idx} className={"px-2 p
 0 shadow-md animate-pulse" : "bg-slate-800

```

```

 /* Cheat Sheet: Complexity & Code for all 3 algo
 <div className="bg-slate-900/80 border border-sla
 <div className="flex flex-col sm:flex-row items
 <div className="flex items-center gap-2.5">
 <BookOpen className="w-5 h-5 text-indigo-400
 <h4 className="text-xl font-bold text-white
 </div>

 <div className="flex bg-slate-950 p-1 rounded
 <button
 onClick={() => setActiveCheatTab('kruskal
 className={"px-3 py-1.5 rounded text-xs f
 == 'kruskal' ? "bg-indigo-600 text-white" : "
 >
 Краскал (Билет 18)
 </button>
 <button
 onClick={() => setActiveCheatTab('prim')}
 className={"px-3 py-1.5 rounded text-xs f
 == 'prim' ? "bg-emerald-600 text-white" : "
 >
 Прима (Билет 19)
 </button>
 <button
 onClick={() => setActiveCheatTab('boruvka
 className={"px-3 py-1.5 rounded text-xs f
 == 'boruvka' ? "bg-rose-600 text-white" : "
 >
 Борувка (Билет 20)
 </button>
 </div>
 </div>

 {activeCheatTab === 'kruskal' && (
 <div className="grid grid-cols-1 lg:grid-cols
 <div className="lg:col-span-1 space-y-4">
 <div className="bg-slate-950 p-4 rounded-
 <p className="text-slate-400 text-xs fo

```

```
2. Жадный выбор ребер
```

```
edges.sort() # $O(E \log E)$
```

```
mst = []
```

```
for w, u, v in edges:
```

```
 if union(u, v):
```

```
 mst.append((u, v, w))`}
```

```
 </pre>
```

```
</div>
```

```
</div>
```

```
</div>
```

```
}}
```

```
{activeCheatTab === 'prim' && {
```

```
 <div className="grid grid-cols-1 lg:grid-cols-
```

```
 <div className="lg:col-span-1 space-y-4">
```

```
 <div className="bg-slate-950 p-4 rounded-
```

```
 <p className="text-slate-400 text-xs for
```

```
 <Clock className="w-3.5 h-3.5 text-em
```

```
 </p>
```

```
 <p className="text-2xl font-black text-r
```

```
 <p className="text-slate-400 text-xs mt
```

```
 </div>
```

```
 <div className="bg-slate-950 p-4 rounded-
```

```
 <p className="text-slate-400 text-xs for
```

```
 <Award className="w-3.5 h-3.5 text-em
```

```
 </p>
```

```
 <p className="text-2xl font-black text-r
```

```
 <p className="text-slate-400 text-xs mt
```

```
 </div>
```

```
 <div className="bg-slate-950 p-4 rounded-
```

```
 <p className="text-slate-400 text-xs for
```

```
 ложении:</p>
```

```
 <p className="text-xs text-slate-300 le
```

```
 ие чистые вершины через самое дешевое досту
```

```
 </div>
```

```
</div>
```

```
<div className="lg:col-span-2">
```

```

 p>
 </div>
 <div className="bg-slate-950 p-4 rounded-1">
 <p className="text-slate-400 text-xs font-medium">
 <Award className="w-3.5 h-3.5 text-rose-500">
 </p>
 <p className="text-2xl font-black text-white">
 <p className="text-slate-400 text-xs mt-1">
 </div>
 <div className="bg-slate-950 p-4 rounded-1">
 <p className="text-slate-400 text-xs font-medium">
 ении:</p>
 <p className="text-xs text-slate-300 leading-5">
 , организовав массовое распараллеленное слияние
 </p>
 </div>
 </div>
 <div className="lg:col-span-2">
 <div className="bg-slate-950 p-4 rounded-1">
 <p className="text-slate-400 text-xs font-medium">
 <Code className="w-3.5 h-3.5 text-rose-500">
 </p>
 <pre className="text-xs text-emerald-400 font-mono">
 80 flex-1">
{`def boruvka(n, edges):
 parent = list(range(n))
 mst = []
 while len(mst) < n - 1:
 # Для каждого колхоза ищем самый дешёвый мост на
 cheapest = [-1] * n
 for u, v, w in edges:
 ru, rv = find(u), find(v)
 if ru != rv:
 if cheapest[ru] == -1 or cheapest[ru][2] > w:
 cheapest[ru] = (u, v, w)
 if cheapest[rv] == -1 or cheapest[rv][2] > w:
 cheapest[rv] = (u, v, w)
 # Объединяем все колхозы по выбранным мостам
 for bridge in cheapest:
 if bridge != -1 and union(bridge[0], bridge[1]):
 mst.append(bridge)`}

```

```

 title: ' Фура по Болоту (Ф-Б)',
 desc: 'Медленный, тяжёлый — зато тащит',
 highlight: 'отрицательные',
 highlightColor: 'text-rose-400',
 rest: 'веса и детектит отрицательные циклы.',
 complexity: 'O(V · E)',
 border: 'border-rose-500/30 hover:border-rose-400/60',
 titleColor: 'text-rose-400',
 badge: 'text-rose-400/70 bg-rose-950/40',
 },
 {
 img: floydImg,
 alt: 'Фсе-ко-Фсем Флойд',
 title: ' Фсе ко Фсем (Флойд)',
 desc: 'Матрица + три цикла (',
 highlight: 'k, i, j',
 highlightColor: 'text-emerald-400',
 rest: '>'. Ищет пути от всех ко всем.',
 complexity: 'O(V³)',
 border: 'border-emerald-500/30 hover:border-emerald-400/60',
 titleColor: 'text-emerald-400',
 badge: 'text-emerald-400/70 bg-emerald-950/40',
 },
];

```

```

export default function MnemonicCards() {
 return (
 <div className="grid grid-cols-1 md:grid-cols-3 gap-4">
 {cards.map((c, i) => (
 <div key={i} className={`bg-slate-800 rounded-2xl p-4">
 <div className="h-48 overflow-hidden">
 <img src={c.img} alt={c.alt}
 className="w-full h-full object-cover grid-item">
 </div>
 <div className="p-4">
 <h3 className={`text-lg font-bold mb-1 ${c.titleColor}`>{c.title}</h3>
 <p className="text-slate-400 text-sm">{c.desc} {c.highlight}

```

```
document.body.style.overflow = "";
});
}, [open]));

useEffect(() => {
 const onKeyDown = (e: KeyboardEvent) => e.key === "Escape";
 window.addEventListener("keydown", onKeyDown);
 return () => window.removeEventListener("keydown", onKeyDown);
}, []));

return (
 <>
 <div className="lg:hidden sticky top-[57px] z-40">
 <button
 onClick={() => setOpen(true)}
 className="w-full flex items-center gap-2.5 p-2 bg-white rounded-lg shadow-md focus:outline-none"
 aria-label="Открыть содержание"
 >

 <List className="w-4 h-4" />

 Содержание · {index + 1} из {total}
 {currentSection}

 </button>
 </div>

 {open && (
 <div className="lg:hidden fixed inset-0 z-[90]">
 <div className="absolute inset-0 bg-slate-950">
 <div className="absolute inset-y-0 left-0 w-[80%] max-h-[calc(100vh-
te-[tocIn_.18s_ease-out])]>
 <div className="flex items-center justify-between p-4">
 Содержание
 <button
```





```
 title="Скачать полный TXT файл со всем кодом"
 >
 <FileText className="w-4 h-4" /> TXT

 <button
 id="download-html-btn"
 onClick={saveBook}
 disabled={busy}
 className="flex items-center justify-center
 er:bg-amber-600/20 border border-amber-500/
 title="Скачать всё пособие одним автономным
 >
 {busy ? <Loader2 className="w-4 h-4 animate
 </button>
 </div>
 </div>
</header>
);
};
```

---

ФАЙЛ 54/81: src/components/PlanarityDemo.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

---

```
export function PlanarityDemo() {
 return (
 <div className="space-y-4">
 <h4 className="text-base font-bold text-white">K5
 <p className="text-xs text-slate-400">Просто смотри
 <div className="grid grid-cols-1 md:grid-cols-2 gap-4">
 { /* K5 */ }
 <div className="bg-slate-950 rounded-xl p-4 border
 <div className="absolute top-2 left-2 bg-slate-
 00/30 w-auto z-20">K5 – НЕПЛАНАРЕН</div>
 <svg className="w-full h-full absolute inset-0">
```

```

 stroke={xing ? "#f43f5e" : "#4f46e5"}
 }
 const labels = ["A", "B", "C", "D", "E"];
 for(const p of pts) {
 circles.push(<g key={`c${p.id}`}>
 <circle cx={p.x} cy={p.y} r={8} fill=
 <text x={p.x} y={p.y+3} textAnchor="m
 </g>);
 }
 return <>{lines}{circles}</>;
 })())
 </svg>
 <div className="absolute bottom-2 left-2 right
 z-20">
 V=5, E=10 {'>'} 3V-6 = 9 {'-->'} {'\u274C'}
 </div>
</div>

{/* K3,3 */}
<div className="bg-slate-950 rounded-xl p-4 bor
 <div className="absolute top-2 left-2 bg-slate
 00/30 w-auto z-20">K3,3 – НЕПЛАНАРЕН</div>
 <svg className="w-full h-full absolute inset-
 {(() => {
 const pts = [
 {id:0,x:60,y:60},{id:1,x:160,y:60},{id:
 {id:3,x:60,y:220},{id:4,x:160,y:220},{id
];
 const edges = [
 [0,3],[0,4],[0,5],
 [1,3],[1,4],[1,5],
 [2,3],[2,4],[2,5]
];
 function findPt(id:number) { return pts.f
 function intersect(a:number,b:number,c:nu
 if (a===c||a===d||b===c||b===d) return
 const p1=findPt(a),p2=findPt(b),p3=find
 if (!p1||!p2||!p3||!p4) return false;

```

△ Запомни:  $K5$  и  $K3,3$  – два кита непланарности.  
гомеоморфный  $K5$  или  $K3,3$  – он непланарен. Теорема

```
 </div>
 </div>
);
}
```

---

ФАЙЛ 55/81: `src/components/QueueViz.tsx`

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

---

```
import React, { useState, useCallback } from 'react';
import { Sparkles } from 'lucide-react';
```

```
interface Customer {
 id: string;
 name: string;
 emoji: string;
 color: string;
 bubbleText: string;
 animState: 'in' | 'idle' | 'eating' | 'out';
}
```

```
const PRESETS = [
 { name: 'Студент Вася', emoji: '👨🎓', color: 'from-blue',
 bubbleText: 'Матана!' },
 { name: 'Кодер Семён', emoji: '👨💻', color: 'from-violet',
 bubbleText: 'Вариацию борща!' },
 { name: 'Кот Борис', emoji: '🐈', color: 'from-amber',
 bubbleText: 'Плиз.' },
 { name: 'Бабушка Люба', emoji: '👵', color: 'from-rose',
 bubbleText: '' },
 { name: 'Инопланетянин', emoji: '👽', color: 'from-emerald',
 bubbleText: 'Пиво для нашего НЛО!' },
 { name: 'Бизнесмен', emoji: '👔', color: 'from-slate',
 bubbleText: 'Тартапы.' },
];
```

```

 };

 setQueue(prev => [...prev, newGuest]);
 addLog(`- ENQUEUE: ${newGuest.name} ${newGuest.emoji}`);

 setTimeout(() => {
 setQueue(prev => prev.map(c => c.id === newGuest.id ? newGuest : c), 450);
 }, [queue, isServing]);

// DEQUEUE: Выдать суп первому (голова / Head, FIFO)
const dequeue = useCallback(() => {
 if (isServing) return;
 if (queue.length === 0) {
 setShake(true);
 addLog(`⚠ DEQUEUE: В очереди никого нет! Повар скучает`);
 setTimeout(() => setShake(false), 400);
 return;
 }

 setIsServing(true);
 const headGuest = queue[0];

 addLog(`- DEQUEUE: Повар наливает суп первому – ${headGuest.name} ${headGuest.emoji}`);

 // Step 1: Head guest gets eating animation
 setQueue(prev => prev.map((c, idx) => idx === 0 ? { ...c, emoji: '🍲' } : c));

 setTimeout(() => {
 // Step 2: Guest leaves with happy face, queue should be updated
 setQueue(prev => prev.map((c, idx) => idx === 0 ? { ...c, emoji: '😊' } : c));

 setServedHistory(prev => [`${headGuest.emoji} ${headGuest.name} поел`]);

 setTimeout(() => {
 setQueue(prev => prev.slice(1));
 setIsServing(false);
 addLog(`- ${headGuest.name} поел и ушел сытым.`);
 }, 400);
 }, 400);
}, [queue, isServing]);

```



```

queue.map((customer, idx) => {
 const isHead = idx === 0;
 const isTail = idx === queue.length
 return (
 <div
 key={customer.id}
 className={`
 flex flex-col items-center gap-4
 ${customer.animState === 'in' ? 'animate-in' : ''}
 ${customer.animState === 'eat' ? 'animate-eat' : ''}
 ${customer.animState === 'out' ? 'animate-out' : ''}
 `}
 >
 {/* Облачко мыслей */}
 <div className={`
 absolute -top-16 left-1/2 -transform
 8 text-center text-slate-200 leading-tight
 ${isHead ? 'opacity-100 border' : ''}
 `}>
 <div className="font-bold text-slate-200">
 {customer.bubbleText}
 <div className="absolute -bottom-16
 it rotate-45"></div>
 </div>

 {/* Персонаж */}
 <div className={`
 relative w-14 h-14 rounded-2xl border-2
 ${customer.color}
 ${isHead ? 'ring-4 ring-amber-400' : ''}
 `}>
 {customer.name}

 {/* Индексы HEAD/TAIL */}
 {(isHead || isTail) && (
 <span className={`
 absolute -bottom-2 text-[0.8em]

```

```

) : {
 <div className="flex flex-col gap-1.5 w-full"
 {servedHistory.map((item, idx) => {
 <div
 key={idx}
 className="w-full py-1.5 px-3 rounded-0 flex items-center gap-2 text-[11px] text-f
 >
 {item}
 </div>
 })}
 </div>
 })
 </div>

 {/* Пол */}
 <div className="w-full h-3 bg-gradient-to-r f
 </div>
</div>

{/* ЛОГ ДЕЙСТВИЙ */}
<div className="bg-slate-900 border border-slate-
 <div className="text-[10px] font-mono text-slate
 {log.map((entry, idx) => {
 <div
 key={idx}
 className={`text-xs font-mono flex items-ce
 >
 {idx === 0 ? <span className="text-indigo-5
 {entry}
 </div>
 })}
 </div>
</div>
);
};

```

```

 const blinkInterval = setInterval(() => {
 setIsBlinking(true);
 setTimeout(() => setIsBlinking(false), 200);
 }, 4000);
 return () => clearInterval(blinkInterval);
 }, []);

// Эффект разговора при смене текста
useEffect(() => {
 setIsTalking(true);
 const timer = setTimeout(() => setIsTalking(false),
 return () => clearTimeout(timer);
 }, [speechText]);

// Построение начального бора (Trie) без fail-ссылок
const buildInitialTrie = (wordList: string[]) => {
 const newNodes: { [key: number]: TrieNode } = {
 0: { id: 0, char: 'ROOT', parent: 0, children: {}
 };
 let count = 0;

 wordList.forEach((word) => {
 let curr = 0;
 for (let i = 0; i < word.length; i++) {
 const c = word[i].toUpperCase();
 if (newNodes[curr].children[c] === undefined) {
 count++;
 newNodes[curr].children[c] = count;
 newNodes[count] = {
 id: count,
 char: c,
 parent: curr,
 children: {},
 fail: 0,
 term_link: 0,
 is_terminal: false,
 words: [],
 depth: newNodes[curr].depth + 1

```



```

 setBfsQueue([]);
 setCurrentNode(null);
 setSpeechText(
 `Отлично! Я построил базовое префиксное дерево (БПД),
 чтобы расставить fail-ссылки!`
);
 };

useEffect(() => {
 buildInitialTrie(words);
}, []);

const handleAddWord = (e: React.FormEvent) => {
 e.preventDefault();
 const cleanWord = newWord.trim().toUpperCase().replace(' ', '');
 if (!cleanWord) return;
 if (words.includes(cleanWord)) {
 setSpeechText(`Слово «${cleanWord}» уже есть в словаре`);
 return;
 }
 const updated = [...words, cleanWord];
 setWords(updated);
 setNewWord('');
 buildInitialTrie(updated);
};

const removeWord = (wordToRemove: string) => {
 if (words.length <= 1) {
 setSpeechText('Оставь хотя бы одно слово, иначе словарь будет пустым');
 return;
 }
 const updated = words.filter((w) => w !== wordToRemove);
 setWords(updated);
 buildInitialTrie(updated);
};

// Запуск BFS
const startBFS = () => {

```

```

 newQueue.push(u);
 let v = rNode.fail;
 while (v !== 0 && updatedNodes[v].children[char] !== 0) {
 v = updatedNodes[v].fail;
 }
 const failTarget = updatedNodes[v].children[char];
 updatedNodes[u].fail = failTarget;

 // Рассчитываем терминальную ссылку (term_link)
 if (updatedNodes[failTarget].is_terminal) {
 updatedNodes[u].term_link = failTarget;
 } else {
 updatedNodes[u].term_link = updatedNodes[failTarget].term_link;
 }

 logs += `Для #${u} ('${char}') fail = #${failTarget}`;
 }

 if (childrenIds.length === 0) {
 logs += 'У этой вершины нет детей, просто переходим к следующему';
 }

 setNodes(updatedNodes);
 setBfsQueue(newQueue);
 setCurrentNode(r);
 setSpeechText(logs);
};

return (
 <div className="bg-slate-800/90 rounded-2xl p-6 border-1px border-slate-700" >
 {/* Шапка виджета */}
 <div className="flex flex-wrap items-center justify-between" >
 <div className="flex items-center space-x-3" >

 <div>
 <h4 className="text-2xl font-bold text-white" >
 Интерактивный конструктор: Говорящий Эхо-бот
 </h4>

```

```

 className="origin-[35px_100px] animate-
/>

 {/* Плавники */}
 <path d="М 90 55 L 70 30 L 110 55 Z" fill=
 <path d="М 90 145 L 70 170 L 110 145 Z" f
 <path
 d="М 120 115 L 100 135 L 130 130 Z"
 fill="#f87171"
 className="origin-[120px_115px] animate
/>

 {/* Глаз (с морганием) */}
 <circle cx="145" cy="85" r="12" fill="#ff
 {isBlinking ? (
 <line x1="133" y1="85" x2="157" y2="85"
) : (
 <
 <circle cx="148" cy="85" r="6" fill="
 <circle cx="151" cy="82" r="2" fill="
 </>
)}

 {/* Рот (анимация разговора) */}
 {isTalking ? (
 <path d="М 165 105 Q 155 120 165 125 Q
) : (
 <path d="М 165 110 Q 155 115 168 118" s
)}

 {/* Улыбка / жабры */}
 <path d="М 130 80 Q 125 100 130 120" stro
 <path d="М 120 85 Q 115 100 120 115" stro
</svg>

 {/* Декоративные пузыри */}
 <div className="absolute -top-1 right-4 tex
 <div className="absolute top-8 right-1 text

```

```

 })
 {simulationStep === 1 && (
 <button
 onClick={stepBFS}
 className="bg-blue-600 hover:bg-blue-500
 shadow-blue-900/40 transition-transform"
 >
 Шаг BFS

 </button>
 </div>
 })
 {simulationStep === 2 && (
 <button
 onClick={() => buildInitialTrie(words)}
 className="bg-indigo-600 hover:bg-indigo-500
 transition-transform active:scale-95"
 >
 <RotateCcw className="w-4 h-4" /> Перезагрузить
 </button>
 })
 </div>
</div>
</div>
</div>

{/* SVG Canvas for Automaton Tree */}
<div className="mb-8 bg-slate-900 border border-slate-800" >
 <h5 className="text-white font-bold mb-3 flex items-center">
 Дерево автомата
 Дерево и Суффиксные (fail) ссылки
 </h5>

 {(() => {
 let maxX = 0;
 let maxY = 0;
 Object.values(nodes).forEach(n => {
 if (n.x && n.x > maxX) maxX = n.x;
 if (n.y && n.y > maxY) maxY = n.y;
 });
 })}

```

```

 fontWeight="bold"
 >
 {node.char}
 </text>
 </g>
);
}}

{/* Draw Fail Links */}
{simulationStep > 0 && Object.values(nodes).map(node => {
 if (node.id === 0 || node.fail === 0) return null;
 // Wait, all nodes get fail link. Let's draw it at step
 // To avoid too many lines, let's draw it at step >= 1 and computed it
 const target = nodes[node.fail];
 if (!target || !node.x || !node.y || !node.fail) return null;

 // Simple bezier curve for fail link
 const isCurrent = currentNode === node.id;

 return (
 <path
 key={`fail-${node.id}`}
 d={`M ${node.x} ${node.y - 10} Q ${node.x + 16} ${node.y - 16} ${node.fail.x} ${node.fail.y - 10}`}
 fill="none"
 stroke={isCurrent ? '#f43f5e' : 'rgb(128, 128, 128)'}
 strokeWidth={isCurrent ? 2 : 1.5}
 strokeDasharray="4 4"
 markerEnd="url(#fail-arrow)"
 />
);
}}

{/* Draw Nodes */}
{Object.values(nodes).map(node => {
 if (!node.x || !node.y) return null;

```

```

 {!isRoot && {
 <text
 x={node.x + 12}
 y={node.y - 12}
 fill="#94a3b8"
 fontSize="9"
 >
 {node.id}
 </text>
 }}
 </g>
);
 }}}
</svg>
);
}}({})
</div>

{/* Управление словарем */}
<div className="grid grid-cols-1 lg:grid-cols-3 gap-4">
 <div className="bg-slate-900/50 p-5 rounded-xl">
 <div>
 <h5 className="text-white font-bold mb-3">
 Словарь (Паттерны)
 </h5>
 <div className="flex flex-wrap gap-2 mb-4">
 {words.map((w) => {
 <span
 key={w}
 className="bg-slate-800 border border-slate-700 padding-2 10px"
 >
 {w}
 <button
 onClick={() => removeWord(w)}
 className="text-slate-500 hover:text-white"
 title="Удалить"
 >

```

```

<div className="max-h-[220px] overflow-y-auto" style={{border: 1px solid #ccc; padding: 10px; margin-top: 10px;}}>
 <table className="w-full text-left border-collapse" style={{width: 100%; border-collapse: collapse;}}>
 <thead>
 <tr className="border-b border-slate-700" style={{border-bottom: 2px solid #334d5b;}}>
 <th className="py-2 px-3" style={{padding: 5px 10px;}}>ID Вершины</th>
 <th className="py-2 px-3" style={{padding: 5px 10px;}}>Символ</th>
 <th className="py-2 px-3" style={{padding: 5px 10px;}}>Родитель</th>
 <th className="py-2 px-3" style={{padding: 5px 10px;}}>fail-ссылка</th>
 <th className="py-2 px-3" style={{padding: 5px 10px;}}>term_link</th>
 <th className="py-2 px-3" style={{padding: 5px 10px;}}>Дети</th>
 <th className="py-2 px-3" style={{padding: 5px 10px;}}>Слова (dict)</th>
 </tr>
 </thead>
 <tbody className="divide-y divide-slate-800" style={{border-top: 1px solid #334d5b;}}>
 {Object.values(nodes).map((node) => {
 const isCurrent = currentNode === node.id
 const isInQueue = bfsQueue.includes(node.id)

 return (
 <tr>
 <td>
 {key={node.id}
 className={\`transition-colors ${
 isCurrent
 ? 'bg-blue-900/40 font-bold text-white'
 : isInQueue
 ? 'bg-slate-800/80'
 : 'hover:bg-slate-800/40'
 }\`}
 </td>
 <td className="py-2.5 px-3 flex items-center" style={{padding: 5px 10px;}}>

 {
 className={\`w-2 h-2 rounded-full ${
 node.id === 0
 ? 'bg-purple-500'
 : isCurrent
 ? 'bg-blue-400 animate-pulse'
 : isInQueue
 ? 'bg-amber-400'

```

```

 .join(', ')}
 </td>
 <td className="py-2.5 px-3">
 {node.words.length === 0 ? (

 {node.words.join(', ')}

) : (

 {node.words.join(', ')}

)}
 </td>
 </tr>
</tbody>
</table>
</div>
</div>
</div>
</div>
);
};

```

ФАЙЛ 57/81: src/components/SegmentTreeVisualizer.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```

import React, { useState, useEffect, useCallback, useMemo } from 'react';
import { Play, Pause, RotateCcw, ChevronLeft, ChevronRight } from 'lucide-react';

```

```

// ===== TYPES =====

```

```

interface Step {
 id: number;
 desc: string;
 codeLine: number;
 treeState: Record<number, number>; // node index
}

```



```

 " tree[v] = tree[2v]+tree[2v+1]", // 8
 "}", // 9
];
void CODE;

function go(v: number, l: number, r: number) {
 if (l === r) {
 steps.push({ id: id++, desc: `build(${v}, ${l}, ${r})`,
 , highlightNodes: [v], arrayHighlight: [l, r] });
 tree[v] = arr[l];
 steps.push({ id: id++, desc: `tree[${v}] = ${arr[l]}`,
 l, r] });
 return;
 }
 const m = (l + r) >> 1;
 steps.push({ id: id++, desc: `build(${v}, ${l}, ${r})`,
 { ...tree }, highlightNodes: [v], arrayHighlight: [l, r] });
 go(2 * v, l, m);
 steps.push({ id: id++, desc: `Вернулись в build(${v}, ${l}, ${r})`,
 lightNodes: [v], arrayHighlight: [l, r] });
 go(2 * v + 1, m + 1, r);
 tree[v] = (tree[2 * v] ?? 0) + (tree[2 * v + 1] ?? 0);
 steps.push({ id: id++, desc: `Объединяем: tree[${v}] = tree[2 * v] + tree[2 * v + 1]`,
 tate: { ...tree }, highlightNodes: [v], mergeChildren: [2 * v, 2 * v + 1] });
}
go(1, 0, n - 1);
steps.push({ id: id++, desc: `Дерево построено! Корень: tree[1]`,
 Highlight: [0, n - 1] });
return steps;
}

// 2. Top-Down Query steps
function genQueryTopDownSteps(arr: number[], qL: number, qR: number) {
 const n = arr.length;
 const tree = buildTreeFull(arr);
 const steps: Step[] = [];
 let id = 0;

```

```

let r = qR + n + 1; // half-open [l, r)
let res = 0;

steps.push({ id: id++, desc: `Старт: l = qL + n = ${qL + n}, codeLine: 3, treeState: tree, highlightNodes: [l]`});

while (l < r) {
 const highlights: number[] = [];
 if (l & 1) {
 res += tree[l] ?? 0;
 steps.push({ id: id++, desc: `l=${l} нечётный → l`, codeLine: 5, treeState: tree, highlightNodes: [l]`});
 highlights.push(l);
 l++;
 }
 if (r & 1) {
 r--;
 res += tree[r] ?? 0;
 steps.push({ id: id++, desc: `r=${r + 1} нечётный`, codeLine: 5, treeState: tree, highlightNodes: [r], arrayHighlight: [qL, qR]`});
 highlights.push(r);
 }
 l >>= 1;
 r >>= 1;
 if (l < r) {
 steps.push({ id: id++, desc: `Поднимаемся: l = ${qL}, codeLine: 5, treeState: tree, highlightNodes: [l], arrayHighlight: [qL, qR], result: res`});
 highlights.push(l);
 }
}
steps.push({ id: id++, desc: `Запрос sum([${qL}..${qR}], codeLine: 5, treeState: tree, highlightNodes: [1], arrayHighlight: [qL, qR], result: res`});
return steps;
}

```

// ===== TREE RENDERING HELPER =====

interface VNode { v: number; l: number; r: number; val:

function buildVisualTree(treeState: Record<number, number>



```

 { /* Header */ }
 <div className={`px-4 py-2.5 flex items-center justify-between border-b-2 border-emerald-950/50' : 'bg-amber-950/50'} border-b-2 border-emerald-950/50`>
 <h4 className="font-bold text-sm text-white flex-1">

 </div>

 { /* Array strip */ }
 <div className="px-3 py-2 bg-slate-950 border-b-2 border-indigo-500">
 {arr.map((v, i) => {
 const inRange = step.arrayHighlight && i >= step.start
 return (
 <div key={i} className={`flex flex-col items-center justify-between p-2 border-indigo-500 text-indigo-200' : 'bg-indigo-950 border-indigo-500' -translate-y-0.5` : 'bg-slate-900 border-indigo-500'}`>
 {v}
 {i}
 </div>
);
 })}
 </div>

 { /* Tree */ }
 <div className="flex-1 p-2 overflow-x-auto bg-slate-950">
 <div className="scale-[0.85] origin-top min-w-max">
 </div>

 { /* Code */ }
 <pre className="px-3 py-2 text-[9px] md:text-[10px] font-mono">
 {code.map((line, i) => {
 const ln = i + 1;
 const active = step.codeLine === ln;
 return (
 <div key={ln} className={`px-2 py-0.5 rounded border border-indigo-500 text-indigo-200' : 'bg-amber-600/20 border-l-2 border-amber-600/20'}`>
 {ln}
 {line}</div>
);
 })}
 </pre>

```

```

 <option value={120}>Турбо</option>
 </select>
 </div>

 {/* Progress bar */}
 <div className="h-1 bg-slate-950"><div className=
 + 1) / total) * 100 : 0}%` }} /></div>
 </div>
);
}

// =====
// MAIN EXPORTED COMPONENT
// =====
export function SegmentTreeVisualizer() {
 const [arr, setArr] = useState<number[]>([5, 8, 3, 12]);
 const [customInput, setCustomInput] = useState('5, 8, 3, 12');
 const [qL, setQL] = useState(1);
 const [qR, setQR] = useState(4);
 const [view, setView] = useState<'build' | 'queries'>('build');

 // Clamp query range when array changes
 useEffect(() => {
 setQL(prev => Math.min(prev, arr.length - 1));
 setQR(prev => Math.min(prev, arr.length - 1));
 }, [arr]);

 const handleApply = (e: React.FormEvent) => {
 e.preventDefault();
 const parsed = customInput.split(',').map(s => parseInt(s));
 if (parsed.length >= 2 && parsed.length <= 16) {
 setArr(parsed);
 }
 };

 const randomize = () => {
 const size = arr.length;
 const a = Array.from({ length: size }, () => Math.floor(

```

```


)))

 </div>
 </form>

 { /* Query range */ }
 <div className="bg-slate-950 p-4 rounded-xl border border-slate-800">
 <label className="text-[10px] font-bold uppercase">
 <div className="flex items-center gap-2">
 <div className="flex items-center gap-1">

 <input
 type="number"
 min={0}
 max={arr.length - 1}
 value={qL}
 onChange={e => setQL(Math.min(Number(e.target.value), arr.length - 1))}
 className="w-14 bg-slate-900 border border-slate-800 focus:border-indigo-500"
 />
 </div>
 <div className="flex items-center gap-1">

 <input
 type="number"
 min={0}
 max={arr.length - 1}
 value={qR}
 onChange={e => setQR(Math.min(Number(e.target.value), arr.length - 1))}
 className="w-14 bg-slate-900 border border-slate-800 focus:border-indigo-500"
 />
 </div>
 </div>
 <div className="text-[10px] text-slate-500 font-medium">
 Запрос: $\text{sum}(A[\{qL\}..\{qR\}]) = \{arr.\text{slice}(qL, qR + 1).reduce((a, b) => a + b, 0)}\}$
 </div>
 </div>

```

```

 <SimPanel
 key={`qtd-${arr.join(' - ')}-${qL}-${qR}`}
 title={`Запрос sum([${qL}..${qR}]) – Сверху`}
 icon="↑"
 steps={queryTDSteps}
 code={QUERY_TD_CODE}
 color="emerald"
 arr={arr}
 />
 <SimPanel
 key={`qbu-${arr.join(' - ')}-${qL}-${qR}`}
 title={`Запрос sum([${qL}..${qR}]) – Снизу`}
 icon="↓"
 steps={queryBUSteps}
 code={QUERY_BU_CODE}
 color="amber"
 arr={arr}
 />
 </div>
})

```

```

{view === 'theory' && {
 <div className="space-y-5">
 {/* Why it splits this way */}
 <div className="bg-slate-950 p-5 rounded-xl b...
 <h4 className="text-base font-bold text-whi...
 во разбилось именно так для N={arr.length}?
 <p className="text-sm text-slate-300 leadin...
 Каждый узел берёт <code className="bg-sla...
 к получает <code className="bg-slate-900 px...
 -slate-900 px-1 rounded font-mono text-emerg...
 евая часть забирает на 1 больше (округление
 </p>
 <div className="bg-slate-900 p-4 rounded-lg...
 <div className="text-slate-400">Корень: L
 <div className="text-slate-300">mid = |{0
 <div className="text-indigo-300">→ Левый:
 <div className="text-emerald-300">→ Правый:

```

```

 <div className="flex justify-between border-1 border-emerald-400 p-5"
 ><span className="text-emerald-400 font-monospace text-2xl font-weight-normal"
 ><div className="flex justify-between" >x сложно</div>
 </div>
 </div>
 </div>

 { /* Verdict */ }
 <div className="bg-slate-950 p-5 rounded-xl border-1 border-slate-800"
 ><h4 className="font-bold text-blue-400 mb-2"
 ><p className="text-sm text-slate-300 leading-relaxed"
 ><strong className="text-white">Итеративный (слева-справа) и рекурсивный
 калькуляторы. Но если нужны <strong className="text-white">рекурсивный (сверху-вниз)
 </p>
 </div>
 </div>
)}
</div>
);
}

```

ФАЙЛ 58/81: src/components/Sidebar.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

```

import React, { useMemo, useState } from "react";
import { ChevronRight, Compass, Search, Sparkles, X } from "lucide-react";
import { chapters } from "../data/content";
import { VIZ_REGISTRY } from "../vizRegistry";

```

```

interface SidebarProps {
 selectedChapterId: string;
 setSelectedChapterId: (id: string) => void;
 /** Мобильный режим: список живёт в выдвижной панели */
}

```



```

 >
 <X className="w-3.5 h-3.5" />
 </button>
 })
 </div>
</div>

<div className="flex-1 overflow-y-auto px-3 pb-4" >
 {groups.length === 0 && <p className="text-sm text-gray-500">
 Нет данных
 </p> ||
 {groups.map(([category, items]) => (
 <div key={category}>
 <div className="text-[10px] font-bold uppercase">{category}</div>
 <div className="space-y-1">
 {items.map((chapter) => {
 const isSelected = chapter.id === selectedChapterId
 const hasViz = Boolean(VIZ_REGISTRY[chapter.id])
 return (
 <button
 key={chapter.id}
 onClick={() => {
 setSelectedChapterId(chapter.id);
 onNavigate?.();
 window.scrollTo({ top: 0, behavior: 'smooth' });
 }}
 className={`w-full text-left px-3 py-2 ${
 isSelected
 ? "bg-indigo-600/15 border-indigo-600/20"
 : "bg-slate-800/40 border-transparent"
 }`}
 >{chapter.title}</button>
)
 })}
 </div>
 </div>
)}
 </div>
 {hasViz && (
 <span
 className="w-1.5 h-1.5 rounded-full border-1px solid #ccc"
 title="Есть интерактивная визуализация"
 />
)}
 Визуализация

```

```

 id: string;
 name: string;
 icon: string;
}

interface Person {
 id: string;
 name: string;
 icon: string;
}

interface Hypothesis {
 id: string;
 text: string;
 probability: number;
}

export const Simulator: React.FC = () => {
 const [activeTab, setActiveTab] = useState<'stack' |

 // Stack state
 const [stack, setStack] = useState<Plate[]>([
 { id: '1', name: 'Тарелка из-под пасты', icon: ' '
 { id: '2', name: 'Тарелка из-под пиццы', icon: ' '
 { id: '3', name: 'Блюдец от торта', icon: ' ' },
]));
 const [popMessage, setPopMessage] = useState<string |

 // Queue state
 const [queue, setQueue] = useState<Person[]>([
 { id: '1', name: 'Студент Вася', icon: ' ' },
 { id: '2', name: 'Голодный кодер', icon: ' ' },
 { id: '3', name: 'Кот Борис', icon: ' ' },
]));
 const [dequeueMessage, setDequeueMessage] = useState<

 // Heap state
 const [heap, setHeap] = useState<Hypothesis[]>([

```

```

 setStack([
 { id: '1', name: 'Тарелка из-под пасты', icon: '🍝' },
 { id: '2', name: 'Тарелка из-под пиццы', icon: '🍕' },
 { id: '3', name: 'Блюдец от торта', icon: '🍰' },
]);
 setPopMessage(" Стопка тарелок восстановлена.");
 });

// Queue handlers
const addPerson = () => {
 const presets = [
 { name: 'Любитель борща', icon: '🥘' },
 { name: 'Курьер доставки', icon: '📦' },
 { name: 'Усталый админ', icon: '🤖' },
 { name: 'Турист с рюкзаком', icon: '🎒' },
];
 const randomPreset = presets[Math.floor(Math.random() * presets.length)];
 const newPerson = {
 id: Date.now().toString(),
 name: randomPreset.name,
 icon: randomPreset.icon
 };
 setQueue(prev => [...prev, newPerson]);
 setDequeueMessage(` Встал в конец очереди: ${newPerson.name} `);
};

const servePerson = () => {
 if (queue.length === 0) {
 setDequeueMessage("⚠ Очередь пуста! Суп ждет новых посетителей.");
 return;
 }
 const firstPerson = queue[0];
 setQueue(prev => prev.slice(1));
 setDequeueMessage(` Выдан суп первому в очереди (Фамилия: ${firstPerson.name}) `);
};

const resetQueue = () => {
 setQueue([
 { id: '1', name: 'Тарелка из-под пасты', icon: '🍝' },
 { id: '2', name: 'Тарелка из-под пиццы', icon: '🍕' },
 { id: '3', name: 'Блюдец от торта', icon: '🍰' },
]);
 setPopMessage(" Стопка тарелок восстановлена.");
};

```

```

const resetHeap = () => {
 setHeap([
 { id: '1', text: 'Кандидат: "Привет, я искусственный интеллект"' },
 { id: '2', text: 'Кандидат: "Привет, я испек вкусный хлеб"' },
 { id: '3', text: 'Кандидат: "Привет, я испанский язык"' }
]);
 setHeapMessage(" Гипотезы восстановлены.");
};

// Sorted heap for display (Max-Heap property view)
const sortedHeap = [...heap].sort((a, b) => b.probability - a.probability);

return (
 <div className="bg-slate-900/90 border border-slate-800 pb-6 mb-6">
 <div className="border-b border-slate-800 pb-6 mb-6">
 <h2 className="text-2xl font-bold text-transparent bg-clip-text bg-gradient-to-r from-blue-500 to-purple-500">
 Интерактивный симулятор структур данных
 </h2>
 <p className="text-slate-400 text-sm mt-2">
 Проверь работу принципов LIFO, FIFO и Приоритетной очереди
 </p>
 </div>

 { /* Tabs */ }
 <div className="grid grid-cols-1 md:grid-cols-3 gap-3">
 <button
 onClick={() => setActiveTab('stack')}
 className={`flex items-center justify-between gap-2 px-4 py-2 rounded-md
 ${activeTab === 'stack' ? 'bg-blue-600/20 border-blue-500 text-blue-400' : 'bg-slate-800/60 border-slate-700/60 text-slate-400'}
 `}
 >
 <div className="flex items-center gap-3">
 <Layers className={`w-5 h-5 ${activeTab === 'stack' ? 'text-blue-400' : 'text-slate-400'}>
 <div className="text-left">
 <div className="font-bold text-sm text-white">

```

```

 <div className="text-left">
 <div className="font-bold text-sm text-white">
 <div className="text-xs opacity-80">Приоритет
 </div>
 </div>

 Beam Search

 </button>
 </div>

```

```

 { /* STACK TAB */}
 {activeTab === 'stack' && {
 <div className="space-y-6">
 <div className="bg-slate-800/80 p-5 rounded-xl text-white">
 <div>
 <h3 className="text-lg font-bold text-white">
 Симуляция стопки тарелок

 1

 </h3>
 <p className="text-slate-400 text-xs mt-1">
 Новые тарелки ставятся наверх. Мыть можно
 </p>
 </div>
 <div className="flex flex-wrap items-center gap-2">
 <button
 onClick={addPlate}
 className="flex-1 md:flex-none flex items-center justify-center gap-2 rounded-xl text-sm font-bold shadow-lg transition"
 >
 <Plus className="w-4 h-4" /> Поставить
 </button>
 <button
 onClick={popPlate}
 className="flex-1 md:flex-none flex items-center justify-center gap-2 border-blue-500/40 px-4 py-2.5 rounded-xl transition"
 >

```

```

 {
 /* Reversed map so top of stack is visible */
 {stack.slice().reverse().map(({plate, index}) => {
 const isTop = index === 0;
 return (
 <div
 key={plate.id}
 className={`w-full flex items-center justify-center
 ${isTop
 ? 'bg-gradient-to-r from-blue-500 to-blue-100'
 : 'bg-slate-900/90 border-slate-500'}
 `}
 style={{width: 100, height: 100}}
 >
 <div className="flex items-center justify-center" style={{width: 100, height: 100}}>
 {plate.name}

 {isTop ? 'Готово к помывке' : 'В очередь'}

 </div>
 </div>
);
 })}
 </div>
 </div>
</div>
)}

<div className="mt-6 text-xs text-slate-500">
 ПОЛОЖИЛ ПОСЛЕДНЕЙ → ПОМЫЛ ПЕРВОЙ (LIFO) •
</div>
</div>
</div>
)}

{ /* QUEUE TAB */ }
{activeTab === 'queue' && {

```

```

{dequeueMessage && {
 <div className="bg-indigo-950/60 border border-
p-3 animate-fade-in">
 <span className="inline-block w-2 h-2 round
 {dequeueMessage}
 </div>
}}

{/* Visualization */}
<div className="bg-slate-950/60 p-6 rounded-2
overflow-x-auto">
 {queue.length === 0 ? {
 <div className="text-center py-12 text-sl
 <div className="text-5xl mb-3"> </div>
 <p className="text-base font-bold">Очер
 <p className="text-xs mt-1">Добавьте ро
 </div>
) : {
 <div className="w-full flex items-center
 <div className="flex flex-col items-cen
300">

 <span className="text-xs font-bold for
 </div>

 <div className="text-indigo-500 flex it
 <ArrowRight className="w-6 h-6 animat
 </div>

 {queue.map((person, index) => {
 const isFirst = index === 0;
 return {
 <div
 key={person.id}
 className={`flex flex-col items-c
 isFirst
 ? 'bg-gradient-to-b from-indi

```

```

{activeTab === 'heap' && {
 <div className="space-y-6">
 <div className="bg-slate-800/80 p-5 rounded-x
 tify-between gap-4">
 <div>
 <h3 className="text-lg font-bold text-whi
 Приоритетная очередь / Куча</sp
 <span className="text-xs font-mono bg-e
 x-Heap
 </h3>
 <p className="text-slate-400 text-xs mt-1
 Здесь не важно, кто пришел первым. Важн
 </p>
 </div>
 <div className="flex items-center gap-3 w-f
 <button
 onClick={popHeap}
 className="flex-1 md:flex-none flex ite
 py-2.5 rounded-xl text-sm font-bold shadow-
 >
 Взять кандидата с макс. вероятностью
 </button>
 <button
 onClick={resetHeap}
 title="Сбросить"
 className="p-2.5 bg-slate-800 hover:bg-
 tion-colors cursor-pointer"
 >
 <RotateCcw className="w-5 h-5" />
 </button>
 </div>
 </div>
 </div>

 {/* Add Form */}
 <form onSubmit={addHypothesis} className="bg-
 s-12 gap-4 items-end">
 <div className="md:col-span-6">
 <label className="block text-xs font-bold

```





```
 </div>
 </div>
 })
 </div>
);
};
```

---

ФАЙЛ 60/81: src/components/SimulatorHub.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

---

```
import React, { useMemo, useState } from "react";
import { BookOpen, Play, Search, X } from "lucide-react";
import { VIZ_REGISTRY } from "../vizRegistry";
import { chapters } from "../data/content";
```

```
interface Props {
 /** Открыть тренажёр в модальном окне. */
 onOpen: (vizId: string) => void;
 /** Перейти к теме, к которой привязан тренажёр. */
 onGoChapter: (chapterId: string) => void;
}
```

```
/**
 * Каталог всех интерактивных тренажёров.
 *
 * Раньше вкладка «Симулятор» показывала один случайный
 * теперь это витрина: видно всё, что вообще можно потыкать.
 */
```

```
export const SimulatorHub: React.FC<Props> = ({ onOpen,
 const [query, setQuery] = useState("");
```

```
 const items = useMemo(() => {
 const titleById = new Map(chapters.map((c) => [c.id, c.title]));
 const all = Object.entries(VIZ_REGISTRY).map(([id, viz],
 id,
```

```

 <X className="w-4 h-4" />
 </button>
 })
 </div>

 {items.length === 0 && <p className="text-slate-500">
 Нет данных
 </p>}

 <div className="grid gap-3 sm:grid-cols-2 xl:grid-cols-3">
 {items.map(({ id, entry, chapterTitle }) => (
 <div
 key={id}
 className="flex flex-col bg-slate-900 border border-slate-800"
 >
 <h3 className="font-bold text-white text-sm">{entry.title}</h3>
 {entry.hint && <p className="text-[12px] text-white">{entry.hint}</p>
 && <div className="flex-1">
 <div>
 <div className="flex items-center gap-2 mt-2">
 <button
 onClick={() => onOpen(id)}
 className="flex items-center gap-1.5 text-white transition-colors"
 >
 <Play className="w-3.5 h-3.5" /> Запустить
 </button>
 {chapterTitle && (
 <button
 onClick={() => onGoChapter(id)}
 title={chapterTitle}
 className="flex items-center gap-1.5 text-white transition-colors min-w-[150px]"
 >
 <BookOpen className="w-3.5 h-3.5 shrink-0" />
 {chapterTitle}
 </button>
)}
 </div>
 </div>
 </div>
)
)}
 </div>
</div>

```

```

 name: "Линия влево",
 hintCase: "Zig-Zig",
 target: 1,
 build: () => {
 const n1 = leaf(1);
 const n2: TNode = { key: 2, left: n1, right: leaf(3) };
 const n4: TNode = { key: 4, left: n2, right: leaf(5) };
 return { key: 6, left: n4, right: leaf(7) };
 },
},
{
 name: "Змейка",
 hintCase: "Zig-Zag",
 target: 3,
 build: () => {
 const n3 = leaf(3);
 const n2: TNode = { key: 2, left: leaf(1), right: n3 };
 const n4: TNode = { key: 4, left: n2, right: leaf(5) };
 return { key: 6, left: n4, right: leaf(7) };
 },
},
{
 name: "Один уровень",
 hintCase: "Zig",
 target: 2,
 build: () => ({ key: 4, left: { key: 2, left: leaf(1), right: leaf(3) }, right: leaf(5) },
},
{
 name: "Смешанный",
 hintCase: "Zig-Zag, затем Zig-Zig",
 target: 5,
 build: () => {
 const n5 = leaf(5);
 const n6: TNode = { key: 6, left: n5, right: leaf(7) };
 const n4: TNode = { key: 4, left: leaf(3), right: n6 };
 return { key: 8, left: n4, right: leaf(9) };
 },
},
},

```

```

/** Какой поворот сделал бы настоящий splay из текущего
export function canonicalNext(root: TNode, key: number)
 const path = findPath(root, key);
 if (path.length < 2) return null;
 const x = path[path.length - 1];
 const p = path[path.length - 2];
 const g = path.length >= 3 ? path[path.length - 3] : null;

 if (!g) return { node: x.key, kind: "Zig" };

 const xLeft = p.left === x;
 const pLeft = g.left === p;
 if (xLeft === pLeft) return { node: p.key, kind: "Zig" };
 return { node: x.key, kind: "Zig-Zag (сначала нижняя)" };
}

/* ----- раскладка -----

interface Placed {
 key: number;
 x: number;
 y: number;
 depth: number;
}

function layout(root: TNode | null): { nodes: Placed[];
 const nodes: Placed[] = [];
 const edges: [number, number][] = [];
 let order = 0;

 const walk = (n: TNode | null, depth: number) => {
 if (!n) return;
 walk(n.left, depth + 1);
 nodes.push({ key: n.key, x: order++, y: depth, depth: depth });
 if (n.left) edges.push([n.key, n.left.key]);
 if (n.right) edges.push([n.key, n.right.key]);
 walk(n.right, depth + 1);
 };

```

```

const { nodes, edges, width, height } = useMemo(() =>
const path = useMemo(() => new Set(findPath(tree, target)
const expected = useMemo(() => (solved ? null : canonicalNext(tree, target));
setHistory((h) => [...h, clone(tree) as TNode]);
setMoves((m) => [...m, { node: key, correct: want?.length > 0 }]);
setTree((t) => rotateUp(clone(t) as TNode, key));
});

const undo = () => {
 if (history.length === 0) return;
 setTree(history[history.length - 1]);
 setHistory((h) => h.slice(0, -1));
 setMoves((m) => m.slice(0, -1));
};

const wrong = moves.filter((m) => !m.correct).length;
const minimal = useMemo(() => findPath(preset.build(), target));

return (
 <div className="w-full bg-slate-950 rounded-2xl border border-slate-800">
 <div className="flex items-start justify-between p-4">
 <div>
 <h4 className="text-sm sm:text-base font-bold">
 <p className="text-[12px] text-slate-400 lead">
 Клик по узлу = один поворот с его родителем

 поворотов выбираешь сам, разбор покажем в к
 </p>
 </div>
 <button

```

```

 y1={na.y}
 x2={nb.x}
 y2={nb.y}
 stroke={onPath ? "#6366f1" : "#475569"}
 strokeWidth={onPath ? 3 : 1.8}
 style={{ transition: "all 700ms cubic-bezier(0.4, 0, 0.2, 1)" }}
 />
);
 }}}

{nodes.map((n) => {
 const isTarget = n.key === target;
 const clickable = !solved && n.depth > 0;
 return (
 <g
 key={n.key}
 onClick={() => clickable && clickNode(n)}
 style={{
 transform: `translate(${n.x}px, ${n.y}px)`,
 transition: "transform 700ms cubic-bezier(0.4, 0, 0.2, 1)",
 cursor: clickable ? "pointer" : "default",
 }}
 >
 {isTarget && (
 <circle r={22} fill="none" stroke="#6366f1" strokeWidth={2} />
)}
 <circle
 r={16}
 fill={isTarget ? "#6366f1" : path.has(n.key) ? "#a5b4fc" : "#475569"}
 stroke={isTarget ? "#a5b4fc" : "#475569"}
 strokeWidth={2}
 />
 <text textAnchor="middle" dominantBaseline="middle">
 {n.key}
 </text>
 </g>
);
}}}
```

```

 Сейчас splay крутил бы узел{" "}
 {expected}
 </div>
))

 {solved && (
 <div
 className={`mt-3 rounded-lg border px-3 py-2.5`}
 wrong === 0
 ? "bg-emerald-950/40 border-emerald-700/60"
 : "bg-rose-950/30 border-rose-800/60 text-rose-50"
 >
 <div className="flex items-center gap-2 font-mono">
 {wrong === 0 ? <CheckCircle2 className="w-4 h-4 text-emerald-500"/> : <X className="w-4 h-4 text-rose-500"/>}
 {wrong === 0
 ? `Узел ${target} в корне за ${moves.length} ходов`
 : `Узел ${target} в корне за ${moves.length} ходов, но не в том порядке`}
 </div>
 {wrong > 0 && (
 <ul className="list-disc pl-5 space-y-0.5">
 {moves.map(
 (m, i) =>
 !m.correct && (
 <li key={i}>
 ход {i + 1}: крутили {m.expected}

)
)
 }

)}
 </div>
))
 {wrong === 0 && moves.length > minimal && (
 <div className="opacity-80">Порядок верный,
 </div>
)}
</div>

```



```

 /** Кто переехал на этом кадре – рисуем призрак и строим
 moved?: NodeId[];
 ms: number;
}

interface Case {
 key: "zig" | "zig-zig" | "zig-zag";
 label: string;
 when: string;
 rotations: string;
 /** Номера узлов: роль → число. */
 keys: Partial<Record<NodeId, number>>;
 /** Обход слева направо – одинаковый на всех кадрах.
 inorder: string[];
 /** Что означают поддеревья A..D. */
 ranges: string;
 frames: Frame[];
}

const VB = { w: 360, h: 285 };
const AIM_MS = 2600;
const RESULT_MS = 3400;
const MOVE = "transform 1200ms cubic-bezier(.34,.01,.2,

/* ----- Zig -----
const ZIG_BEFORE = {
 pos: { p: [195, 50], x: [105, 130], C: [280, 130], A:
 edges: [["p", "x"], ["p", "C"], ["x", "A"], ["x", "B"]
} as unknown as Pick<Frame, "pos" | "edges">;

const ZIG: Case = {
 key: "zig",
 label: "Zig",
 when: "родитель x – уже корень, деда нет",
 rotations: "1 поворот",
 keys: { x: 20, p: 40 },
 inorder: ["A", "20", "B", "40", "C"],
 ranges: "A < 20 · B между 20 и 40 · C > 40",

```

---

```
} as unknown as Pick<Frame, "pos" | "edges">;
```

```
const ZIGZIG: Case = {
 key: "zig-zig",
 label: "Zig-Zig",
 when: "x и p – дети с ОДНОЙ стороны (оба слева или оба справа)",
 rotations: "2 поворота, первым – верхний",
 keys: { x: 20, p: 40, g: 60 },
 inorder: ["A", "20", "B", "40", "C", "60", "D"],
 ranges: "A < 20 · B: 20–40 · C: 40–60 · D > 60",
 frames: [
 {
 ...ZZ_S0,
 kind: "start",
 title: "Было: прямая линия 60 → 40 → 20",
 text: "Три узла выстроились в «бамбук»: каждый следующий узел
зглаживает.",
 ms: RESULT_MS,
 },
 {
 ...ZZ_S0,
 kind: "aim",
 title: "Прицел 1: ВЕРХНЕЕ ребро 60 – 40",
 text: "Главная хитрость этого случая: первым крутит
фокус: ["g", "p"],",
 ms: AIM_MS,
 },
 {
 ...ZZ_S1,
 kind: "result",
 title: "Поворот 1: 40 поднялось над 60",
 text: "40 встало в корень, 60 опустилось к нему по
убине 1, а не 2.",
 moved: ["p", "g", "C"],
 ms: RESULT_MS,
 },
 {
 ...ZZ_S1,
 kind: "result",
 title: "Поворот 2: 20 поднялось над 40",
 text: "20 встало в корень, 40 опустилось к нему по
убине 2, а не 1.",
 moved: ["x", "p", "C"],
 ms: RESULT_MS,
 },
],
}
```

```
frames: [
 {
 ...ZG_S0,
 kind: "start",
 title: "Было: 60 → влево на 20 → вправо на 40",
 text: "Узлы идут «змейкой»: сначала шаг влево, по",
 ms: RESULT_MS,
 },
 {
 ...ZG_S0,
 kind: "aim",
 title: "Прицел 1: НИЖНЕЕ ребро 20 – 40",
 text: "Дед (60) стоит на месте и в этом повороте",
 focus: ["p", "x"],
 ms: AIM_MS,
 },
 {
 ...ZG_S1,
 kind: "result",
 title: "Поворот 1: змейка выпрямилась",
 text: "40 заняло место 20 и забрало его к себе ле",
 moved: ["x", "p", "C"],
 ms: RESULT_MS,
 },
 {
 ...ZG_S1,
 kind: "aim",
 title: "Прицел 2: ребро 60 – 40",
 text: "Остался одиночный поворот вокруг деда – и",
 focus: ["g", "x"],
 ms: AIM_MS,
 },
 {
 pos: { x: [180, 50], p: [90, 130], g: [275, 130],
 edges: [["x", "p"], ["x", "g"], ["p", "A"], ["p",
 kind: "result",
 title: "Стало: 20 и 60 – два сына сорока",
```

```

 /* призраки: где узел стоял до поворота */
 {ghosts.map((id) => {
 const from = prev[id]!;
 const to = pos[id]!;
 const r = isSubtree(id) ? 15 : 20;
 const dx = to[0] - from[0];
 const dy = to[1] - from[1];
 const len = Math.hypot(dx, dy) || 1;
 const pad = r + 8;
 return (
 <g key={`ghost-${id}`} opacity={0.55}>
 <circle cx={from[0]} cy={from[1]} r={r} fill={
 {len > pad * 2 + 6 && (
 <line
 x1={from[0] + (dx / len) * pad}
 y1={from[1] + (dy / len) * pad}
 x2={to[0] - (dx / len) * pad}
 y2={to[1] - (dy / len) * pad}
 stroke="#a5b4fc"
 strokeWidth={1.5}
 strokeDasharray="4 3"
 markerEnd="url(#splay-arrow)"
 />
)}
 />
 </g>
);
 })}

 {edges.map(([a, b], i) => {
 const pa = pos[a];
 const pb = pos[b];
 if (!pa || !pb) return null;
 const hot = Boolean(focus && focus.includes(a));
 return (
 <line
 key={`-${a}-${b}-${i}`}
 x1={pa[0]}
 y1={pa[1]}

```

```

 <text textAnchor="middle" y={-r - 7} font{
 {id}
 }>
 </text>
)}
 </g>
);
}}
</svg>
);
};

```

```

export const SplayRotationsViz: React.FC = () => {
 const [caseIdx, setCaseIdx] = useState(0);
 const [frame, setFrame] = useState(0);
 const [playing, setPlaying] = useState(false);
 const [slow, setSlow] = useState(false);

 const active = CASES[caseIdx];
 const total = active.frames.length;
 const idx = Math.min(frame, total - 1);
 const current = active.frames[idx];
 const prevPos = active.frames[Math.max(idx - 1, 0)].p;
 const duration = useMemo(() => Math.round(current.ms - prevPos), [current]);
 const last = idx === total - 1;

 useEffect(() => {
 setFrame(0);
 }, [caseIdx]);

 useEffect(() => {
 if (!playing) return;
 const t = setTimeout(() => setFrame((f) => (f + 1) % total), duration);
 return () => clearTimeout(t);
 }, [playing, frame, duration, total]);

 return (
 <div className="w-full bg-slate-950 rounded-2xl border border-slate-800">
 <div className="flex gap-2 mb-4 overflow-x-auto">

```

```

<div className="bg-slate-900 rounded-xl border bo
 <Tree frame={current} prev={prevPos} keys={acti

 {/* обход слева направо – одинаковый на всех ка
 <div className="mt-2 pt-2 border-t border-slate
 <div className="flex items-center gap-1.5 fle
 06ход
 {active.inorder.map((t, i) => {
 <span
 key={i}
 className={`px-1.5 py-0.5 rounded font-m
 /\d/.test(t) ? "bg-emerald-500/15 tex
 }`}
 >
 {t}

)}}

 </div>
 <p className="text-[11px] text-slate-500 mt-1
 </div>
</div>

```

```

{/* индикатор автопрокрутки */}
<div className="h-1 w-full bg-slate-800 rounded-f
 <div
 key={` ${caseIdx}- ${frame}- ${playing}- ${slow}`}
 className="h-full bg-indigo-500"
 style={
 playing
 ? { animation: `demoProgress ${duration}m
 : { width: "100%", background: "#334155"
 }
 }
 </div>
</div>

```

```

<div className="flex flex-wrap items-center gap-2

```

```

 <Gauge className="w-3.5 h-3.5" /> {slow ? "0.5" : "1"}
 </button>
 <button
 onClick={() => {
 setFrame(0);
 setPlaying(false);
 }}
 className="flex items-center gap-1.5 px-3 py-3
 ext-xs font-bold transition-colors"
 >
 <RotateCcw className="w-3.5 h-3.5" /> Сброс
 </button>

 <div className="flex items-center gap-3 ml-auto"
 {[["x", "p", "g"] as const].map((id) =>
 active.keys[id] === undefined ? null : (
 <span key={id} className="flex items-center"
 <span className="w-3 h-3 rounded-full"
 style={{background-color: active[id]}} />
 {active[id]}

)
)}
 </div>
 </div>

 <p className="mt-2 text-[11px] text-slate-500">
 Листай кнопкой «Дальше» в своём темпе – кадры не
 </p>
 </div>
);
};

```

ФАЙЛ 63/81: src/components/SplayTreeViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```
import React, { useState } from "react";
```

```

 x.right = y.left;
 y.left = x;
 return y;
};

// Clone tree helper
const cloneTree = (node: SplayNode | null): SplayNode | null => {
 if (!node) return null;
 return {
 key: node.key,
 left: cloneTree(node.left),
 right: cloneTree(node.right),
 x: node.x,
 y: node.y,
 };
};

export const SplayTreeViz: React.FC = () => {
 // Let's create an elegant initial unbalanced or semi-balanced tree
 // Values: 10, 20, 30, 40, 50, 60
 const createInitialTree = (): SplayNode => {
 let r: SplayNode | null = null;
 const initialKeys = [40, 20, 50, 10, 30, 60];
 initialKeys.forEach((key) => {
 r = insertBST(r, key);
 });
 return r!;
 };

 const [tree, setTree] = useState<SplayNode>(createInitialTree());
 const [inputValue, setInputValue] = useState<string>('');
 const [log, setLog] = useState<string[]>([
 'Дерево инициализировано. Попробуйте кликнуть на узел.',
]);
 const [highlightedNode, setHighlightedNode] = useState<SplayNode | null>(null);

 // Splay operation that tracks steps!
 const splayStepByStep = (

```



```

 });
}

// Now let's implement standard Splay logic recursively
const splayAction = (
 node: SplayNode | null,
 k: number,
): SplayNode | null => {
 if (!node || node.key === k) return node;

 if (k < node.key) {
 if (!node.left) return node;

 if (k < node.left.key) {
 // Zig-Zig (Left-Left)
 node.left.left = splayAction(node.left.left,
 k);
 node = rightRotate(node);
 steps.push(
 `Вращение Zig-Zig (Правый поворот родителя)`
);
 } else if (k > node.left.key) {
 // Zig-Zag (Left-Right)
 node.left.right = splayAction(node.left.right,
 k);
 if (node.left.right) {
 node.left = leftRotate(node.left);
 steps.push(`Компонент Zig-Zag: левый поворот`);
 }
 }
 }

 if (!node.left) return node;
 steps.push(
 `Финальный поворот Zig (Правое вращение корня)`
);
 return rightRotate(node);
} else {
 if (!node.right) return node;

 if (k < node.right.key) {

```

```

 e.preventDefault();
 const val = parseInt(inputValue);
 if (isNaN(val)) return;

 // Standard BST insert, then splay!
 let newTree = cloneTree(tree);
 newTree = insertBST(newTree, val);
 const { newRoot, steps } = splayStepByStep(newTree,
 if (newRoot) {
 setTree(newRoot);
 }
 setLog([`Вставили вершину [${val}].`, ...steps]);
 setInputValue("");
 setHighlightedNode(val);
};

const handleFind = (e: React.FormEvent) => {
 e.preventDefault();
 const val = parseInt(inputValue);
 if (isNaN(val)) return;
 handleSplay(val);
 setInputValue("");
};

const resetTree = () => {
 setTree(createInitialTree());
 setHighlightedNode(null);
 setLog(["Дерево возвращено в исходное состояние."]);
};

// Assign coordinate positions using a simple recursive
const computeCoordinates = (
 node: SplayNode | null,
 x: number,
 y: number,
 levelWidth: number,
): void => {
 if (!node) return;

```

```

 y2={node.right.y}
 stroke="#475569"
 strokeWidth="2"
 />,
);
 lines = lines.concat(renderLines(node.right));
 }
 return lines;
};

```

```

const renderNodes = (node: SplayNode | null): React.ReactNode[] => {
 if (!node) return [];
 let circles: React.ReactNode[] = [];
 const isHighlighted = highlightedNode === node.key;

 circles.push(
 <g
 key={`n-${node.key}`}
 className="cursor-pointer group"
 onClick={() => handleSplay(node.key)}
 data-title={`Кликните, чтобы выполнить Splay(${node.key})`}
 >
 <circle
 cx={node.x}
 cy={node.y}
 r="18"
 fill={isHighlighted ? "#4f46e5" : "#1e293b"}
 stroke={isHighlighted ? "#818cf8" : "#64748b"}
 strokeWidth={isHighlighted ? "3" : "2"}
 className="transition-all duration-300 group-hover:stroke-4"
 />
 <text
 x={node.x}
 y={node.y}
 dy=".3em"
 textAnchor="middle"
 fill="white"
 fontSize="12px"
 />
 </g>
);
};

```

```

<div className="absolute bottom-4 left-4 right-4"
 <form onSubmit={handleFind} className="flex flex-col gap-4"
 <input
 type="number"
 placeholder="Поиск..."
 value={inputValue}
 onChange={(e) => setInputValue(e.target.value)}
 className="w-24 bg-slate-950 text-white"
 00 focus:outline-none"
 />
 <button
 type="submit"
 onClick={handleFind}
 className="bg-indigo-600 hover:bg-indigo-500 text-white"
 >
 Поиск / Splay
 </button>
 <button
 type="button"
 onClick={() => {
 e.preventDefault();
 const val = parseInt(inputValue);
 if (!isNaN(val)) {
 let newTree = cloneTree(tree);
 newTree = insertBST(newTree, val);
 const { newRoot, steps } = splayStep(newTree, val);
 if (newRoot) setTree(newRoot);
 setLog([`Вставили ${val}`], ...steps);
 setInputValue("");
 setHighlightedNode(val);
 }
 }}
 className="bg-indigo-950 hover:bg-slate-900 text-white"
 0"
 >
 ВСТАВИТЬ
 </button>
 </form>

```

```
 оба левые или оба правые. Вращаем деда, по
 </p>
 <p>

 колено. Вращаем узел в разные стороны.
 </p>
 </div>
</div>
</div>

{/* Answers Section inside the visualizer widget */}
<div className="bg-slate-950/80 p-5 border-t border-slate-800">
 <h4 className="text-indigo-400 font-bold text-sm">
 <HelpCircle className="w-4 h-4 text-indigo-400"/>
 Разбор экзаменационных вопросов (Без нитья):
 </h4>

 <div className="grid grid-cols-1 md:grid-cols-3 gap-4">
 <div className="bg-slate-900/60 p-4 rounded-lg">
 <div>
 <p className="font-bold text-slate-300 mb-2">
 1. Отсутствующий элемент
 </p>
 <p className="text-slate-400 leading-relaxed">
 Допустим, мы ищем{" "}
 <code className="text-indigo-400 font-sans">{" "}</code>
 его нет. Алгоритм спустится к листу, на котором
 неудачей (например, {" "})
 <code className="text-indigo-400">[20]</code>
 <code className="text-indigo-400">[30]</code>
 поднимет в корень{" "}
 <strong className="text-white">
 последний успешно просмотренный узел

 , на котором он споткнулся! Это оставляем
 сверху.
 </p>
 </div>
 </div>
 </div>
</div>
```

```

). Дорогая операция инвестирует в дешевые операции. Поэтому амортизированное время
 <strong className="text-white">O(log n).
 </p>
 </div>
 <div className="mt-3 text-[10px] text-emerald-500">
 Каждое "растяжение" компенсируется "сжатием".
 </div>
 </div>
</div>
);
};

```

ФАЙЛ 64/81: src/components/StackViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```

import React, { useState, useCallback } from 'react';
import { ArrowUp, Sparkles } from 'lucide-react';

```

```

interface Plate {
 id: string;
 name: string;
 emoji: string;
 color: string;
 isDirty: boolean;
 animState: 'in' | 'idle' | 'washing' | 'clean';
}

```

```

const PLATE_PRESETS = [
 { name: 'Тарелка из-под спагетти', emoji: '🍝', color: 'from-pink-500 to-pink-300' },
 { name: 'Тарелка из-под пиццы', emoji: '🍕', color: 'from-pink-500 to-pink-300' },
 { name: 'Блюдец от торта', emoji: '🍰', color: 'from-pink-500 to-pink-300' },
 { name: 'Тарелка от тако', emoji: '🌮', color: 'from-pink-500 to-pink-300' },

```

```

 id: Date.now().toString(),
 ...preset,
 isDirty: true,
 animState: 'in',
 });

 setDirtyStack(prev => [...prev, newPlate]);
 addLog(` PUSH: Положили ${newPlate.emoji} сверху с

 setTimeout(() => {
 setDirtyStack(prev => prev.map(p => p.id === newP
 }, 500);
}, [dirtyStack, isWashing]);

// POP: Помыть верхнюю тарелку (LIFO)
const popPlate = useCallback(() => {
 if (isWashing) return;
 if (dirtyStack.length === 0) {
 setShake(true);
 addLog('⚠ POP: Нечего мыть! Все тарелки чистые.')
 setTimeout(() => setShake(false), 400);
 return;
 }

 setIsWashing(true);
 const topPlate = dirtyStack[dirtyStack.length - 1];

 addLog(` POP: Начинаем мыть верхнюю тарелку с ${to

 // Step 1: Start washing animation (sponge and soap
 setDirtyStack(prev => prev.map(p => p.id === topPla
 setShowSoap(true);

 setTimeout(() => {
 // Step 2: Wash complete. Plate becomes clean and
 const cleanPlate: Plate = {
 ...topPlate,
 isDirty: false,

```

А когда приходит время мыть, ты берёшь именинник  
Попытаешься вытащить нижнюю – всё рухнет!

```
</p>
</div>
</div>

{/* УПРАВЛЕНИЕ */}
<div className="flex items-center gap-3 flex-wrap" >
 <button
 onClick={pushPlate}
 disabled={isWashing}
 className="flex-1 min-w-[150px] bg-gradient-to-r from-slate-800 to-slate-700 opacity-40 disabled:cursor-not-allowed text-white font-medium rounded-2xl px-5 py-3.5 transition-all cursor-pointer flex items-center justify-center" >
 Положить грязную (PUSH)
 </button>

 <button
 onClick={popPlate}
 disabled={isWashing || dirtyStack.length === 0}
 className="flex-1 min-w-[150px] bg-gradient-to-r from-slate-800 to-slate-700 opacity-40 disabled:cursor-not-allowed text-white font-medium rounded-2xl px-5 py-3.5 transition-all cursor-pointer flex items-center justify-center" >
 Помыть верхнюю (POP)
 </button>

 <button
 onClick={reset}
 className="px-5 py-3.5 rounded-2xl bg-slate-800 text-white font-medium cursor-pointer border border-slate-700" >
 Сброс
 </button>
</div>

{/* ИНТЕРАКТИВНАЯ КУХНЯ */}
```



```

 { /* Указатель TOP */ }
 <div
 className="absolute right-8 flex items-
 style={{ bottom: `${24 + topIndex * 44}`
 >
 <span className="text-xs font-mono text-
er gap-1">
 <ArrowUp className="w-3.5 h-3.5" />
 ВЕРШИНА (TOP)

 <span className="text-blue-400 animate-
 </div>

 { /* Тарелки (отрисовываем в обратном поряд
 {dirtyStack.slice().reverse().map((plate,
 const idx = dirtyStack.length - 1 - rev
 const isTop = idx === topIndex;
 return (
 <div
 key={plate.id}
 className={`
 w-full max-w-[280px] h-10 rounded
shadow-md select-none transition-all duration
 ${plate.color}
 ${plate.animState === 'in' ? 'ani
 ${plate.animState === 'washing' ?
 ${isTop ? 'ring-4 ring-blue-500/5
 `}
 >
 {plate.em
 <span className="text-slate-800 tex
 {isTop && (
 <span className="text-[9px] bg-bl
 LIFO

 </div>
 </div>
);

```

```

 }}}
 </div>
 })
 </div>

 { /* Столешница сушилки */ }
 <div className="w-full h-4 bg-gradient-to-r from-blue-500 to-purple-500" >
 </div>
</div>

{ /* ЛОГ ДЕЙСТВИЙ */ }
<div className="bg-slate-900 border border-slate-700 p-4" >
 <div className="text-[10px] font-mono text-slate-200" >
 {log.map((entry, idx) => (
 <div
 key={idx}
 className={`text-xs font-mono flex items-center justify-between p-2`} >
 {idx === 0 ? Действие :
 {entry}
 </div>
))}
 </div>
</div>
);
};

```

---

ФАЙЛ 65/81: src/components/StringAlgorithmsViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

---

```

import { useState, useEffect, useRef, useMemo } from "react";
import { Play, Pause, SkipForward, Undo, RefreshCw } from "lucide-react";

```

```

function generateKmpSteps(pattern: string, text: string) {
 if (!pattern) pattern = " ";
 const n = pattern.length;
 const m = text.length;
 const steps: Step[] = [];
 for (let i = 0; i < m; i++) {
 let j = 0;
 while (j < n && pattern[j] === text[i + j]) j++;
 steps.push({ i, j });
 }
}

```

```

function generateZSteps(pattern: string, text: string)
 if (!pattern) pattern = " ";
 const s = pattern + "#" + text;
 const n = s.length;
 const z = new Array(n).fill(0);
 const steps: any[] = [];

 let l = 0, r = 0;
 steps.push({ i: 0, l, r, z: [...z], desc: `Начало.` });

 for (let i = 1; i < n; i++) {
 steps.push({ i, l, r, z: [...z], desc: `Шаг i=$` });

 if (i <= r) {
 steps.push({ i, l, r, z: [...z], desc: `Внимание!` });
 z[i] = Math.min(r - i + 1, z[i - l]);
 steps.push({ i, l, r, z: [...z], desc: `Предел. Копипаста сработала.` });
 }

 steps.push({ i, l, r, zTarget: z[i], zSource: i });
 while (i + z[i] < n && s[z[i]] === s[i + z[i]])
 steps.push({ i, l, r, zTarget: z[i], zSource: i, desc: `Предел. Падают! Окно растёт.` });
 z[i]++;
 }

 if (i + z[i] < n) {
 steps.push({ i, l, r, zTarget: z[i], zSource: i, desc: `Предел. [i]]` разные. Стоп.` });
 } else {
 steps.push({ i, l, r, z: [...z], desc: `Упёрся.` });
 }

 steps.push({ i, l, r, z: [...z], desc: `Фиксируем.` });

 if (i + z[i] - 1 > r) {

```

```

 if (autoPlay) {
 timer.current = setInterval(() => {
 setStepIdx(prev => {
 if (prev >= steps.length - 1) { set
 return prev + 1;
 });
 }, 1200);
 }
 return () => { if (timer.current) clearInterval
 }, [autoPlay, steps.length]);

const playPause = () => setAutoPlay(!autoPlay);
const nextStep = () => { setAutoPlay(false); setSte
const prevStep = () => { setAutoPlay(false); setSte
const reset = () => { setAutoPlay(false); setStepId

const step = steps[stepIdx] || steps[0];

return (
 <div className="space-y-4">
 <div className="flex items-center justify-b
 <div className="flex bg-slate-900 borde
 <button onClick={() => setMode("kmp
 className={`px-3 py-1.5 text-xs
e" : "text-slate-400 hover:text-slate-300"}
 КМП (π-ФУНКЦИЯ)
 </button>
 <button onClick={() => setMode("z")
 className={`px-3 py-1.5 text-xs
" : "text-slate-400 hover:text-slate-300"}`
 Z-ФУНКЦИЯ
 </button>
 </div>

 <div className="flex items-center gap-2
 <div className="bg-slate-800 p-1 fl
 <span className="text-[10px] te
 <input value={pattern} onChange=

```

```

 bgColor = step.match ===
-900/40");
 }
}

return {
 <div key={index} className=
 {/* L/R markers for Z */
 {mode === "z" && step.l
 <div className="abs
 }}
 {mode === "z" && step.r
 <div className="abs
 }}

 {/* Index */}
 <span className={`text-

 {/* Char Box */}
 <div className={`w-8 h-
xtColor} font-mono text-lg transition-color
 {char}
 </div>

 {/* Value array box */}
 <div className="text-[1
slate-700/50 font-mono font-bold">
 {mode === "kmp" ? s
 </div>

 {/* KMP fingers */}
 {mode === "kmp" && step
 <div className="abs
z-20">
 <span classNam
 </div>
 }}
 {mode === "kmp" && step

```

```

 </div>
)
 }}}
</div>
</div>

<div className="flex flex-col sm:flex-row gap-4"
 {/* Controls */}
 <div className="flex bg-slate-900 border border-slate-800 rounded-lg disabled:opacity-30 disabled:hover:opacity-50"
 <button onClick={prevStep} disabled={steps.length === 0}
 <Undo strokeWidth={3} size={16} />
 </button>
 <button onClick={playPause} className="flex-grow-1 items-center justify-center min-w-[80px]"
 {autoplay ? <><Pause size={16} /> : <><Play size={16} />}
 </button>
 <button onClick={nextStep} disabled={steps.length === steps.length}
 <SkipForward strokeWidth={3} size={16} />
 </button>
 <button onClick={reset} className="border border-slate-800"
 <RefreshCw strokeWidth={3} size={16} />
 </button>
 </div>

 {/* Description Log */}
 <div className="flex-1 bg-slate-900/50 overflow-hidden"
 <div className="absolute top-0 right-0 bottom-0 left-0"
 <span className="font-mono text-sm text-slate-400"
 {steps.length} / {steps.length}

 </div>
 <div className="text-[10px] text-slate-400"
 {steps.length}
 </div>
 <div className="text-sm text-slate-400"
 {step.desc}
 </div>
 </div>

```

```
 </pre>
 </div>
 </div>
);
}
```

---

ФАЙЛ 66/81: src/components/TopologicalSortViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

---

```
import React, { useState, useEffect } from "react";
import {
 Play,
 Pause,
 RotateCcw,
 Clock,
} from "lucide-react";
```

```
interface TNode {
 id: number;
 x: number;
 y: number;
 label: string;
 name: string;
}
```

```
interface TEdge {
 u: number;
 v: number;
}
```

```
const dagNodes: TNode[] = [
 { id: 0, x: 100, y: 150, label: "0", name: "Матан 1 (Математика)" },
 { id: 1, x: 280, y: 70, label: "1", name: "Дискретка (Дискретная математика)" },
 { id: 2, x: 280, y: 230, label: "2", name: "Линейный (Линейная алгебра)" },
 { id: 3, x: 460, y: 150, label: "3", name: "Алгоритмы (Алгоритмы)" },
];
```

```

 topoList: [...topoList],
 dfsStack: [...dfsStack],
 });
};

recordStep(
 "Начало топологической сортировки. Используем кла
 null,
);

// Adj list
const adj: number[][] = Array.from({ length: 5 }, () => []);
dagEdges.forEach((e) => adj[e.u].push(e.v));

const dfs = (u: number) => {
 visited.add(u);
 dfsStack.push(u);
 recordStep(
 `DFS: зашли в вершину ${u} (${dagNodes[u].name
 u,
);

 for (const to of adj[u]) {
 if (!visited.has(to)) {
 dfs(to);
 } else if (!fullyProcessed.has(to)) {
 // Explaining potential cycle detected (not in
 recordStep(
 `⚠ Внимание: Рёбра в серую вершину ${to} о
 u,
);
 }
 }
}

fullyProcessed.add(u);
dfsStack.pop();
topoList.push(u); // prepend behavior: we write in
recordStep(
```



```

 setCurrentStepIdx((prev) => prev + 1);
 }, 1600);
 } else {
 setIsPlaying(false);
 }
 return () => clearInterval(timer);
}, [isPlaying, currentStepIdx, steps.length]);

const togglePlay = () => setIsPlaying(!isPlaying);
const reset = () => {
 setIsPlaying(false);
 setCurrentStepIdx(0);
};

const getTopoListString = () => {
 return [...step.topoList]
 .reverse()
 .map((id) => dagNodes[id].name)
 .join(" → ");
};

return (
 <div className="bg-slate-900 rounded-xl border border-
 <div className="bg-slate-800 p-4 border-b border-
 <h3 className="font-bold text-white flex items-
 <Clock className="w-5 h-5 text-indigo-400" />
 Топологическая сортировка (Зависимости обучен
 </h3>

 <div className="flex items-center gap-2 bg-slate-
 <button
 onClick={togglePlay}
 className="flex items-center gap-2 px-3 py-
 500 text-white"
 >
 {isPlaying ? (
 <Pause className="w-4 h-4 ml-1" />
) : (

```

```

 orient="auto-start-reverse"
 >
 <path d="M 0 1 L 10 5 L 0 9 z" fill="#8
 </marker>
</defs>

{/* Edges */}
{dagEdges.map((edge, idx) => {
 const u = dagNodes.find((n) => n.id === e
 const v = dagNodes.find((n) => n.id === e

 const isActive =
 (step.currentNode === edge.u || step.cu
 step.visited.has(edge.v);

 return (
 <line
 key={`edge-${idx}`}
 x1={u.x}
 y1={u.y}
 x2={v.x}
 y2={v.y}
 stroke={isActive ? "#818cf8" : "#334155"}
 strokeWidth={isActive ? "3" : "2"}
 markerEnd={`url(#${isActive ? "dag-ar
 className="transition-all duration-300
 />
);
}})

{/* Nodes */}
{dagNodes.map((node) => {
 const isCurrent = step.currentNode === no
 const isVisited = step.visited.has(node.i
 const isProcessed = step.fullyProcessed.h

 let fill = "#1e293b";
 let stroke = "#334155";

```

```

 >
 ({node.label}) {node.name.split(" ")
 </text>
 <text
 x={node.x}
 y={node.y + 12}
 textAnchor="middle"
 fill="#94a3b8"
 fontSize="9px"
 className="pointer-events-none"
 >
 {node.name.split(" ").slice(1).join(" ")}
 </text>
 </g>
);
 }}}
</svg>
</div>

{/* Console column */}
<div className="lg:col-span-4 bg-slate-950 p-5 text-slate-100 overflow-y-auto">
 <h4 className="text-xs font-bold text-slate-400">
 Статус сортировки
 </h4>

 <div className="bg-slate-900 border border-slate-800 p-4">
 <p className="text-xs text-indigo-300 min-h-[100px]">
 {step.desc}
 </p>
 </div>

 <div className="flex-1 space-y-4 overflow-y-auto">
 {/* Topological List output */}
 <div>

 РЕЗУЛЬТАТ (МАТРИЦА ОБУЧЕНИЯ):

 </div>
 </div>

```

```

 disabled={currentStepIdx === 0}
 onClick={() => setCurrentStepIdx((prev)
 className="bg-slate-900 border border-s
t-white"
 >
 Назад
 </button>
 <button
 disabled={currentStepIdx >= steps.length
 onClick={() => setCurrentStepIdx((prev)
 className="bg-slate-900 border border-s
t-white"
 >
 Вперед
 </button>
 </div>
 </div>
</div>
</div>
</div>
);
};

```

ФАЙЛ 67/81: src/components/vizRegistry.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

```

import React from "react";

import { ArticulationPointsViz } from "./ArticulationPo
import BellmanFordViz from "./BellmanFordViz";
import BfsViz from "./BfsViz";
import { DPVisualizer } from "./DPVisualizer";
import { DfsBridgesSimulator } from "./DfsBridgesSimula
import DijkstraViz from "./DijkstraViz";
import { EulerSimulator } from "./EulerSimulator";

```

```


 </p>
 <SplayRotationSandbox />
 </div>
);

export interface VizEntry {
 /** Заголовок панели/модалки. */
 title: string;
 /** Короткое пояснение, что здесь можно потыкать. */
 hint?: string;
 Component: React.FC;
}

/**
 * Единый реестр интерактивных демонстраций.
 * Используется и для панели под главой, и для всплывающ
 * и для модального окна «Открыть симулятор».
 */
export const VIZ_REGISTRY: Record<string, VizEntry> = {
 "stack-dfs": {
 title: "Стек и обход в глубину",
 hint: "Кладите и снимайте тарелки — это ровно то, ч
 Component: StackViz,
 },
 "queue-bfs": {
 title: "Очередь и обход в ширину",
 hint: "Очередь слева, живой BFS по графу справа.",
 Component: () => (
 <div className="space-y-10">
 <QueueViz />
 <BfsViz />
 </div>
),
 },
 "heap-beam-search": {
 title: "Куча (priority queue)",
 hint: "Добавляйте гипотезы — куча сама держит лучшую

```

```

"top-sort": { title: "Топологическая сортировка", Component: TopSortViz },
"scc-kosaraju": { title: "Компоненты сильной связности", Component: SCCViz },
"graph-articulation": { title: "Точки сочленения", Component: GraphArticulationViz },
"bridges-code": { title: "Мосты: DFS + tin/low", Component: BridgesCodeViz },
"euler-path-vs-cycle": { title: "Эйлеров путь и цикл", Component: EulerPathVsCycleViz },
"planarity-euler-formula": { title: "Планарность: K5 и K3,3", Component: PlanarityViz },
dijkstra: {
 title: "Дейкстра",
 hint: "Жадно забираем ближайшую вершину и релаксируем",
 Component: () => (
 <>
 <ChapterImage vizType="dijkstra" />
 <DijkstraViz />
 </>
),
},
"bellman-ford": {
 title: "Форд-Беллман",
 Component: () => (
 <>
 <ChapterImage vizType="bellman-ford" />
 <BellmanFordViz />
 </>
),
},
floyd: {
 title: "Флойд-Уоршелл",
 Component: () => (
 <>
 <ChapterImage vizType="floyd" />
 <FloydViz />
 </>
),
},
"johnson-algo": { title: "Алгоритм Джонсона", Component: JohnsonAlgoViz },
"mst-kruskal": { title: "Краскал и DSU", Component: KruskalViz },
"mst-prim": { title: "Прим", Component: KruskalSimulViz },
"mst-boruvka": { title: "Борувка", Component: KruskalSimulViz },

```

```

 y: number;
 depth: number;
 fail: number;
 term_link: number;
 is_terminal: boolean;
 words: string[];
}

const WATERFALL_NODES: { [key: number]: WNode } = {
 0: { id: 0, label: 'ROOT', char: 'ø', x: 400, y: 60, depth: 0,
 // Ветка H -> E -> R -> S
 1: { id: 1, label: 'H', char: 'H', x: 280, y: 160, depth: 1,
 2: { id: 2, label: 'HE', char: 'E', x: 200, y: 260, depth: 2,
 3: { id: 3, label: 'HER', char: 'R', x: 150, y: 360, depth: 3,
 4: { id: 4, label: 'HERS', char: 'S', x: 100, y: 460, depth: 4,
 // Ветка H -> I -> S
 5: { id: 5, label: 'HI', char: 'I', x: 350, y: 260, depth: 2,
 6: { id: 6, label: 'HIS', char: 'S', x: 320, y: 360, depth: 3,
 // Ветка S -> H -> E
 7: { id: 7, label: 'S', char: 'S', x: 550, y: 160, depth: 1,
 8: { id: 8, label: 'SH', char: 'H', x: 580, y: 260, depth: 2,
 9: { id: 9, label: 'SHE', char: 'E', x: 610, y: 360, depth: 3,
 = 2 (HE)
 }
 }
 }
 }
 }
 }
 }
 }
 }
 };

// Нормальные переходы бора
const TRIE_TRANSITIONS: { [key: number]: { [char: string]: number } } = {
 0: { H: 1, S: 7 },
 1: { E: 2, I: 5 },
 2: { R: 3 },
 3: { S: 4 },
 4: {},
 5: { S: 6 },
 6: {},
 7: { H: 8 },
 8: { E: 9 },
 9: {},
};

```

```

 if (currentIndex >= textToScan.length) {
 setIsSearchDone(true);
 setSearchLog('Мы проплыли весь текст! Лосось доволен!');
 setActiveSearchCodeLine(4);
 return;
 }

 const char = textToScan[currentIndex];
 let curr = currentNode;
 let logMsg = `[Символ '${char}']`;

 let jumpType: 'normal' | 'fail' | null = null;
 void jumpType;
 let originalCurr = curr;

 if (TRIE_TRANSITIONS[curr][char] !== undefined) {
 const nextNode = TRIE_TRANSITIONS[curr][char];
 logMsg += `У пусла #${curr} (${WATERFALL_NODES[curr].label}).`;
 setJumpPath({ from: curr, to: nextNode, type: 'normal' });
 curr = nextNode;
 setActiveSearchCodeLine(3);
 } else {
 setActiveSearchCodeLine(2);
 let jumps = 0;
 while (curr !== 0 && TRIE_TRANSITIONS[curr][char] === undefined) {
 curr = WATERFALL_NODES[curr].fail;
 jumps++;
 }
 const nextNode = TRIE_TRANSITIONS[curr][char] ?? 0;
 if (originalCurr === 0) {
 logMsg += `В корне нет перехода по '${char}'. Лосось недоволен!`;
 setJumpPath(null);
 } else {
 logMsg += `Поток по '${char}' у вершины #${originalCurr} не достиг`;
 logMsg += `рыжок по fail-ссылке в #${curr} (${WATERFALL_NODES[curr].label}).`;
 setJumpPath({ from: originalCurr, to: nextNode, type: 'fail' });
 }
 }

```



```

 codeLine: 1,
 scoutPos: 0,
 inspectedParentFail: 0,
 checkTargetChar: 'Ø',
 checkingBranchesFrom: null,
 },
 {
 targetNodeId: 1,
 title: 'Шаг 1 (Depth 1): Вершина #1 (H)',
 desc: 'Мы обрабатываем первую вершину из очереди - просто не существует. fail[H] автоматически нап
 codeLine: 5,
 scoutPos: 0,
 inspectedParentFail: 0,
 checkTargetChar: 'H',
 checkingBranchesFrom: null,
 },
 {
 targetNodeId: 7,
 title: 'Шаг 2 (Depth 1): Вершина #7 (S)',
 desc: 'Обрабатываем дочернюю вершину #7 (S) на De
 ,
 codeLine: 5,
 scoutPos: 0,
 inspectedParentFail: 0,
 checkTargetChar: 'S',
 checkingBranchesFrom: null,
 },
 {
 targetNodeId: 2,
 title: 'Шаг 3 (Depth 2): Вершина #2 (HE)',
 desc: 'Переходим на Depth 2 к вершине #2 (HE). Ро
 а-разведчик плывет в ROOT и ищет ветку по "E".
 codeLine: 3,
 scoutPos: 0,
 inspectedParentFail: 0,
 checkTargetChar: 'E',
 checkingBranchesFrom: 0,
 },

```

```

desc: 'Обрабатываем #6 (HIS) на Depth 3. Родитель
но ветка "S" → #7 (S) совпадает! fail[HIS] = #7
codeLine: 4,
scoutPos: 0,
inspectedParentFail: 0,
checkTargetChar: 'S',
checkingBranchesFrom: 0,
},
{
targetNodeId: 9,
title: 'Шаг 8 (Depth 3): Вершина #9 (SHE) — КАК С
desc: ' МАГИЯ АХО-КОРАСИК! Обрабатываем #9 (SHE)
ба плывет в #1 (H) и ищет ветку "E". У вершины :
codeLine: 4,
scoutPos: 1,
inspectedParentFail: 1,
checkTargetChar: 'E',
checkingBranchesFrom: 1,
},
{
targetNodeId: 4,
title: 'Шаг 9 (Depth 4): Вершина #4 (HERS)',
desc: 'Финальная вершина #4 (HERS) на Depth 4. Ро
#7 (S). fail[HERS] = #7 (S). Автомат полностью
codeLine: 4,
scoutPos: 0,
inspectedParentFail: 0,
checkTargetChar: 'S',
checkingBranchesFrom: 0,
},
];

```

```

const currentTrainInfo = TRAINING_STEPS_INFO[trainStep];
const activeFishPosNode = activeMode === 'training' ?
const targetTrainingNode = WATERFALL_NODES[currentTrainInfo.targetNode];

```

```

return (
<div className="bg-slate-800/95 rounded-2xl p-6 border

```

```

 </div>
 </div>

 { /* ПАНЕЛЬ УПРАВЛЕНИЯ В ЗАВИСИМОСТИ ОТ РЕЖИМА */
 {activeMode === 'training' ? {
 /* ПАНЕЛЬ РЕЖИМА ОБУЧЕНИЯ */
 <div className="grid grid-cols-1 lg:grid-cols-3" >
 { /* Управление шагами обучения */
 <div className="bg-slate-900/90 p-5 rounded-x" >
 <div>
 <h5 className="text-indigo-300 font-bold" >
 Полный обход в ширину (В
 </h5>
 <p className="text-xs text-slate-300 mb-3" >
 Показаны все шаги очереди BFS (начиная с
 ар 8 (SHE → HE), где наглядно видно
 </p>
 <div className="space-y-1.5 mb-6 overflow" >
 {TRAINING_STEPS_INFO.map(({step, idx} =>
 <button
 key={idx}
 onClick={() => setTrainStep(idx)}
 className={`w-full text-left px-3 p-
 ${
 idx === trainStep
 ? 'bg-indigo-600 text-white font
 : 'bg-slate-800 text-slate-400 l
 }` }
 >
 {step
 {idx === trainStep && <span classNa
 </button>
)}}
 </div>
 </div>

 <div className="flex gap-2">
 <button

```

```

 -l-4 border-indigo-400 text-indigo-200 font-
 2. let v = fail[r]; // Подъем к
ainInfo.inspectedParentFail].label}}
 {currentTrainInfo.codeLine === 2 && <
к родителю}
 </div>
 <div className={`p-1.5 rounded flex item
l-4 border-amber-500 text-amber-300 font-bo
 3. while (v !== root && trie[v]
 {currentTrainInfo.codeLine === 3 && <
т, возврат в корень}
 </div>
 <div className={`p-1.5 rounded flex item
r-l-4 border-emerald-500 text-emerald-300 f
 4. if (trie[v][char] !== undefin
/span>
 {currentTrainInfo.codeLine === 4 && <
ан IF!}
 </div>
 <div className={`p-1.5 rounded flex item
-4 border-blue-500 text-blue-300 font-bold'
 5. else fail[u] = root; // Bero
 {currentTrainInfo.codeLine === 5 && <
root}
 </div>
 </div>
</div>

<div>
 <h5 className="text-slate-200 font-bold ml
 <Lightbulb className="w-4 h-4 text-ambe
 </h5>
 <div className="bg-slate-800 p-4 rounded-
w-inner min-h-[72px] flex items-center">
 {currentTrainInfo.desc}
 </div>
</div>
</div>

```



```

o-900 to-transparent pointer-events-none"></div>
<div className="absolute top-0 left-1/2 -transl
 <div className="w-8 bg-blue-400 h-full animat
 <div className="w-12 bg-blue-300 h-full anima
 <div className="w-6 bg-blue-500 h-full animat
</div>

<div className="flex items-center justify-betwe
 <h5 className="text-white font-bold text-base
 Ветвящийся синий водопад (Бо
 </h5>
 <div className="flex flex-wrap gap-4 text-xs"
 <span className="flex items-center gap-1 te
 <span className="w-3 h-0.5 bg-blue-500 in

 <span className="flex items-center gap-1 te
 <span className="w-3 h-0.5 border-t borde

 </div>
</div>

{/* Само SVG дерево водопада */}
<div className="w-full overflow-x-auto custom-s
 <svg viewBox="0 0 800 520" className="w-full

 {/* Горизонтальные линии и подписи уровней */}
 {[
 { depth: 0, y: 60, label: 'Depth 0 (Корень'
 { depth: 1, y: 160, label: 'Depth 1 (H, S
 { depth: 2, y: 260, label: 'Depth 2 (HE, I
 { depth: 3, y: 360, label: 'Depth 3 (HER,
 { depth: 4, y: 460, label: 'Depth 4 (HERS
]}.map((level) => {
 <g key={`depth-line-${level.depth}`}>
 <line
 x1="20"
 y1={level.y}
 x2="780"

```

```

 className={isHighlighted || isTrainingExamined ? (isMatchBranch ? '#10b981' : '#f43f5a') : ''}
 />
 { /* Символ перехода */ }
 <circle cx={{(fromNode.x + toNode.x) / 2}} cy={{(fromNode.y + toNode.y) / 2}}
 r={10} fill={isTrainingExamined ? (isMatchBranch ? '#10b981' : '#f43f5a') : 'white'}
 stroke={isMatchBranch ? '#10b981' : '#f43f5a'} strokeWidth={2} />
 <text
 x={{(fromNode.x + toNode.x) / 2}}
 y={{(fromNode.y + toNode.y) / 2}}
 fill={isTrainingExamined ? (isMatchBranch ? '#10b981' : '#f43f5a') : 'white'}
 fontSize="12"
 fontWeight="bold"
 textAnchor="middle"
 >
 {char}
 </text>

 { /* ВИЗУАЛЬНЫЕ МЕТКИ ПРОВЕРКИ IF В РЕЖИМЕ ОБУЧЕНИЯ */ }
 {isTrainingExamined && (
 <g transform={`translate(${(fromNode.x + toNode.x) / 2}, ${
 (fromNode.y + toNode.y) / 2})`} >
 <rect x="-15" y="-12" width="30" height="24"
 fill="white" stroke={isMatchBranch ? '#10b981' : '#f43f5a'}
 strokeWidth="2" />
 <text x="0" y="5" fontSize="14"
 fill="white" stroke={isMatchBranch ? '#10b981' : '#f43f5a'}
 strokeWidth="2" />
 {isMatchBranch ? ' ' : 'X'}
 </text>
 </g>
)}
 </g>
 </g>
)
);
});
}}}

/* Рендер fail-ссылок (пунктирные водопады) */
Object.values(WATERFALL_NODES).map((node) => {
 if (node.id === 0) return null;
 const targetNode = WATERFALL_NODES[node.failId];

 // В режиме обучения показываем fail-ссылку

```

```
const isCurrent = activeFishPosNode.id ===
const isTargetChild = activeMode === 'tra
const hasWords = node.words.length > 0;
```

```
return (
 <g key={`node-${node.id}`} className="c
 {/* Внешнее свечение для активной вер
 {isCurrent && (
 <circle cx={node.x} cy={node.y} r="
)}
 {isTargetChild && (
 <circle cx={node.x} cy={node.y} r="
)}

 {/* Основной круг вершины */}
 <circle
 cx={node.x}
 cy={node.y}
 r={isCurrent || isTargetChild ? '24
 fill={isCurrent ? '#2563eb' : isTar
 stroke={isCurrent ? '#ffffff' : isTa
 strokeWidth={isCurrent || isTargetC
 className="transition-all duration-
 />

 {/* Текст внутри вершины */}
 <text
 x={node.x}
 y={node.y + 5}
 fill={isCurrent || isTargetChild ?
 fontSize={isCurrent || isTargetChil
 fontWeight="bold"
 textAnchor="middle"
 >
 {node.label}
 </text>

 {/* Метка искомой вершины в режиме об
```



```

 y={activeFishPosNode.y - 19}
 fontSize="18"
 textAnchor="middle"
 style={{ transition: 'x 0.8s cubic-bezier(0.25, 0.8, 0.25, 0.75)' }}
 className="animate-float select-none"
 >
 {activeMode === 'training' ? ' ♂ ' : ' ♀ '}
 </text>
 </g>

 </svg>
</div>
</div>

{/* Список найденных совпадений (только в режиме поиска) */}
{activeMode === 'search' && (
 <div className="mt-6 bg-slate-900/50 p-5 rounded" style={{ width: '100%' }}>
 <h5 className="text-white font-bold mb-3 text-sm">
 ⚙ Улов лосося (Найденные слова)
 </h5>
 {foundMatches.length === 0 ? (
 <p className="text-sm text-slate-400 italic">
 Пока ничего не поймали. Запустите шаги симуляции.
 </p>
) : (
 <div className="flex flex-wrap gap-2">
 {foundMatches.map((match, idx) => (
 <div
 key={idx}
 className="bg-emerald-950 border border-emerald-500 text-white text-sm font-bold shadow animate-[scaleUp_0.3s_ease-out]"
 style={{ width: '100%', padding: '5px' }}
 >
 <CheckCircle2 className="w-4 h-4 text-emerald-400" />
 {match.word}

 Индекс {match.index}

 </div>
))}
 </div>
)}
 </div>
)
```

```

];

const cellBase =
 "flex items-center justify-center rounded-md font-monospace" +
 "w-[clamp(30px,9vw,46px)] h-[clamp(30px,9vw,46px)] text-center";

export const PrefixSumViz: React.FC = () => {
 const [mode, setMode] = useState<"1d" | "2d">("1d");
 return (
 <div className="w-full bg-slate-950 rounded-2xl border" >
 <div className="flex gap-2 mb-4 overflow-x-auto" >
 {(
 [
 ["1d", "1. Одномерные суммы"],
 ["2d", "2. Двумерные суммы"],
] as const
).map(([key, label]) => (
 <button
 key={key}
 onClick={() => setMode(key)}
 className={`px-3 py-2 rounded-lg text-xs sm:text-sm
 ${mode === key ? "bg-indigo-600 text-white" : ""}
 `}
 >
 {label}
 </button>
))}
 </div>
 {mode === "1d" ? <Prefix1D /> : <Prefix2D />}
 </div>
);
};

/* ----- 1D -----
const Prefix1D: React.FC = () => {
 const pref = useMemo(
 () => A1.reduce<number[]>((acc, v) => [...acc, acc[acc.length - 1] + v], [])
);

```

```

 onMouseLeave={() => setHover(null)}
 onClick={() => setRange((r) => (i < r
 className={`${cellBase} cursor-pointer
 inCover
 ? "bg-indigo-500 text-white ring-1
 : inRange
 ? "bg-indigo-900/70 text-indigo-500
 : "bg-slate-800 text-slate-300
 }`}
 title={`a[${i}] = ${v}`}
 >
 {v}
 </div>
);
 }}}
</div>

{/* массив P */}
<div className="flex gap-1.5 items-center">
 <div className="w-[46px] shrink-0 text-right">
 {pref.map((v, i) => {
 const active = hover?.kind === "pref" && i === hover.i;
 const isL = i === range.l;
 const isR = i === range.r + 1;
 return (
 <div
 key={i}
 onMouseEnter={() => setHover({ kind: "pref", i: i })}
 onMouseLeave={() => setHover(null)}
 className={`${cellBase} cursor-help ${
 active
 ? "bg-emerald-500 text-white ring-1"
 : isR
 ? "bg-emerald-700 text-white"
 : isL
 ? "bg-rose-700 text-white"
 : "bg-slate-900 text-slate-300"
 }`}
 >
 {v}
 </div>
);
 })}
 </div>
</div>

```

```

) : (
 <p className="text-slate-500">
 Наведите курсор на любое число: у <b classN
 у <b className="text-indigo-400">a – ку
 </p>
)}
 </div>

 { /* Запрос */
 <div className="bg-slate-900 border border-indigo
 <div className="text-xs text-slate-400 mb-2">
 Запрос суммы на отрезке (кликайте по массиву
 </div>
 <p className="font-mono text-sm sm:text-base te
 sum(a[{range.l}..{range.r}]) = P[{range.r + 1
 {pref[range
 {pref[range.l
 {
 </p>
 <p className="text-[11px] text-slate-500 mt-1">
 Один вычитание вместо цикла: запрос за $O(1)$ п
 </p>
 </div>
 </div>
);
};

```

```

/* ----- 2D -----
const Prefix2D: React.FC = () => {
 const n = A2.length;
 const m = A2[0].length;

 const P = useMemo(() => {
 const p = Array.from({ length: n + 1 }, () => Array
 for (let i = 1; i <= n; i++) {
 for (let j = 1; j <= m; j++) {
 p[i][j] = A2[i - 1][j - 1] + p[i - 1][j] + p[i]
 }
 }
 });

```

```

)
 }
 className={` ${cellBase} cursor-po
 inRect ? "bg-indigo-500 text-wh
 }` }
 >
 {v}
 </div>
);
 }}}
</div>
)))
</div>
</div>

```

```

{/* Матрица префиксных сумм */}
<div className="overflow-x-auto">
 <div className="text-xs text-slate-400 mb-2">
 <div className="inline-block">
 {P.map((row, i) => {
 <div key={i} className="flex gap-1.5 mb-1">
 {row.map((v, j) => {
 const corner = cornerOf(i, j);
 const isHover = hover?.i === i && hov
 const cornerColor =
 corner === "A"
 ? "bg-emerald-600 text-white"
 : corner === "D"
 ? "bg-sky-600 text-white"
 : corner
 ? "bg-rose-600 text-white"
 : "bg-slate-900 border border
 return {
 <div
 key={j}
 onMouseEnter={() => setHover({ i,
 onMouseLeave={() => setHover(null
 className={` ${cellBase} cursor-he

```

```
 <p className="text-[11px] text-slate-500 mt-1">
 Вычли два «хвоста» и вернули дважды вычитенный
 </p>
 </div>
 </div>
);
};
```

---

ФАЙЛ 70/81: src/components/visualizers/SparseTableViz.t  
КАТЕГОРИЯ: Визуализаторы (вложенные) | СТРОК: 709 | РАЗМ:

---

```
import React, { useState } from "react";
import { motion, AnimatePresence } from "motion/react";
import {
 Play,
 RotateCcw,
 ChevronRight,
 ChevronLeft,
 Lock,
 ChevronDown,
} from "lucide-react";

// Data for 1D Array
const arr1D = [2, 3, 5, 62, 3, 21, 1, 4];
const N = arr1D.length;
const LOG = 4;

const st1D: number[][] = Array.from({ length: N }, () => []);
for (let i = 0; i < N; i++) st1D[i][0] = arr1D[i];
for (let j = 1; j < LOG; j++) {
 for (let i = 0; i + (1 << j) <= N; i++) {
 st1D[i][j] = Math.min(st1D[i][j - 1], st1D[i + (1 << j - 1)]);
 }
}
```

```

 ST2D[r][c][kx][ky-1],
 ST2D[r][c + (1 << (ky-1))][kx][ky-1]
);
 }
 }
}

```

```

export const SparseTableViz: React.FC = () => {
 const [tab, setTab] = useState<"1d" | "2d_build" | "2d_query"> "1d"

 return (
 <div className="w-full bg-slate-950 p-3 sm:p-4 md:p-5">
 <div className="flex gap-2 sm:gap-4 mb-5 border-b border-slate-800">
 <button
 onClick={() => setTab("1d")}
 className={`px-3 sm:px-4 py-2 rounded-lg text-slate-400 text-white" : "bg-slate-800 text-slate-400`}
 >
 1. Сворачивание 1D
 </button>
 <button
 onClick={() => setTab("2d_build")}
 className={`px-3 sm:px-4 py-2 rounded-lg text-slate-400 text-white" : "bg-slate-800 text-slate-400`}
 >
 2. Сворачивание 2D (Построение)
 </button>
 <button
 onClick={() => setTab("2d_query")}
 className={`px-3 sm:px-4 py-2 rounded-lg text-slate-400 text-white" : "bg-slate-800 text-slate-400`}
 >
 3. Запрос 2D (Формула)
 </button>
 </div>
 </div>
)
}

```

```

</div>
<div className="flex gap-2">
 <button
 onClick={() => setStep(Math.max(0, step - 1)}
 className="p-2 bg-slate-800 hover:bg-slate-700"
 disabled={step === 0}
 >
 <ChevronLeft size={20} />
 </button>
 <button
 onClick={() => setStep(Math.min(maxStep, step + 1)}
 className="p-2 bg-indigo-600 hover:bg-indigo-500"
 disabled={step === maxStep}
 >
 <ChevronRight size={20} />
 </button>
 <button
 onClick={() => setStep(0)}
 className="p-2 bg-slate-800 hover:bg-slate-700"
 >
 <RotateCcw size={20} />
 </button>
</div>
</div>

<div className="bg-slate-900 rounded-xl border border-slate-800" style={{ padding: 10px }}>
 <div className="text-[11px] uppercase tracking-wider font-mono">
 <div className="flex gap-1 sm:gap-1.5 overflow-x-hidden">
 {arr1D.map((v, idx) => {
 const inCover = !!covered && idx >= covered - 1;
 return (
 <div key={idx} className="flex flex-col items-center justify-center gap-1" style={{ width: 44px, height: clamp(26px, 8vw, 44px) }}>
 <div
 className={inCover ? "bg-indigo-500 text-white" : "bg-slate-800 text-slate-400"}
 style={{ padding: 2px 5px, border: 1px solid transparent }}
 <div
 className="text-[10px] font-mono"
 style={{ padding: 2px 5px, border: 1px solid transparent }}
 {v}
 </div>
);
 })}
 </div>
 </div>
</div>

```



```

 rap">
 2^0 (L=1)
 </div>
 <div className="text-center font-bold text-
 rap">
 2^1 (L=2)
 </div>
 <div className="text-center font-bold text-
 rap">
 2^2 (L=4)
 </div>
 <div className="text-center font-bold text-
 rap">
 2^3 (L=8)
 </div>

 {/* Matrix */}
 {Array.from({ length: N }).map((_, i) => {
 <React.Fragment key={i}>
 <div className="text-slate-500 font-mon
 {i}
 </div>
 {Array.from({ length: LOG }).map((_, j) => {
 const isValid = i + (1 << j) <= N;
 const isVisible =
 isValid &&
 (j === 0 ||
 computeSteps.findIndex((s) => s.i
 step));

 // Active computing state
 let isActive = false;
 let isSrc1 = false;
 let isSrc2 = false;

 if (step > 0 && step <= maxStep) {
 const currentStep = computeSteps[st
 if (currentStep.i === i && currentS

```

```

 ? "bg-emerald-500 text-white"
 : isSrc2
 ? "bg-amber-500 text-white"
 : isVisible
 ? "bg-slate-800 text-white"
 : "bg-transparent text-white"
)` }
 >
 {stlD[i][j]}
 </motion.div>
)}

{/* Arrows visualization */}
{(isSrc1 || isSrc2) && (
 <motion.svg
 initial={{ opacity: 0 }}
 animate={{ opacity: 1 }}
 className="absolute pointer-events-none"
 style={{
 width: 100,
 height: isSrc2 ? 40 + (1 <<
 right: -50,
 top: 24,
 overflow: "visible",
 }}
 >
 <path
 d={`M 0 0 C 30 0, 30 ${isSrc1 ? "0 0" : "0 40"}`}
 fill="none"
 stroke={isSrc1 ? "#10b981" : "#f59e00"}
 strokeWidth="3"
 />
 </motion.svg>
)}
</div>
);
}})
</React.Fragment>

```

```

 Нажмите далее, чтобы увидеть, как блоки сво
 </p>
 })
 </div>
</div>
);
};

```

```

const Viz2DBuild: React.FC = () => {
 const [k, setK] = useState(1);
 const [hover, setHover] = useState<{ r: number; c: number}>({ r: 0, c: 0 });
 const [locked, setLocked] = useState<{ r: number; c: number}>({ r: 0, c: 0 });

 const L = 1 << k;
 const activeCell = locked || hover;

 return (
 <div className="flex flex-col xl:flex-row gap-6">
 <div className="flex-1 min-w-0 bg-slate-900 p-3 scrollable">
 <h3 className="text-xl font-bold text-indigo-400">Визуализация
 <p className="text-sm text-slate-400 mb-6 text-wrap">
 Для наглядности показываем только <b className="text-rose-300">

 иц!
 </p>

 <div className="flex bg-slate-950 p-1 rounded-lg">
 {[0, 1, 2, 3].map(step => (
 <button
 key={step}
 onClick={() => { setK(step); setHover(null); setLocked(null); }}
 className={`px-4 py-2 rounded-md text-sm text-slate-400 hover:text-white hover:bg-slate-900`}
 >
 {step === 0 ? "Оригинал (1x1)" : `Шаг ${step + 1}`}
 </button>
))}
 </div>
 </div>
 </div>
);
};

```

```

 zIndex = 'z-10';
 } else {
 const prevL = 1 << (k - 1);
 const isTop = r < activeCell;
 const isLeft = c < activeCell;

 if (isTop && isLeft) {
 highlightClass = 'bg-rose';
 } else if (isTop && !isLeft) {
 highlightClass = 'bg-blue';
 } else if (!isTop && isLeft) {
 highlightClass = 'bg-emerald';
 } else {
 highlightClass = 'bg-amber';
 }
 zIndex = 'z-10';
 }
}

return (
 <div
 key={idx}
 onMouseEnter={() => { if(isCurr) setCurr(r, c); }}
 onMouseLeave={() => { if(isCurr) setCurr(r, c); }}
 onClick={() => {
 if(isCurr) {
 if(locked && locked === true) return;
 else setLocked({r, c});
 }
 }}
 className={`flex items-center justify-center gap-2`}
 style={{
 width: sCurr ? `w-[clamp(24px,6.5vw,38px)]` : `w-[clamp(20px,5.2vw,30px)]`,
 height: `h-[clamp(20px,5.2vw,30px)]`,
 text: `text-[clamp(8px,2vw,10px)]`,
 color: `color-[var(--color-text)]`,
 background: `background-color: ${highlightClass}`,
 border: `border-[1px solid var(--color-text)]`,
 opacity: `opacity-[0.8]`,
 transition: `transition-[background-color 0.3s, border-color 0.3s]`,
 }}
 />

```

```

e="text-amber-300">1], <span
 <span className="text-blue
ext-amber-300">1][k-<span className=
pan>

 <span className="text-emer
="text-amber-300">1][k-<span classNa
span>

 <span className="text-ambe
white">c + pL][k-<span className="te
ame="text-slate-500">// Правый Низ<b
 {'}}''));
</div>

```

```

{activeCell ? (
 <div className="bg-slate-950 p-5 rounded
.2)] mt-2 transition-all">
 <p className="text-slate-300 mb-3 t
 Пример для ячейки:
 {locked && <span className="text
ded border border-rose-500/20"><Lock size={
 </p>
 <p className="text-indigo-300 borde
 <span className="font-bold text-w
}}[k]][k]

 </p>

```

```

 <div className="space-y-2 pt-3 pl-2
 {((() => {
 const prevL = 1 << (k - 1);
 return (
 <>
 <div className="flex ite
 <span className="flex
hadow-[0_0_8px_#f43f5e]"></div> ST[{activeC
 <span className="text
tiveCell.r][activeCell.c][k-1][k-1]}
 </div>

```

```

 </div>
 </div>
 </div>
);
};

const Viz2DQuery: React.FC = () => {
 const [r1, setR1] = useState(1);
 const [c1, setC1] = useState(1);
 const [r2, setR2] = useState(5);
 const [c2, setC2] = useState(6);

 const H = r2 - r1 + 1;
 const W = c2 - c1 + 1;
 const kx = Math.floor(Math.log2(H));
 const ky = Math.floor(Math.log2(W));
 const Lx = 1 << kx;
 const Ly = 1 << ky;

 const matRows = 8 - Lx + 1;
 const matCols = 8 - Ly + 1;

 return (
 <div className="flex flex-col xl:flex-row gap-6" >
 <div className="flex-1 min-w-0 bg-slate-900" >
 <h3 className="text-xl font-bold text-rose-500" >Визуализация
 <p className="text-slate-400 text-[13px] font-normal" >
 еpx, r2, c2 - правый низ).</p>

 <div className="flex flex-col items-center justify-center" >
 <div className="grid grid-cols-[repeat(8, 1fr)] gap-2" >
 <div className="relative shadow-inner w-max max-w-full" >
 {Array.from({length: 64}).map((_, idx) => {
 const r = Math.floor(idx / 8);
 const c = idx % 8;
 const isInQuery = r >= r1 && r <= r2 && c >= c1 && c <= c2;

 return (

```

```

 <label className="flex items-center"
 <span className="text-slate-700"
 <div className="flex gap-2">
 <input type="number" min={0} max={100} value={x} />
14 bg-slate-900 border border-slate-700 py-1 px-1
 <input type="number" min={0} max={100} value={y} />
14 bg-slate-900 border border-slate-700 py-1 px-1
 </div>
 </label>
 <label className="flex items-center"
 <span className="text-slate-700"
 <div className="flex gap-2">
 <input type="number" min={0} max={100} value={x} />
14 bg-slate-900 border border-slate-700 py-1 px-1
 <input type="number" min={0} max={100} value={y} />
14 bg-slate-900 border border-slate-700 py-1 px-1
 </div>
 </label>
 </div>
 </div>
 </div>

 <div className="flex-1 bg-slate-900 p-6 rounded"
 <h3 className="text-xl font-bold text-emerald-400">Задача 1</h3>
 <p className="text-sm font-sans text-slate-300">
 Мы знаем, что максимальная степень дв
 rounded text-white">{Lx}. Аналогично
 te">{Ly}.
 </p>
 <p className="text-sm font-sans text-slate-300">
 Чтобы покрыть весь прямоугольник, мы
 Name="font-mono bg-[#0a0f1e] px-1 rounded border
 роса. (Цветные прямоугольники слева).
 </p>

 <div className="bg-[#0a0f1e] rounded-xl p-4"
 text-slate-300 shadow-inner overflow-x-auto
 <div className="absolute right-3 top-3"

```

```

border-slate-700 shadow-sm">ST[][][kx]][ky]
</h4>
<div className="grid gap-[2px] p-2.5
Columns: `repeat(${matCols}, 1fr)`}}>
 {Array.from({length: matRows * matCols}, (v, idx) => {
 const r = Math.floor(idx / matCols);
 const c = idx % matCols;
 let isTL = r === r1 && c === c1;
 let isTR = r === r1 && c === c2;
 let isBL = r === r2 - 1 && c === c1;
 let isBR = r === r2 - 1 && c === c2;

 let color = "bg-slate-800 text-white";
 let txt = ST2D[r][c][kx][ky];

 let badge = null;
 if (isTL) { color = "bg-rose-500 text-white";
 border-2"; badge = "TL"; }
 else if (isTR) { color = "bg-lime-500 text-white";
 ue-300 border-2"; badge = "TR"; }
 else if (isBL) { color = "bg-emerald-500 text-white";
 -emerald-300 border-2"; badge = "BL"; }
 else if (isBR) { color = "bg-amber-500 text-white";
 mber-300 border-2"; badge = "BR"; }

 return (
 <div key={idx} className={color}
 tion-all duration-300 relative shrink-0 ${c === c1 || c === c2 ? 'w-full' : 'w-[50%]'} h-[50px]
 {txt}
 {badge && <div className="bg-slate-700 text-white px-1 font-bold rounded
 </div>
)
 })
 }
</div>
</div>
</div>

```



```

function setMode(mode) {
 document.getElementById('btn-mode-normal').className =
 ? 'px-3 py-1 rounded text-sm font-bold'
 : 'px-3 py-1 rounded text-sm font-bold'

 document.getElementById('btn-mode-wtf').className =
 ? 'px-3 py-1 rounded text-sm font-bold'
 : 'px-3 py-1 rounded text-sm font-bold'

 const aside = document.querySelector('aside')
 const headerMain = document.querySelector('h1')
 const questionsContainer = document.getElementById('questions')
 const wtfContainer = document.getElementById('wtf')

 if (mode === 'wtf') {
 document.body.classList.add('wtf-mode')
 if (aside) aside.style.display = 'none'
 if (headerMain) headerMain.style.display = 'none'
 if (questionsContainer) questionsContainer.style.display = 'none'
 if (wtfContainer) wtfContainer.classList.remove('hidden')

 // Remove the URL hash to avoid scrolling
 history.pushState("", document.title, window.location.pathname + window.location.search)
 window.scrollTo(0, 0);
 } else {
 document.body.classList.remove('wtf-mode')
 if (aside) aside.style.display = '';
 if (headerMain) headerMain.style.display = '';
 if (questionsContainer) questionsContainer.style.display = '';
 if (wtfContainer) wtfContainer.classList.add('hidden')
 }
}

function setTheme(theme) {
 const body = document.body;
 body.classList.remove('theme-dark', 'theme-light');
 body.classList.add('theme-' + theme);
}

```

```

 setTimeout(() => drawerOverlay.classList.add("open"))
 }
 document.body.style.overflow = "hidden"
} else {
 mobileDrawer.classList.add("-translate-x-100")
 if (drawerOverlay) {
 drawerOverlay.classList.add("open")
 setTimeout(() => drawerOverlay.classList.add("open"))
 }
 document.body.style.overflow = ""
}
}

if (mobileMenuBtn && mobileDrawer) {
 mobileMenuBtn.addEventListener("click", () => {
 toggleDrawer()
 })
}
if (closeDrawerBtn && mobileDrawer) {
 closeDrawerBtn.addEventListener("click", () => {
 toggleDrawer()
 })
}
if (drawerOverlay) {
 drawerOverlay.addEventListener("click", () => {
 toggleDrawer()
 })
}

// Close on link click
document.querySelectorAll("#mobile-drawer a").forEach(link => {
 link.addEventListener("click", () => {
 if (mobileDrawer && !mobileDrawer.classList.contains("open")) {
 toggleDrawer()
 }
 })
})

const observerOptions = {
 root: null,
 rootMargin: '-10% 0px -70% 0px',
 threshold: 0
};

```

```

<script id="MathJax-script" async src="https://cdn.
<script>
 window.MathJax = {
 tex: {
 inlineMath: [['$', '$'], ['\\(', '\\)']]
 displayMath: [['$$', '$$'], ['\\[', '\\
 }
 };
</script>
<style>
 @import url('https://fonts.googleapis.com/css2?
 @reference "../src/index.css";
 .math-font { font-family: 'Fira Code', monospac
 .uno-card {
 width: 70px; height: 105px; border-radius:
 box-shadow: 0 4px 8px rgba(0,0,0,0.5); posi
 align-items: center; justify-content: cente
 color: white; overflow: hidden; flex-shrink
 text-shadow: 2px 2px 0 #000, -1px -1px 0 #0
 font-family: 'Arial Black', Impact, sans-se
 }
 .uno-card::before, .uno-card::after {
 content: attr(data-val); position: absolute
 text-shadow: 1px 1px 0 #000, -1px -1px 0 #0
 }
 .uno-card::before { top: 2px; left: 4px; }
 .uno-card::after { bottom: 2px; right: 4px; tra
 .uno-card.mini { width: 45px !important; height
 .uno-card.mini::before, .uno-card.mini::after {

/* Global formula scroll settings */
.formula-scroll {
 overflow-x: auto;
 overflow-y: hidden;
 -webkit-overflow-scrolling: touch;
 scrollbar-width: none; /* Firefox */
 -ms-overflow-style: none; /* IE and Edge */
}

```

```

.uno-inner {
 position: absolute; width: 110%; height: 75%;
 border-radius: 50%; transform: rotate(-30deg);
 box-shadow: inset 0 0 5px rgba(0,0,0,0.3);
}
.uno-val { position: relative; z-index: 2; }
.card-red { background-color: #e52521; } .card-
.card-blue { background-color: #004dcf; } .card-
.card-green { background-color: #339e35; } .card-
.card-yellow { background-color: #ffc400; } .ca
.card-black { background-color: #222; } .card-b
.card-black::before, .card-black::after { text-
.info-block { @apply bg-slate-800/80 border-l-4
.info-title { @apply font-bold mb-2 flex items-

/* Визуализация (Аналогии) */
.analogy-block { @apply bg-slate-900 border-2 b
 ems-center gap-6; }
.analogy-badge { @apply absolute -top-3 left-4 l
 king-widest border border-slate-500 shadow-md
.img-placeholder { @apply flex flex-col items-c
 r shrink-0 shadow-lg w-full md:w-auto; }
.img-caption { @apply font-mono text-sm mt-4 fo

/* THEME OVERRIDES */
body.theme-light {
 background-color: #f8fafc !important;
 color: #0f172a !important;
}
body.theme-light .bg-slate-900, body.theme-light
body.theme-light .bg-slate-800 { background-col
body.theme-light .bg-slate-700, body.theme-light
 important; }
body.theme-light .text-slate-200, body.theme-li
body.theme-light .text-slate-400 { color: #47556
body.theme-light .border-slate-600, body.theme-
body.theme-light .border-slate-500 { border-col
body.theme-light #top-controls { background-col

```

```

 }
 .creepy-emoji {
 display: inline-block;
 transform-origin: center;
 animation: creepy-blink 3.8s infinite;
 }
 </style>
</head>
<body class="bg-slate-900 text-slate-200 font-sans lead">

 <div class="sticky top-0 z-50 w-full bg-slate-950/90
 center items-center gap-4 sm:gap-8 transition-colors">
 <div class="flex items-center gap-3">

 <div class="bg-slate-800 p-1 rounded-lg flex items-center gap-2
 transition-colors">Список тем</div>
 <button onclick="setMode('normal')" id="btn-normal" class="bg-slate-800
 text-white hover:bg-slate-700 transition-colors">WTF
 </div>
 </div>
 <div class="flex items-center gap-3">

 <div class="bg-slate-800 p-1 rounded-lg flex items-center gap-2
 transition-colors">Темная</div>
 <button onclick="setTheme('dark')" id="btn-dark" class="bg-slate-800
 text-white hover:bg-slate-700 transition-colors">Light
 </div>
 <button onclick="setTheme('notebook')" id="btn-notebook" class="bg-slate-800
 text-white hover:bg-slate-700 transition-colors">Notebook
 </div>
 </div>
 </div>
 </div>

 <!-- ОПРЕДЕЛЕНИЕ МАРКЕРОВ SVG -->
 <svg width="0" height="0" class="absolute">
 <defs>
 <marker id="arrow-yellow" markerWidth="6" markerHeight="6" color="yellow"

```

```

 <line x1="2" y1="2" x2="38" y2="2" stro
 </g>
 <!-- МЯСОРУБКА -->
 <g id="meat-grinder">
 <path d="М 10 40 L 15 20 Q 25 15 35 20
 <rect x="5" y="40" width="40" height="2
 <!-- Ручка -->
 <path d="М 45 50 Q 60 50 60 30 L 75 30"
 <circle cx="75" cy="30" r="4" fill="#33
 <!-- Выход -->
 <path d="М 5 45 L -5 45 L -5 55 L 5 55
 <circle cx="25" cy="50" r="15" fill="#f
 </g>
 <g id="lego-green">
 <rect x="0" y="10" width="20" height="1
 <rect x="3" y="5" width="14" height="5"
 </g>
 <g id="lego-white">
 <rect x="0" y="0" width="40" height="20
 <rect x="6" y="-5" width="8" height="6"
 <rect x="26" y="-5" width="8" height="6
 <line x1="0" y1="2" x2="40" y2="2" stro
 <text x="20" y="14" fill="#475569" font
 </g>
</defs>
</svg>

<!-- Мобильная кнопка меню (Гамбургер) -->
<button id="mobile-menu-btn" class="lg:hidden fixed
 0/50 z-50 hover:bg-blue-500 transition-transform l
 <svg xmlns="http://www.w3.org/2000/svg" class="l
 <path stroke-linecap="round" stroke-linejoin
 </svg>
 </button>

<!-- Затемнение фона на мобилках при открытом меню
<div id="drawer-overlay" class="lg:hidden fixed ins

```

```

 <a href="#question-2" class=
er-indigo-500 pl-3 font-bold text-indigo-300">
 2. Комплексные числа

 </summary>
 <div class="pl-6 space-y-2 text-indigo-300">

 </div>
 </details>

 <details class="group">
 <summary class="list-none cursor-pointer text-teal-300">
 <a href="#question-3" class=
er-teal-500 pl-3 font-bold text-teal-300">
 3. Тригонометрическая форма

 </summary>
 <div class="pl-6 space-y-2 text-teal-300">

 </div>
</details>

 <details class="group">
 <summary class="list-none cursor-pointer text-amber-300">
 <a href="#question-4" class=
er-amber-500 pl-3 font-bold text-amber-300">
 4. Степени и корни

 </summary>
 <div class="pl-6 space-y-2 text-amber-300">


```

```
 <details class="group">
 <summary class="list-none cursor-pointer text-pink-500 pl-3 font-bold text-pink-300">
 7. Взаимно простые числа

 </summary>
 <div class="pl-6 space-y-2 text-pink-300">

 </div>
 </details>
```

```
 <details class="group">
 <summary class="list-none cursor-pointer text-violet-500 pl-3 font-bold text-violet-300">
 8. Сравнимость и Поля Виллиса

 </summary>
 <div class="pl-6 space-y-2 text-violet-300">

 </div>
 </details>
```

```
 <details class="group">
 <summary class="list-none cursor-pointer text-green-500 pl-3 font-bold text-green-400">
```



```
нственности НОД

ПИРАЛЬ)

вращается!)
 </div>
 </details>

 <details class="group">
 <summary class="list-none cursor-pointer text-blue-500 pl-3 font-bold text-blue-400">
 12. Взаимно простые мно

 </summary>
 <div class="pl-6 space-y-2 text-blue-500">

деление)
 </div>
 </details>

 <details class="group">
 <summary class="list-none cursor-pointer text-emerald-500 pl-3 font-bold text-emerald-400">
 13. Корни многочленов.

 </summary>
 <div class="pl-6 space-y-2 text-emerald-500">

 </div>
 </details>

 <details class="group">
```

```

<div class="pl-6 mt-2 space-y-2" data-bbox="474 95 1000 191">

</div>
</details>

<div class="mt-4 mb-2 text-xs font-l" data-bbox="407 266 1000 286">

-500 pl-3 text-slate-400">

-500 pl-3 text-slate-400">

ald-500 pl-3 text-slate-400">

-500 pl-3 text-slate-400">

go-500 pl-3 text-slate-400">

sia-500 pl-3 text-slate-400">

-red-500 pl-3 mt-4 text-red-500 font-bold ml-4" data-bbox="274 901 1000 921">
 ▲ Ликбез: Алгоритмы выживания


```

```

 a.style.fontWeight = 'normal';
 if (a.href.endsWith('#')) {
 a.style.fontSize = '1.2em';
 }
 });

 // Open the parent modal if the link is a details link
 const correspondingLink = details.correspondingLink;
 if (correspondingLink) {
 const details = document.querySelector(details.correspondingLink);
 if (details && details.open) {
 document.querySelector(details.correspondingLink).click();
 if (otherDetails) {
 otherDetails.close();
 }
 }
 }
 });
}

});
}, {
 root: null,
 rootMargin: '-10% 0px -70% 0px',
 threshold: [0, 0.5]
});

sections.forEach(section => {
 observer.observe(section);
});
});

function fallbackModal(contentRaw, ...args) {
 const modal = document.createElement('div');
 modal.style.position = 'fixed';
 modal.style.top = '0';
 modal.style.left = '0';
 modal.style.width = '100vw';

```

```

btnContainer.style.display = 'flex';
btnContainer.style.gap = '10px';
btnContainer.style.marginTop =

```

```

const copyBtn = document.createElement('button');
copyBtn.innerText = 'Копировать';
copyBtn.style.padding = '10px';
copyBtn.style.backgroundColor = '#333';
copyBtn.style.color = 'white';
copyBtn.style.border = 'none';
copyBtn.style.borderRadius = '5px';
copyBtn.style.cursor = 'pointer';
copyBtn.onclick = () => {
 if (navigator.clipboard) {
 navigator.clipboard.writeText(textarea.value);
 copyBtn.innerText = 'Скопировано';
 } else {
 textarea.select();
 document.execCommand('copy');
 copyBtn.innerText = 'Скопировано';
 }
 setTimeout(() => copyBtn.innerText = 'Копировать', 2000);
};

```

```

const closeBtn = document.createElement('button');
closeBtn.innerText = 'Закрыть';
closeBtn.style.padding = '10px';
closeBtn.style.backgroundColor = '#333';
closeBtn.style.color = 'white';
closeBtn.style.border = 'none';
closeBtn.style.borderRadius = '5px';
closeBtn.style.cursor = 'pointer';
closeBtn.onclick = () => modal.close();

```

```

box.appendChild(header);
box.appendChild(subHeader);
if (title !== 'PDF') {
 box.appendChild(textarea);
}

```

```
 link.download = 'vyisshaya_algebra.pdf';
 document.body.appendChild(link);
 link.click();
 document.body.removeChild(link);
 }

 function setTheme(theme) {

 const body = document.body;
 body.classList.remove('bg-slate-500');

 document.getElementById('theme-dark');
 document.getElementById('theme-light');
 document.getElementById('theme-slate');

 let oldStyle = document.getElementById('theme-style');
 if (oldStyle) oldStyle.remove();
 const style = document.createElement('style');
 style.id = 'theme-style';

 if (theme === 'dark') {
 body.classList.add('bg-slate-500');
 document.getElementById('theme-dark').checked = true;
 } else if (theme === 'light') {
 body.classList.add('bg-white');
 document.getElementById('theme-light').checked = true;
 style.innerHTML = `
 body.bg-white section h1 { color: #000000; }
 body.bg-white main { color: #000000; }
 body.bg-white main p { color: #000000; }
 /* Universal text color */
 body.bg-white .text-slate-500 { color: #000000; }
 body.bg-white .text-slate-400 { color: #000000; }
 body.bg-white .text-slate-300 { color: #000000; }
 body.bg-white [class*="text-slate"] { color: #000000; }
 `;
 }
 }
}
```

```

 body.bg-white .text-fuc
 body.bg-white .text-lim
 body.bg-white .text-blur
 body.bg-white .text-gre
 body.bg-white .text-ind

 `;
 } else if (theme === 'terminal')
 body.classList.add('bg-black')
 document.getElementById('theme')
 style.innerHTML = `
 body.bg-black .bg-slate-950 {
 background-color: t
 }
 body.bg-black section h1,
, body.bg-black button {
 color: #4ade80 !imp
 border-color: #22c5
 font-family: monosp
 }
 body.bg-black svg path,
 stroke: #22c55e !im
 fill: transparent !
 }
 `;
 }

 document.head.appendChild(style)
}
</script>

```

```

<!-- ФИКСИРОВАННЫЕ БЛОКИ: СМЕНА ТЕМЫ И ...
<div class="mt-8 flex flex-col gap-4 bo
 <!-- Кнопки Конвертации / Экспорта
 <div class="flex flex-col gap-2">

```

```

<!-- Основной контент -->
<main class="flex-1 bg-slate-800 p-4 sm:p-8 md:p-12 lg:p-16"
 -base min-w-0">
 <header class="text-center mb-12 border-b border-white pb-4">
 <h1 class="text-4xl md:text-5xl font-bl
4">
 ВЫСШАЯ АЛГЕБРА
 </h1>
 <p class="text-slate-400 italic text-lg">
 Высшая алгебра: теория и практика

 <!-- Print Table of Contents -->
 <div class="hidden print:block mt-8 text-center">
 <h2 class="font-bold text-xl mb-4">Содержание
 <ul class="text-base space-y-2">
 1. Алгебра
 2. Коммутативная алгебра
 3. Тренировки
 4. Структуры
 5. Группы
 6. Декартовы произведения
 7. Алгебраические расширения
 8. Клейм
 9. Коомология
 10. Алгебраические группы
 11. Алгебраические геометрии
 12. Алгебраические кривые
 13. Алгебраические поверхности
 14. Алгебраические многообразия
 15. Алгебраические функции
 17. Алгебраические структуры
 18. Алгебраические операции
 19. Алгебраические свойства
 20. Алгебраические методы
 21. Алгебраические приложения
 22. Алгебраические задачи


```

```

 <div class="font-bold text-slate-700">

 <a href="#question-10" onclick="setMainQuestion(10)"
- l-4 border-orange-500 hover:bg-slate-700 transition-colors
 <div class="text-xs text-orange-500">
 <div class="font-bold text-slate-700">

 <a href="#question-3" onclick="setMainQuestion(3)"
- 4 border-cyan-500 hover:bg-slate-700 transition-colors
 <div class="text-xs text-cyan-500">
 <div class="font-bold text-slate-700">

 <!-- Row 3: НОД и НОК / Алгоритм Евклида -->
 <a href="#question-7" onclick="setMainQuestion(7)"
- 4 border-yellow-500 hover:bg-slate-700 transition-colors
 <div class="text-xs text-yellow-500">
 <div class="font-bold text-slate-700">

 <a href="#question-12" onclick="setMainQuestion(12)"
- l-4 border-orange-500 hover:bg-slate-700 transition-colors
 <div class="text-xs text-orange-500">
 <div class="font-bold text-slate-700">

 <!-- Empty 3rd row cell -->
 <div class="hidden md:block"></div>

 <!-- Row 4: Факторизация / Структурная теория -->
 <a href="#question-8" onclick="setMainQuestion(8)"
- 4 border-yellow-500 hover:bg-slate-700 transition-colors
 <div class="text-xs text-yellow-500">
 <div class="font-bold text-slate-700">

 <a href="#question-11" onclick="setMainQuestion(11)"
- l-4 border-orange-500 hover:bg-slate-700 transition-colors
 <div class="text-xs text-orange-500">
 <div class="font-bold text-slate-700">


```



```

<div class="col-span-1 md:col-span-3" style="border: 2px solid #008000; padding: 10px; margin-bottom: 10px;">

 <div class="text-xs text-teal-500" style="margin-bottom: 5px;">Вопрос 17: Какое из следующих утверждений верно?
```

```

 <span class="text-w
 <span class="text-w
ово.

</div>

<!-- От противного -->
<div class="bg-slate-800/80 p-6
 <h3 class="text-xl text-fuc
 <p class="text-sm text-slat
 <ul class="list-disc pl-5 s
 <span class="text-w
i>
 <span class="text-w
т.д.).
 <span class="text-w
что так и задумано: "Ой, противоречие, теор

</div>

<!-- Единственность -->
<div class="bg-slate-800/80 p-6
 <h3 class="text-xl text-eme
 <p class="text-sm text-slat
 <ul class="list-disc pl-5 s
 <span class="text-w
, r_2$).
 <span class="text-w
 <span class="text-w
счет ограничений (размер остатка) докажи, ч

</div>

<!-- Индукция -->
<div class="bg-slate-800/80 p-6
 <h3 class="text-xl text-yel
 <p class="text-sm text-slat
 <ul class="list-disc pl-5 s

```

```
 <p class="text-xs font-mono">
 $$$. Пока получается 0 – корень жив. Как то.
 корня минус один.</p>
```

```
 </div>
```

```
 <div class="bg-slate-800 p-4">
 <h4 class="text-red-400">
 <p class="text-sm text-slate-400">
 <p class="text-xs font-mono">
 ициентов (они делятся на $$), кроме старшего
 зложить.</p>
```

```
 </div>
```

```
 <div class="bg-slate-800 p-4">
 <h4 class="text-red-400">
 <p class="text-sm text-slate-400">
 <p class="text-xs font-mono">
 . Чаще всего после этого Фейс-контроль $$.
```

```
 </div>
```

```
</div>
```

```
 <h2 class="text-2xl font-bold bg-white">
 (Правила экстрасенса)</h2>
```

```
 <p class="text-sm text-slate-400">
 тношения), а ты их не помнишь. Юзай базовые
 нейность) для делимости.</p>
```

```
 <ul class="space-y-4 pb-10">
```

```
 <li class="bg-slate-800/50 p-4">

 <div>
```

```
 <h4 class="text-cyan-400">
 <p class="text-sm text-slate-400">
 <code class="text-xs font-mono">
```

```
 </div>
```

```

```

---

## РАЗДЕЛ: УТИЛИТЫ

---

---

ФАЙЛ 73/81: src/utils/cn.ts

КАТЕГОРИЯ: Утилиты | СТРОК: 7 | РАЗМЕР: 169 В

---

```
import { clsx, type ClassValue } from "clsx";
import { twMerge } from "tailwind-merge";

export function cn(...inputs: ClassValue[]) {
 return twMerge(clsx(inputs));
}
```

---

ФАЙЛ 74/81: src/utils/exportHtml.ts

КАТЕГОРИЯ: Утилиты | СТРОК: 228 | РАЗМЕР: 9.8 KB

---

```
import type { Chapter } from "../types";
import { GLOSSARY_BY_ID } from "../data/glossary";
import { annotateTerms } from "../hooks/useTermHints";

/**
 * Выгрузка главы (или всего пособия) в самостоятельный
 *
 * Файл получается автономным: стили защиты внутрь, интерактив
 * его можно открыть с флешки, отправить в мессенджер и т.д.
 * Живые React-визуализации в статику не переносятся — вместо них
 * подставляется заметка со ссылкой на онлайн-версию, а комментарии
 * которые в приложении всплывают по наведению, собираются в блок
 * в конце документа.
 */
```

```

.export-foot { margin-top: 3rem; border-top: 1px solid #ccc; }
@media print {
 body { background: #fff; color: #0f172a; }
 .export-note, details { break-inside: avoid; }
 details { display: block; }
 details > div, details > p { display: block !important; }
 pre { white-space: pre-wrap; }
}
```

```
const escapeHtml = (s: string) =>
 s.replace(/&/g, "&").replace(/</g, "<").replace(/>/g, ">");
```

```
/**
```

```
 * Размечает термины и превращает их в якорные ссылки на глоссарий
```

```
 * Возвращает готовый HTML главы и список найденных терминов
```

```
 */
```

```
function prepareBody(html: string): { body: string; terms: string[] } {
 const host = document.createElement("div");
 host.innerHTML = html;
```

```
 try {
```

```
 annotateTerms(host);
```

```
 } catch {
```

```
 /* если браузер не осилил регулярку – выгружаем текст как есть */
```

```
 }
```

```
 const ids: string[] = [];
```

```
 host.querySelectorAll<HTMLElement>(".term-hint").forEach((el) => {
```

```
 const id = el.dataset.termId;
```

```
 if (!id) return;
```

```
 if (!ids.includes(id)) ids.push(id);
```

```
 const link = document.createElement("a");
```

```
 link.className = "term-hint";
```

```
 link.href = `#term-${id}`;
```

```
 link.title = GLOSSARY_BY_ID.get(id)?.short ?? "";
```

```
 link.textContent = el.textContent ?? "";
```

```
 el.replaceWith(link);
```



```

 termIds.forEach((t) => {
 if (!allTerms.includes(t)) allTerms.push(t);
 });
 bodies.push(`<div id="ch-`${escapeHtml(c.id)}`" style=
<p style="text-align:right;font-size:.72rem;margin:0 0 10px 0;"`
));

 const body = `<nav id="toc" style="background:#0f172a;
 rem">
 <h2 style="color:#fff;font-size:1.05rem;font-weight:800;"`
 <ol style="margin:0;padding-left:1.1rem;font-size:.85rem;"`
 </nav>
 `${bodies.join("\n")}`;

 const html = wrap({
 title: "Универсальное пособие по алгоритмам",
 subtitle: `Все темы, ${chapters.length} шт.`,
 css,
 body,
 gloss: glossarySection(allTerms),
 origin: window.location.origin,
 });
 triggerDownload(html, "posobie-vse-temy.html");
 }

```

## РАЗДЕЛ: СКРИПТЫ СБОРКИ И ЭКСПОРТА

ФАЙЛ 75/81: scripts/audit-coverage.mjs

КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРОК: 83 | РАЗМЕР: 1.5 КБ

```
/**
```

- \* Аудит покрытия глав пособия: у каких тем нет «объяснения»
- \* (блок «Аналогия») и/или 2D-визуализации (интерактивные)

```

 analogies,
 hasAnalogy: analogies > 0,
 inlineSvg: /<svg[\s>]/i.test(html),
 image: /<img[\s>]/i.test(html),
 interactive: vizIds.has(c.id),
 chars: html.length,
 });
});

const has2D = (r) => r.interactive || r.inlineSvg || r.image;
const gapBoth = rows.filter((r) => !r.hasAnalogy && !has2D(r));
const gapAnalogy = rows.filter((r) => !r.hasAnalogy && has2D(r));
const gapViz = rows.filter((r) => r.hasAnalogy && !has2D(r));
const orphanViz = [...vizIds].filter((id) => !chapters.has(id));

const nums = rows.map((r) => r.num).filter(Boolean);
const missingNums = Array.from({ length: 24 }, (_, i) => i + 1).filter(n => !nums.includes(n));

const flag = (b) => (b ? " " : "-");

if (process.argv.includes("--md")) {
 console.log("| # | Глава | Аналогия | 2D-виз. | SVG |");
 console.log("| --- | --- | --- | --- | --- |");
 for (const r of rows) {
 console.log(
 `| ${r.num || "-"} | ${r.title} | ${r.hasAnalogy ? "да" : "нет"} |`
);
 }
} else {
 console.log(`Глав в пособии: ${rows.length}\n`);
 const dump = (title, list) => {
 console.log(`${title} (${list.length}):`);
 list.forEach((r) => console.log(`  * ${r.title} [${r.num}]`));
 console.log();
 };
 dump("НЕТ НИ АНАЛОГИИ, НИ 2D", gapBoth);
 dump("НЕТ АНАЛОГИИ (2D есть)", gapAnalogy);
}

```



```
// те же лимиты, что и в src/hooks/useTermHints.ts
const MAX_PER_SURFACE = 2;
const MAX_PER_TERM = 8;

const bad = [];
const used = new Set();
console.log("тем | подсв. | глава");
for (const c of chapters) {
 const text = c.content
 .replace(/<(script|style|code|pre)[\s\S]*?<\/\1>/gi, "")
 .replace(/<[>]+>/g, " ");

 const perTerm = new Map();
 const perSurface = new Map();
 let highlights = 0;
 const ids = new Set();

 for (const m of text.matchAll(re)) {
 const surface = m[1].toLowerCase();
 const id = map.get(surface);
 if (!id) continue;
 const ut = perTerm.get(id) ?? 0;
 const us = perSurface.get(surface) ?? 0;
 if (ut >= MAX_PER_TERM || us >= MAX_PER_SURFACE) continue;
 perTerm.set(id, ut + 1);
 perSurface.set(surface, us + 1);
 highlights += 1;
 ids.add(id);
 used.add(id);
 }

 if (ids.size === 0) bad.push(c.title);
 console.log(
 String(ids.size).padStart(3), "|", String(highlights).padStart(3),
 c.title.slice(0, 46).padEnd(47), [...ids].slice(0, 10).join(", ")
);
}
console.log("\nГлав без единой подсказки:", bad.length,
```

```
 size: [A4.width, A4.height],
 margin: 0,
 autoFirstPage: false,
 info: {
 Title: "Универсальное пособие – полный исходный код",
 Author: "CI: rebuild-pdf",
 Subject: "Экспорт всего кода проекта (код -> txt)",
 CreationDate: new Date(),
 },
 });

const stream = fs.createWriteStream(PDF_PATH);
doc.pipe(stream);

for (const file of pages) {
 doc.addPage({ size: [A4.width, A4.height], margin: 0 });
 doc.image(path.join(PAGES_DIR, file), 0, 0, { width: A4.width });
}

doc.end();
await new Promise((resolve, reject) => {
 stream.on("finish", resolve);
 stream.on("error", reject);
});

console.log("[3/3] png -> pdf");
console.log(`    страниц: ${pages.length}`);
console.log(`    выход:   ${path.relative(ROOT, PDF_PATH)}`);
}

main().catch((err) => {
 console.error("Ошибка сборки PDF:", err.message);
 process.exit(1);
});
```

```
[/^[^/]+$/, "Конфигурация проекта"],
];

const categoryOf = (rel) => CATEGORIES.find(([re]) => re.test(rel));

/** Порядок разделов в документе. */
const ORDER = [
 "Конфигурация проекта",
 "Ядро приложения",
 "Контент и данные пособия",
 "Билеты (учебный контент)",
 "Интерактивные компоненты и симуляторы",
 "Визуализаторы (вложенные)",
 "HTML-разметка (layout)",
 "Утилиты",
 "Скрипты сборки и экспорта",
 "CI / автоматизация",
 "Прочее",
];

function listFiles() {
 let files;
 try {
 files = execFileSync("git", ["ls-files", "--cached"], {
 cwd: ROOT,
 encoding: "utf8",
 })
 .split("\n")
 .filter(Boolean);
 } catch {
 files = [];
 const walk = (dir) => {
 for (const e of fs.readdirSync(dir, { withFileTypes: true })) {
 const abs = path.join(dir, e.name);
 const rel = path.relative(ROOT, abs).split(path.sep);
 if (EXCLUDE_DIRS.some((d) => rel === d || rel.startsWith(d + path.sep))) continue;
 if (e.isDirectory()) walk(abs);
 else files.push(rel);
 }
 };
 walk(ROOT);
 }
}
```

```
function main() {
 const files = listFiles();
 const chapters = extractChapters(files);
 ensureDir(OUT_DIR);

 const out = [];
 const push = (s = "") => out.push(s);

 const stamp = new Date().toLocaleString("ru-RU", { time: true });
 const totalBytes = files.reduce((s, f) => s + fs.statSync(f).size, 0);

 push(HR);
 push(" УНИВЕРСАЛЬНОЕ ПОСОБИЕ ПО АЛГОРИТМАМ И СТРУКТУРАМ");
 push(" ПОЛНЫЙ ИСХОДНЫЙ КОД ПРОЕКТА (ВСЕ ФАЙЛЫ И КОД)");
 push(HR);
 push();
 push(`Дата генерации:                ${stamp}`);
 push(`Файлов исходного кода:         ${files.length}`);
 push(`Суммарный объём кода:            ${humanSize(totalBytes)}`);
 push(`Глав/билетов в пособии:         ${chapters.length}`);
 push();

 push(HR);
 push(" ОГЛАВЛЕНИЕ: ГЛАВЫ И БИЛЕТЫ");
 push(HR);
 chapters.forEach((c, i) => {
 push(`  ${String(i + 1).padStart(3, " ")}. ${c.title}`);
 });
 push();

 push(HR);
 push(" ОГЛАВЛЕНИЕ: ФАЙЛЫ ИСХОДНОГО КОДА");
 push(HR);
 let current = null;
 files.forEach((rel, i) => {
 const cat = categoryOf(rel);
 if (cat !== current) {
 push(`  ${String(i + 1).padStart(3, " ")}. ${cat}`);
 current = cat;
 }
 });
}
```

```
 console.log("[1/3] код -> txt");
 console.log(`    файлов:  ${files.length}`);
 console.log(`    глав:    ${chapters.length}`);
 console.log(`    строк:   ${txt.split("\n").length}`);
 console.log(`    выход:   ${path.relative(ROOT, TXT_PATH)}`);
 }

 main();
```

---

ФАЙЛ 79/81: scripts/export/common.mjs

КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРОК: 66 | РАЗМЕР: 1.2 КБ

---

```
/**
 * Общие настройки и утилиты пайплайна экспорта.
 */
* код -> full_code_all_pages.txt (collect-code.mjs)
* txt -> page-XXXX.png (render-pages.mjs)
* png -> full_code_all_pages.pdf (build-pdf.mjs)
*/
import fs from "node:fs";
import path from "node:path";
import { fileURLToPath } from "node:url";
import { execFileSync } from "node:child_process";

export const ROOT = path.resolve(path.dirname(fileURLToPath(import.meta.url)), "..");

/** Куда кладём финальные артефакты (эта папка отдаётся клиенту) */
export const OUT_DIR = path.join(ROOT, "public", "export");

/** Промежуточные PNG-страницы (в .gitignore, в CI уезжают) */
export const PAGES_DIR = path.join(ROOT, "tmp", "export");

export const TXT_PATH = path.join(OUT_DIR, "full_code_all_pages.txt");
export const PDF_PATH = path.join(OUT_DIR, "full_code_all_pages.pdf");

/** Параметры вёрстки страницы (подобраны под A4 @150dpi) */
```

```

 throw new Error(
 "ImageMagick не найден. Установите его: sudo apt-get
);
}
```

ФАЙЛ 80/81: scripts/export/render-pages.mjs

КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРОК: 117 | РАЗМ:

```

/**
 * ШАГ 2: TXT -> PNG
 *
 * Разбивает full_code_all_pages.txt на страницы формата
 * каждую страницу в PNG через ImageMagick (шрифт DejaVu)
 *
 * Результат: tmp/export-pages/page-0001.png, page-0002
 */
import fs from "node:fs";
import path from "node:path";
import os from "node:os";
import { execFile } from "node:child_process";
import { promisify } from "node:util";
import {
 ROOT, PAGES_DIR, TXT_PATH, PAGE, ensureDir, humanSize
} from "../common.mjs";

const execFileAsync = promisify(execFile);

/** Раскрываем табы и жёстко переносим длинные строки,
function wrap(line) {
 const expanded = line.replace(/\t/g, " ").replace(
 if (expanded.length <= PAGE.cols) return [expanded];
 const parts = [];
 const indent = " ".repeat(Math.min((expanded.match(/^
 let rest = expanded;
 let first = true;
```

```

 fs.writeFileSync(txtFile, body + "\n", "utf8");
 return { txtFile, pngFile, num };
 });

const args = (job) => [
 "-density", String(PAGE.density),
 "-page", PAGE.size,
 "-font", fs.existsSync(PAGE.fontFile) ? PAGE.fontFile : "",
 "-pointsize", String(PAGE.pointsized),
 `text:${job.txtFile}`,
 "-background", "white",
 "-flatten",
 "-colorspace", "Gray",
 ...(PAGE.monochrome ? ["-monochrome"] : []),
 "-strip",
 "-define", "png:compression-level=9",
 job.pngFile,
];

const concurrency = Math.max(2, Math.min(os.cpus().length, 4));
let done = 0;
let cursor = 0;

async function worker() {
 while (cursor < jobs.length) {
 const job = jobs[cursor++];
 await execFileAsync(im, args(job));
 fs.rmSync(job.txtFile, { force: true });
 done += 1;
 if (done % 25 === 0 || done === total) {
 process.stdout.write(`отрендерено ${done} страниц\n`);
 }
 }
}

await Promise.all(Array.from({ length: concurrency }, worker));
process.stdout.write("\n");

```

```
push:
 branches:
 - "**"
 paths-ignore:
 # чтобы коммит самого workflow не запускал бесконечный цикл
 - "public/export/**"
 - "**/*.md"
workflow_dispatch:
 inputs:
 density:
 description: "DPI растеризации страниц (по умолчанию 300)"
 required: false
 default: "150"

concurrency:
 group: rebuild-pdf-${{ github.ref }}
 cancel-in-progress: true

permissions:
 contents: write

jobs:
 rebuild-pdf:
 name: код → txt → png → pdf
 runs-on: ubuntu-latest
 # не реагируем на собственный коммит бота
 if: "!contains(github.event.head_commit.message, '[bot]')"
 steps:
 - name: Checkout
 uses: actions/checkout@v4
 with:
 fetch-depth: 0
 persist-credentials: true

 - name: Setup Node.js
 uses: actions/setup-node@v4
 with:
```



```
 echo ""
 echo "| Артефакт | Размер |"
 echo "| --- | --- |"
 echo "| \`public/export/full_code_all_pages"
 echo "| \`public/export/full_code_all_pages"
 echo "| PNG-страниц | ${ls tmp/export-pages.
 } >> "$GITHUB_STEP_SUMMARY"

- name: Upload artifacts (PDF + TXT)
 uses: actions/upload-artifact@v4
 with:
 name: full-code-export
 path: |
 public/export/full_code_all_pages.pdf
 public/export/full_code_all_pages.txt
 if-no-files-found: error
 retention-days: 30

- name: Upload artifacts (PNG-страницы)
 uses: actions/upload-artifact@v4
 with:
 name: full-code-pages-png
 path: tmp/export-pages/*.png
 if-no-files-found: error
 retention-days: 7

- name: Commit rebuilt export back to the branch
 run: |
 git config user.name "github-actions[bot]"
 git config user.email "41898282+github-action
 git add -- public/export/full_code_all_pages.
 if git diff --cached --quiet; then
 echo "Экспорт не изменился — коммит не нужен"
 exit 0
 fi
 git commit -m "chore(export): пересборка full
 git push origin "HEAD:${GITHUB_REF_NAME}"
```